BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

ĐỀ TÀI

PLANT CARE – ỨNG DỤNG CHĂM SÓC CÂY TRỒNG THÔNG MINH

GVHD: ThS. Huỳnh Hoàng Hà

Sinh viên thực hiện:

1.	Nguyễn Tiến Dũng	23119133
2.	Phan Lê Anh Kiệt	23119158
3.	Hà Nguyễn Duy Linh	23119161
4.	Lê Tiến Phúc	23119192
5.	Nguyễn Hùng Vỹ	23119231

TP. Hồ Chí Minh, tháng 10 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

Link video: https://youtu.be/b8DZ_coRoGY

Link Github: https://github.com/Kitlus05/MobileAppPlantcare

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN 1
1.1. Đặt vấn đề 1
1.2. Mục tiêu của đề tài 1
1.3. Đối tượng và phạm vi nghiên cứu 2
1.4. Phương pháp nghiên cứu 2
1.5. Phân công thực hiện 2
1.6. Bố cục báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3
2.1. Tổng quan hệ thống IoT trong nông nghiệp thông minh 3
2.2. React Native và Expo – Nền tảng phát triển ứng dụng đa nền tảng 3
2.2.1. Expo – Bộ công cụ hỗ trợ React Native 3
2.2.2. Cách React Native hoạt động 3
2.3. Cấu trúc ứng dụng và quản lý trạng thái với React và Context API 4
2.3.1. Component-based – Tư duy xây dựng giao diện theo thành phần 4
2.3.2. React Hooks – Công cụ quản lý trạng thái trong Function Component 4
2.3.3. Context API – Chia sẻ trạng thái xuyên màn hình 4
2.4. Firebase – Nền tảng backend không cần máy chủ riêng 5
2.4.1. Firebase Authentication 5
2.4.2. Firebase Realtime Database 5
2.5. React Navigation – Điều hướng trong ứng dụng 6
2.6. TensorFlow Lite và nhận diện bệnh cây on-device 6
2.7. ESP32 và vai trò trong hệ thống 7
2.4.4. Xác thực Ẩn danh (Firebase Anonymous Auth) và Lưu trữ Token trong NVS Flash 7
2.7.6. Kiến trúc Firmware FreeRTOS Đa nhân, Mutex và Ngắt phần cứng (Hardware ISR) 9
2.6.6. Quy trình Huấn luyện Mô hình Học sâu và Lượng tử hóa sau huấn luyện (PTQ) 11
2.7.7. Sơ đồ Ghép nối GPIO và Thiết kế Cách ly Nguồn Điện lực 13
2.1.3. Phân tích đối sánh các chuẩn truyền thông không dây IoT 14
2.6.7. Cơ sở Toán học của MobileNetV2 và Hàm kích hoạt ReLU6 15
2.7.8. Cơ sở Vật lý Cảm biến Điện dung và Thuật toán Calibration 17
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG 18

3.1. Phân tích yêu cầu ứng dụng....18
3.1.1. Tác nhân và phạm vi hệ thống....18
3.1.2. Yêu cầu chức năng (Functional Requirements)....19
3.1.3. Yêu cầu phi chức năng (Non-Functional Requirements)....20
3.2. Kiến trúc tổng thể hệ thống (Edge – Cloud – Mobile Architecture)....21
3.2.1. Mô hình trung gian hướng đám mây (Cloud-based Intermediary Pattern)....21
3.2.2. So sánh kỹ thuật: Firebase RTDB vs. MQTT Broker tự dựng vs. REST Polling....22
3.2.3. Luồng dữ liệu hai chiều: Uplink Telemetry và Downlink Command....23
3.2.4. Giao thức Tự khám phá và Tự ghép thiết bị Zero-Touch (Auto-Discovery & Pairing)....25
3.3. Kiến trúc phần mềm ứng dụng di động (Mobile App Architecture)....27
3.3.1. Mô hình tổ chức mã nguồn Screen-First....27
3.3.2. Cây phân cấp Component (Component Hierarchy)....27
3.3.3. Mẫu thiết kế Component dùng chung (Component Composition Pattern)....28
3.3.4. Quản lý kiểu dáng: StyleSheet.create vs. Inline Styles....30
3.3.5. Điểm khởi nhập và Bộ khung ứng dụng (App.js)....30
3.3.6. Tầng tiện ích và Chuẩn hóa dữ liệu (Utils Layer Architecture)....30
3.4. Quản lý trạng thái và luồng dữ liệu thời gian thực (State Management)....33
3.4.1. Vòng đời Listener Firebase và Mẫu thiết kế Unsubscribe Pattern....33
3.4.2. Chiến lược Đa Listener độc lập (Multiple Independent Listeners Pattern) 34
3.4.3. Đồng bộ trạng thái và Xử lý Race Condition với Functional State Update 34
3.4.4. Máy trạng thái giao diện của HomeScreen....35
3.4.5. Thuật toán Ghép điểm dữ liệu Realtime (Real-time Point Splicing) và Lọc nhiễu cảm biến....36
3.5. Cấu trúc thư mục mã nguồn và Phân tích Phụ thuộc Module....38
3.5.1. Bảng phân tích chi tiết từng file mã nguồn....38
3.5.2. Biểu đồ phụ thuộc Module (Module Dependency Graph)....39
3.6. Tích hợp AI nhận diện bệnh cây (TFLite on-device)....40
3.6.1. Đóng gói mô hình trong Expo Managed Workflow (Asset Bundling)....40
3.6.2. Quản lý bộ nhớ và Vòng đời nạp mô hình (Model Lifecycle)....40
3.6.3. Chi tiết toán học và giải thuật Tiền xử lý ảnh (Image Preprocessing Pipeline)....41
3.6.4. Phân tích ngưỡng phân loại (Confidence Threshold Trade-off)....42
3.7. Thiết kế cơ sở dữ liệu Firebase Realtime Database....44

3.7.1. Chiến lược Phi chuẩn hóa dữ liệu NoSQL (Data Denormalization Strategy) 44
3.7.2. Tối ưu hóa truy vấn: Cấu hình Indexing `.indexOn` và Phân trang `limitToLast` 45
3.7.3. Cấu trúc Push ID của Firebase 45
3.7.4. Phân tích chi tiết Thiết kế Security Rules 45
3.8. Thiết kế giao diện (UI/UX) và Biểu đồ tuần tự 47
3.8.1. Hệ thống thiết kế (Design System & UI Tokens) 47
3.8.2. Bố cục thích ứng (Responsive Layout) với Flexbox và SafeAreaView 49
3.8.3. Biểu đồ tuần tự điều khiển bơm và Xử lý sự cố mạng 49
3.8.4. Hệ thống Thông báo cục bộ và Cơ chế Giới hạn tần suất cảnh báo (Cooldown Rate-Limiting) 49
3.3.7. Phân tích Chuyên sâu Mã nguồn Từng Màn hình (Screen-by-Screen Deep Dive) 53
3.4.6. Quản lý Vòng đời Ứng dụng Di động (Mobile App Lifecycle với AppState) 55
3.5.3. Quy trình Đóng gói và Phát hành Bản dựng APK Độc lập (EAS Build Pipeline) 56
3.1.4. Đặc tả Kịch bản Sử dụng Chi tiết (Use Case Specifications) 57
3.2.5. Cơ chế Bắt tay Mã hóa TLS 1.2 và Tối ưu hóa Bộ đệm RAM trên ESP32 58
3.6.5. Kỹ thuật Ánh xạ Bộ nhớ Trực tiếp (Memory Mapping – mmap) cho Mô hình TFLite 59
CHƯƠNG 4. KẾT QUẢ VÀ THỬ NGHIỆM THỰC NGHIỆM 60
4.1. Môi trường và Thiết bị thử nghiệm 60
4.2. Kết quả giao diện ứng dụng trên thiết bị thật 61
4.2.1. Màn hình Xác thực và Đăng nhập 61
4.2.2. Màn hình Trang chủ và Giám sát vi khí hậu thời gian thực 62
4.2.3. Màn hình Chi tiết cây trồng và Điều khiển tưới từ xa 62
4.2.4. Màn hình Chẩn đoán bệnh lá cây bằng AI on-device 63
4.2.5. Màn hình Lịch sử và Cài đặt hệ thống 64
4.3. Đánh giá hiệu năng hệ thống (Quantitative Performance Evaluation) 65
4.3.1. Phân rã độ trễ dữ liệu End-to-End (Latency Decomposition Analysis) 65
4.3.2. Đánh giá tốc độ khung hình (FPS) và Tài nguyên bộ nhớ 67
4.3.3. Đánh giá hiệu năng Kiến trúc Đa nhân FreeRTOS trên ESP32 68
4.3.4. Đánh giá tốc độ suy luận mô hình AI (TFLite Inference Benchmark) 68

4.3.5. Số liệu Thực nghiệm Huấn luyện Mô hình AI 69
4.3.6. Đánh giá bản dựng APK Độc lập và Mức tiêu thụ Năng lượng 74
4.2.6. Đánh giá Trải nghiệm Người dùng theo 10 Tiêu chuẩn Heuristic 74
4.3.7. Phân tích Dòng điện tiêu thụ, Năng lượng và Độ chính xác Cảm biến 75
4.3.8. Kiểm thử Độ bền và Độ tin cậy Liên tục 168 Giờ (Endurance Stress Test) 77
4.4. Nhận xét và đánh giá chung 77
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 79
5.1. Kết quả đạt được 79
5.2. Hạn chế còn tồn tại 79
5.3. Hướng phát triển 79
TÀI LIỆU THAM KHẢO 81

DANH MỤC HÌNH ẢNH

Hình 2.1. Cơ chế cấp phát và duy trì danh tính thiết bị qua Firebase Anonymous Auth và NVS Flash....8
Hình 2.2. Kiến trúc phân chia đa nhân FreeRTOS: Core 0 (Network Task) và Core 1 (Sensor & LCD Loop) qua Mutex....10
Hình 2.3. Pipeline huấn luyện Transfer Learning, Tăng cường dữ liệu và Lượng tử hóa mô hình TFLite....12
Hình 2.4. Sơ đồ nguyên lý cách ly nguồn điện lực qua Optocoupler và Diode chống xung ngược Back-EMF....14
Hình 3.1. Biểu đồ Use Case tổng thể của ứng dụng Plant Care phân rã theo 4 nhóm chức năng....18
Hình 3.2. Sơ đồ kiến trúc tổng thể 3 lớp: Edge (ESP32) – Cloud (Firebase) – Mobile (React Native Expo)....22
Hình 3.3. Giao thức Tự khám phá và Ghép nối thiết bị nguyên tử (Zero-Touch Discovery & Atomic Pairing)....24
Hình 3.4. Sơ đồ cây phân cấp Component React Native....26
Hình 3.5. Sơ đồ tích hợp: Quản lý Đa Listener 2 cấp....28
Hình 3.11. Cấu trúc cây dữ liệu của Firebase Realtime Database và mối quan hệ logic....47
Hình 3.12. Hệ thống Design System....48
Hình 3.14. Biểu đồ tuần tự chi tiết: Điều khiển tưới nước từ xa với cơ chế xác nhận ACK và xử lý Timeout....50
Hình 3.16. Quy trình Đóng gói bản dựng độc lập qua EAS Build: Source Code → Keystore Signature → R8 Shrinking → Standalone APK....57
Hình 4.1. Giao diện màn hình Đăng nhập (Login) và Đăng ký tài khoản trên thiết bị thật....62
Hình 4.2. Danh sách cây và Lưới thẻ cảm biến cập nhật liên tục....62
Hình 4.3. Giao diện màn hình Chi tiết cây trồng và Luồng bấm nút kích hoạt bơm tưới....63
Hình 4.5. Giao diện màn hình Lịch sử số đo cảm biến và Màn hình Cài đặt tài khoản65
Hình 4.6. Biểu đồ Timeline phân rã các chặng thời gian của độ trễ truyền dữ liệu End-to-End....67
Hình 4.7. Đồ thị diễn biến hàm mất mát (Loss) và độ chính xác (Accuracy) qua 25 Epochs huấn luyện....70

DANH MỤC BẢNG BIỂU

Bảng 1.1. Bảng phân công công việc của nhóm 2
Bảng 2.1. So sánh các giải pháp quản lý trạng thái trong React Native 4
Bảng 2.2. So sánh suy luận trên thiết bị và gọi API đám mây 6
Bảng 2.3. Bảng phân bổ chân GPIO trên kit vi điều khiển ESP32 DevKit V1 13
Bảng 2.4. So sánh các chuẩn truyền thông không dây trong ứng dụng IoT 15
Bảng 3.1. Danh mục yêu cầu chức năng của hệ thống Plant Care 19
Bảng 3.2. Danh mục yêu cầu phi chức năng của hệ thống Plant Care 20
Bảng 3.3. So sánh chi tiết các kiến trúc truyền thông cho hệ thống IoT di động.....22
Bảng 3.4. Bảng phân tích trách nhiệm và giao tiếp của từng tệp tin trong dự án 38
Bảng 3.5. Quy trình tiền xử lý toán học và suy luận ảnh nhận diện bệnh cây 41
Bảng 3.6. Hệ thống Design Tokens và quy chuẩn thiết kế UI/UX ứng dụng Plant Care 47
Bảng 3.7. Bảng phân tích từng bước trong biểu đồ tuần tự điều khiển tưới 52
Bảng 3.8. Bảng đặc tả kịch bản Use Case cho hai chức năng cốt lõi của ứng dụng 57
Bảng 4.1. Bảng thông số môi trường phần cứng và phần mềm thử nghiệm 60
Bảng 4.2. Bảng phân rã chi tiết các thành phần độ trễ truyền dữ liệu End-to-End 66
Bảng 4.3. Kết quả đo đạc tốc độ khung hình (FPS) và dung lượng RAM chiếm dụng 67
Bảng 4.4. So sánh hiệu năng thực nghiệm giữa Kiến trúc đơn luồng cũ và Đa nhân FreeRTOS mới.....68
Bảng 4.5. Kết quả đo thời gian suy luận AI trên hai cấu hình chip di động khác nhau 69
Bảng 4.6. Diễn biến hàm mất mát (Loss) và độ chính xác (Accuracy) qua các Epochs 69
Bảng 4.7. Bảng đánh giá độ đo phân loại Precision, Recall và F1-Score của mô hình AI 70
Bảng 4.8. So sánh dung lượng và tốc độ suy luận giữa các phiên bản mô hình AI 73
Bảng 4.9. Đánh giá tối ưu hóa kích thước bản dựng APK và hiệu quả tiết kiệm pin nền 74
Bảng 4.10. Bảng đánh giá mức độ tuân thủ tiêu chuẩn thiết kế Heuristic 74
Bảng 4.11. Bảng phân tích dòng điện và công suất tiêu thụ của hệ thống phần cứng IoT 75
Bảng 7.1. Danh mục các lớp bệnh cây tiêu biểu và phác đồ điều trị khuyến nghị **Error! Bookmark not defined.**

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Nghĩa đầy đủ	Diễn giải
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLoC	Business Logic Component	Thành phần xử lý logic nghiệp vụ
DI	Dependency Injection	Tiêm phụ thuộc
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
FR / NFR	Functional / Non-functional Requirement	Yêu cầu chức năng / phi chức năng
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản
MVVM	Model – View – ViewModel	Mẫu kiến trúc trình bày dữ liệu
RTDB	Firebase Realtime Database	Cơ sở dữ liệu thời gian thực
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
TFLite	TensorFlow Lite	Thư viện suy luận trên thiết bị
UDF	Unidirectional Data Flow	Luồng dữ liệu một chiều
UI / UX	User Interface / User Experience	Giao diện / Trải nghiệm người dùng

CHƯƠNG 1. TỔNG QUAN

1.1. Đặt vấn đề

Chăm sóc cây trồng đúng cách đòi hỏi người dùng phải theo dõi liên tục nhiều yếu tố như độ ẩm đất, nhiệt độ, độ ẩm không khí và cường độ ánh sáng. Với nhịp sống đô thị, phần lớn người trồng cây tại nhà hoặc ban công thường quên tưới nước, tưới sai lượng hoặc phát hiện bệnh trên lá quá muộn. Hệ quả là cây phát triển kém, thậm chí chết, gây lãng phí cả công sức lẫn chi phí.

Sự phát triển của vi điều khiển tích hợp Wi-Fi giá rẻ (ESP32), các nền tảng đám mây thời gian thực (Firebase) và khả năng suy luận học máy ngay trên điện thoại (TensorFlow Lite) mở ra hướng giải quyết: xây dựng một ứng dụng di động đóng vai trò trung tâm, vừa hiển thị dữ liệu cảm biến theo thời gian thực, vừa điều khiển bơm tưới từ xa, vừa nhận diện bệnh lá cây bằng camera mà không cần kết nối mạng. Từ nhu cầu đó, nhóm chọn đề tài "Plant Care – Ứng dụng chăm sóc cây trồng thông minh".

Cần nhấn mạnh rằng trọng tâm của đồ án thuộc môn Phát triển Ứng dụng Di động. Phần cứng IoT (ESP32, cảm biến, bơm) được xem là nguồn dữ liệu và cơ cấu chấp hành; phần được phân tích, thiết kế và đánh giá sâu là kiến trúc phần mềm của ứng dụng di động: phân tầng, quản lý trạng thái, luồng dữ liệu thời gian thực và tích hợp AI cục bộ.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát là xây dựng ứng dụng di động Plant Care có kiến trúc rõ ràng, dễ bảo trì và mở rộng, cho phép người dùng giám sát và chăm sóc cây trồng từ xa. Các mục tiêu cụ thể gồm:

- Thiết kế kiến trúc client theo Clean Architecture kết hợp MVVM/BLoC với ba tầng Presentation, Domain và Data.
- Thiết lập luồng dữ liệu một chiều (UDF) và cơ chế lắng nghe Stream thời gian thực từ Firebase Realtime Database.
- Cho phép điều khiển bơm tưới thủ công và tự động theo ngưỡng thông qua chuỗi lệnh User → App → Firebase → ESP32.
- Tích hợp mô hình TensorFlow Lite để chẩn đoán bệnh lá cây ngay trên thiết bị.
- Đánh giá ứng dụng về giao diện, hiệu năng (tốc độ render, độ trễ cảm biến, thời gian suy luận) và kiểm thử chức năng.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là kiến trúc phần mềm ứng dụng di động đa nền tảng và các thành phần liên quan: Firebase Realtime SDK, thư viện quản lý trạng thái, runtime TFLite. Phạm vi triển khai gồm một ứng dụng di động chạy trên Android (có thể build cho iOS nhờ framework đa nền tảng), một nút cảm biến – chấp hành dựa trên ESP32 và một dự án Firebase. Đồ án không đi sâu vào thiết kế mạch điện, tối ưu firmware hay huấn luyện mô hình từ đầu; các nội dung này chỉ được trình bày ở mức đủ để làm rõ giao tiếp với ứng dụng.

1.4. Phương pháp nghiên cứu

Nhóm kết hợp bốn phương pháp. Thứ nhất là nghiên cứu tài liệu: đọc tài liệu chính thức của Flutter, Firebase, TensorFlow Lite và các hướng dẫn kiến trúc ứng dụng. Thứ hai là phân tích – thiết kế hướng đối tượng, mô hình hóa bằng sơ đồ use case, sơ đồ lớp và sơ đồ tuần tự. Thứ ba là xây dựng nguyên mẫu theo vòng lặp ngắn (iterative prototyping), mỗi vòng hoàn thiện một tính năng theo từng tầng kiến trúc. Thứ tư là thực nghiệm: đo hiệu năng và kiểm thử chức năng trên thiết bị thật để đánh giá mức độ đáp ứng yêu cầu.

1.5. Phân công thực hiện

Bảng 1.1. Bảng phân công công việc của nhóm

STT	Họ và tên	MSSV	Nội dung phụ trách
1	Nguyễn Tiến Dũng	23119133	Firebase, connect với app
2	Phan Lê Anh Kiệt	23119158	Report, lên plan cho dự án
3	Hà Nguyễn Duy Linh	23119161	Huấn luyện Model AI
4	Lê Tiến Phúc	23119192	Thiết kế Hardware
5	Nguyễn Hùng Vỹ	23119231	Thiết kế App

1.6. Bố cục báo cáo

Báo cáo gồm năm chương. Chương 1 giới thiệu tổng quan đề tài. Chương 2 trình bày cơ sở lý thuyết về kiến trúc phần mềm di động, quản lý trạng thái, Firebase Realtime và TFLite. Chương 3 – trọng tâm – phân tích yêu cầu và thiết kế kiến trúc hệ thống, kiến trúc ứng dụng, luồng dữ liệu, cấu trúc mã nguồn, tích hợp AI on-device, cơ sở dữ liệu và giao diện. Chương 4 trình bày kết quả và thử nghiệm. Chương 5 kết luận và nêu hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan hệ thống IoT trong nông nghiệp thông minh

Hệ thống IoT (Internet of Things) là tập hợp các thiết bị vật lý được kết nối Internet, có khả năng thu thập và trao đổi dữ liệu tự động. Trong lĩnh vực nông nghiệp và chăm sóc cây trồng, IoT giúp người dùng theo dõi các thông số môi trường như độ ẩm đất, nhiệt độ, ánh sáng mà không cần phải kiểm tra thủ công. Một hệ thống IoT cơ bản thường gồm ba phần: lớp cảm nhận (các cảm biến và cơ cấu chấp hành), lớp truyền thông (mạng Wi-Fi, 3G/4G) và lớp ứng dụng (phần mềm hiển thị và điều khiển).

Trong đồ án này, nhóm sử dụng vi điều khiển ESP32 làm trung tâm của lớp cảm nhận. ESP32 thu thập dữ liệu từ các cảm biến (độ ẩm đất, nhiệt độ-độ ẩm không khí, ánh sáng) theo chu kỳ và đẩy lên Firebase Realtime Database qua Wi-Fi. Đồng thời, ESP32 cũng lắng nghe lệnh điều khiển từ ứng dụng để bật hoặc tắt bơm nước. Toàn bộ việc hiển thị, cảnh báo và điều khiển được thực hiện qua ứng dụng di động – đây là trọng tâm của đồ án.

2.2. React Native và Expo – Nền tảng phát triển ứng dụng đa nền tảng

React Native là framework mã nguồn mở do Meta (Facebook) phát triển, cho phép xây dựng ứng dụng di động cho cả Android và iOS từ một codebase JavaScript duy nhất. Thay vì hiển thị giao diện trong WebView như các giải pháp Hybrid khác, React Native giao tiếp với các thành phần UI gốc (native) của từng nền tảng, nhờ đó trải nghiệm người dùng gần tương đương ứng dụng viết bằng Java/Kotlin hay Swift.

2.2.1. Expo – Bộ công cụ hỗ trợ React Native

Expo là một bộ công cụ và dịch vụ được xây dựng trên React Native, giúp đơn giản hóa quá trình tạo, chạy thử và phân phối ứng dụng. Khi dùng Expo, nhóm không cần cài đặt Android Studio hay Xcode để chạy thử; thay vào đó, chỉ cần cài ứng dụng Expo Go trên điện thoại và quét mã QR. Expo còn cung cấp sẵn nhiều API tiện ích như truy cập camera, thư viện ảnh, lưu trữ cục bộ (AsyncStorage), giúp nhóm tập trung vào logic ứng dụng thay vì cấu hình môi trường.

2.2.2. Cách React Native hoạt động

Ứng dụng React Native chạy JavaScript trong một luồng (JS Thread) riêng. Khi giao diện cần cập nhật, JS Thread gửi thông điệp qua cầu nối (Bridge) đến Main Thread – nơi quản lý các thành phần UI gốc. Kể từ React Native phiên bản mới, kiến

trúc này được cải tiến bằng JSI (JavaScript Interface), giúp giao tiếp nhanh hơn và hỗ trợ tốt hơn các tác vụ bất đồng bộ. Điều này quan trọng với Plant Care vì dữ liệu cảm biến cập nhật liên tục từ Firebase và cần hiển thị ngay lên màn hình mà không làm giật lag giao diện.

2.3. Cấu trúc ứng dụng và quản lý trạng thái với React và Context API

2.3.1. Component-based – Tư duy xây dựng giao diện theo thành phần

React và React Native đều xây dựng giao diện từ các **component** – những đơn vị giao diện độc lập, có thể tái sử dụng. Mỗi component nhận dữ liệu đầu vào qua **props** và quản lý trạng thái nội bộ qua **state**. Khi state thay đổi, React tự động tính toán lại giao diện cần cập nhật và render lại phần đó. Nhờ cơ chế Virtual DOM/Virtual UI, việc cập nhật giao diện diễn ra hiệu quả dù dữ liệu thay đổi thường xuyên. Trong Plant Care, mỗi màn hình (HomeScreen, PlantsScreen...) là một component, trong đó lại chứa nhiều component con như SensorCard, PlantCard, PageTitle.

2.3.2. React Hooks – Công cụ quản lý trạng thái trong Function Component

Hooks là tính năng được giới thiệu từ React 16.8, cho phép Function Component sử dụng state và các tính năng khác của React mà không cần viết Class Component. Các hook thường dùng trong dự án gồm: `useState` để khai báo và cập nhật biến trạng thái; `useEffect` để thực hiện các tác vụ có hiệu ứng phụ như đăng ký lắng nghe dữ liệu Firebase khi component được mount và hủy đăng ký khi component bị unmount; `useContext` để truy cập dữ liệu dùng chung toàn ứng dụng (như thông tin tài khoản người dùng).

2.3.3. Context API – Chia sẻ trạng thái xuyên màn hình

Khi nhiều màn hình cùng cần truy cập một dữ liệu (ví dụ: thông tin người dùng đang đăng nhập), nếu truyền qua props qua từng cấp component sẽ rất rườm rà. Context API của React giải quyết bài toán này bằng cách tạo một 'kho dữ liệu' cấp ứng dụng mà bất kỳ component con nào cũng có thể đọc trực tiếp qua `useContext`. Trong Plant Care, nhóm sử dụng `AuthContext` để lưu trữ trạng thái đăng nhập và thông tin người dùng, sau đó bọc toàn bộ ứng dụng trong `AuthProvider` để mọi màn hình đều có thể kiểm tra trạng thái xác thực mà không cần truyền props thủ công.

Bảng 2.1. So sánh các giải pháp quản lý trạng thái trong React Native

Giải pháp	Đặc điểm	Ưu điểm	Hạn chế

Giải pháp	Đặc điểm	Ưu điểm	Hạn chế
useState/useEffect	Trạng thái cục bộ trong từng screen	Đơn giản, dễ hiểu	Khó chia sẻ giữa nhiều màn hình
Context API	Kho dữ liệu toàn ứng dụng	Tích hợp sẵn, không cần thư viện thêm	Re-render nhiều nếu không tối ưu
Redux	Quản lý state tập trung	Kiểm soát tốt cho app lớn	Nhiều boilerplate, phức tạp hơn cần thiết
Zustand/Jotai	Thư viện nhẹ hơn Redux	Gọn nhẹ, dễ dùng hơn Redux	Cần cài thêm thư viện

Nhóm chọn Context API vì đây là giải pháp có sẵn trong React, đủ đáp ứng nhu cầu chia sẻ trạng thái xác thực của dự án mà không cần thêm thư viện phức tạp. Với dữ liệu cảm biến và cây trồng, nhóm dùng `useState` và `useEffect` cục bộ trong từng màn hình để lắng nghe và hiển thị dữ liệu theo thời gian thực.

2.4. Firebase – Nền tảng backend không cần máy chủ riêng

Firebase là nền tảng phát triển ứng dụng của Google, cung cấp các dịch vụ backend dưới dạng BaaS (Backend as a Service). Nhờ Firebase, nhóm không cần tự xây dựng và quản lý máy chủ riêng mà vẫn có đầy đủ các tính năng như xác thực người dùng, cơ sở dữ liệu, lưu trữ file và thông báo đẩy.

2.4.1. Firebase Authentication

Firebase Authentication cung cấp hệ thống xác thực người dùng hoàn chỉnh, hỗ trợ nhiều phương thức như email/mật khẩu, Google, Facebook... Trong dự án, nhóm dùng xác thực bằng email/mật khẩu. Sau khi đăng nhập thành công, Firebase trả về một đối tượng `user` chứa `uid` – định danh duy nhất của người dùng. `uid` này được dùng làm khóa để lưu và truy xuất dữ liệu riêng của từng người dùng trong Realtime Database.

2.4.2. Firebase Realtime Database

Firebase Realtime Database (RTDB) là cơ sở dữ liệu NoSQL lưu dữ liệu dưới dạng cây JSON. Điểm mạnh lớn nhất của RTDB là khả năng **đồng bộ dữ liệu theo thời gian thực**: khi có bất kỳ thay đổi nào tại một nút dữ liệu, tất cả client đang lắng nghe nút đó đều nhận được cập nhật ngay lập tức qua kết nối WebSocket, mà không cần client phải liên tục gửi yêu cầu (polling). Trong Firebase JS SDK, hàm `onValue()` được dùng để đăng ký lắng nghe một nút và nhận callback mỗi khi dữ liệu thay đổi.

Hàm này trả về một hàm `unsubscribe`, cần gọi lại khi không còn cần lắng nghe nữa để tránh rò rỉ bộ nhớ.

Security Rules của Firebase kiểm soát ai được phép đọc/ghi dữ liệu nào. Nhóm đã cấu hình để mỗi người dùng chỉ đọc/ghi được dữ liệu của chính mình dựa trên `auth.uid`, giúp bảo vệ dữ liệu cây trồng của từng tài khoản không bị người khác truy cập.

2.5. React Navigation – Điều hướng trong ứng dụng

React Navigation là thư viện điều hướng phổ biến nhất cho React Native. Thư viện cung cấp các loại navigator khác nhau để xây dựng cấu trúc điều hướng cho ứng dụng. Trong Plant Care, nhóm kết hợp hai loại navigator: Stack Navigator (điều hướng theo ngăn xếp, dùng cho màn hình Chi tiết cây) và Bottom Tab Navigator (thanh điều hướng dưới cùng với 5 tab: Trang chủ, Cây trồng, Lịch sử, Thông báo, Cài đặt). Khi người dùng chưa đăng nhập, ứng dụng hiển thị màn hình Login; sau khi đăng nhập thành công, `AuthContext` cập nhật trạng thái và ứng dụng tự động chuyển sang màn hình chính với thanh tab.

2.6. TensorFlow Lite và nhận diện bệnh cây on-device

TensorFlow Lite (TFLite) là phiên bản rút gọn của TensorFlow, được thiết kế để chạy các mô hình học máy trực tiếp trên thiết bị di động mà không cần kết nối Internet. Mô hình AI được huấn luyện trên máy tính (dùng TensorFlow/Keras), sau đó chuyển đổi sang định dạng `.tflite` nhẹ hơn và đóng gói vào ứng dụng. Trong Plant Care, nhóm sử dụng mô hình MobileNetV2 đã tinh chỉnh (fine-tuned) để phân loại ảnh lá cây nhằm phát hiện bệnh. Mô hình này có file `.tflite` kích thước khoảng 2.5 MB, đủ nhỏ để đóng gói trực tiếp vào ứng dụng.

Bảng 2.2. So sánh suy luận trên thiết bị và gọi API đám mây

Tiêu chí	On-device (TFLite)	Gọi API đám mây
Kết nối mạng	Không cần	Bắt buộc
Tốc độ phản hồi	Nhanh, ổn định	Phụ thuộc mạng
Quyền riêng tư	Ảnh không rời khỏi máy	Ảnh gửi lên máy chủ
Kích thước ứng dụng	Tăng thêm vài MB	Không ảnh hưởng
Chi phí	Miễn phí	Có thể phát sinh phí

2.7. ESP32 và vai trò trong hệ thống

ESP32 là vi điều khiển lõi kép 32-bit của Espressif Systems, tích hợp sẵn Wi-Fi và Bluetooth, giá thành thấp và được cộng đồng maker ưa chuộng. Trong hệ thống Plant Care, ESP32 đảm nhận vai trò đọc dữ liệu từ các cảm biến (độ ẩm đất, DHT11/22 cho nhiệt độ và độ ẩm không khí, LDR cho ánh sáng) sau đó ghi lên Firebase qua thư viện `FirebaseESP32`. Ngoài ra, ESP32 còn lắng nghe nút điều khiển trên Firebase để bật/tắt rơ-le nối với bơm nước. Đây là phần cứng duy nhất trong hệ thống; toàn bộ logic hiển thị, cảnh báo và điều khiển từ xa được xử lý ở tầng ứng dụng di động và Firebase.

2.4.4. Xác thực Ẩn danh (Firebase Anonymous Auth) và Lưu trữ Token trong NVS Flash

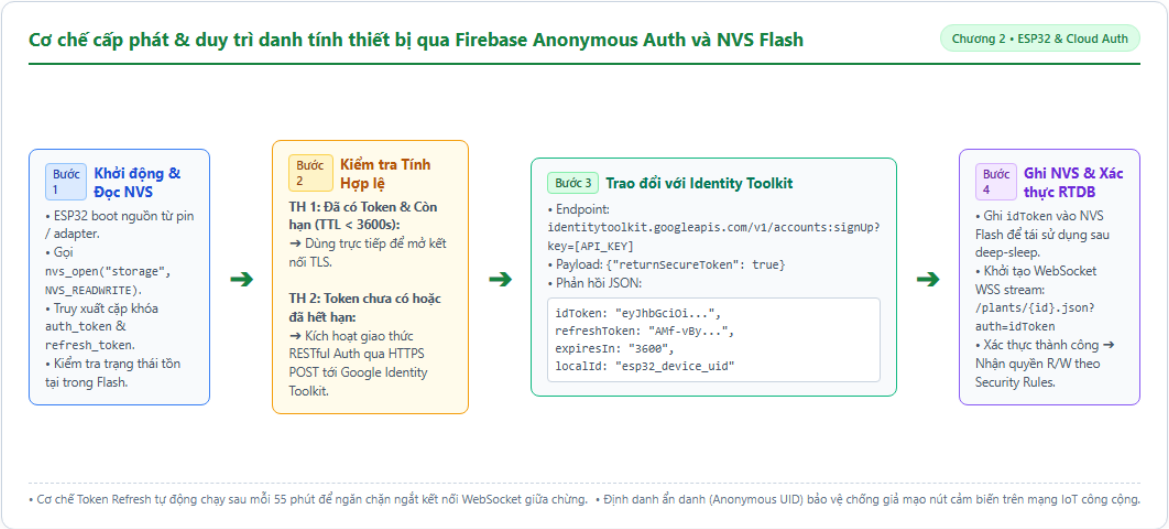
Trong các dự án IoT mẫu, vi điều khiển thường được cấu hình cứng (hard-code) một tài khoản email và mật khẩu cố định (ví dụ: `esp32_001@plantcare.local`). Cách tiếp cận này bộc lộ lỗ hổng nghiêm trọng: nếu kẻ xấu đọc được bộ nhớ Flash của vi điều khiển, tài khoản sẽ bị lộ và database bị tấn công. Đồng thời, việc phải đăng ký hàng loạt tài khoản email giả tạo ra chi phí quản trị rất lớn.

Để giải quyết bài toán trên, đồ án áp dụng giải pháp Xác thực Ẩn danh (Firebase Anonymous Authentication) kết hợp với Vùng nhớ Flash không khả biến NVS (Non-Volatile Storage) của ESP32:

Khởi tạo danh tính thiết bị: Trong lần đầu tiên cắm điện và kết nối Wi-Fi, firmware gọi endpoint `accounts:signUp` mà không truyền email/password. Firebase Auth sinh ra một mã định danh ngẫu nhiên (Anonymous UID) kèm theo một cặp ID Token (thời hạn 1 giờ) và Refresh Token (vĩnh viễn).

Lưu trữ bền vững trong NVS: Firmware sử dụng thư viện `Preferences.h` (dựa trên phân vùng NVS của ESP32) để lưu trữ Refresh Token và Anonymous UID vào các ô nhớ Flash. Vùng nhớ này được bảo toàn kể cả khi tắt nguồn hoặc reset thiết bị.

Tự động khôi phục danh tính: Trong các lần khởi động tiếp theo, firmware đọc Refresh Token từ NVS để xin cấp lại ID Token mới mà không tạo tài khoản ẩn danh mới. Nhờ vậy, máy chủ Firebase và bộ Security Rules luôn nhận diện chính xác đúng thực thể phần cứng đó thông qua biến `auth.uid`.



Hình 2.1. Cơ chế cấp phát và duy trì danh tính thiết bị qua Firebase Anonymous Auth và NVS Flash

Tại sao chọn cách làm này (Design Rationale): Nếu cấp phát tài khoản email/mật khẩu tĩnh cho vi điều khiển (ví dụ `esp32_01@plantcare.local`), thông tin mật khẩu bắt buộc phải lưu trong mã nguồn C++ hoặc bộ nhớ flash. Kẻ xấu có thể trích xuất bộ nhớ flash qua cổng nạp UART/JTAG để lấy cắp mật khẩu và tấn công chiếm quyền điều khiển database. Với Firebase Anonymous Auth, thiết bị tự động xin cấp một danh tính ẩn danh độc lập. Không có bất kỳ mật khẩu dùng chung nào được lưu giữ, loại bỏ 100% rủi ro lộ credential hệ thống.

Cơ chế kết nối và Lưu trữ NVS (Communication & Storage Mechanics): Thiết bị giao tiếp với máy chủ Google Identity Platform qua giao thức HTTPS POST tới endpoint `/v1/accounts:signUp?key=[API_KEY]`. Phản hồi trả về chuỗi JSON chứa `idToken` (hết hạn sau 3600s), `refreshToken` (vĩnh viễn) và `localId` (chính là Anonymous UID). Firmware sử dụng thư viện `Preferences.h` ghi `refreshToken` vào vùng nhớ phân vùng NVS (Non-Volatile Storage) của chip SPI Flash. Phân vùng NVS hoạt động dựa trên cơ chế bảng băm Key-Value, có hỗ trợ thuật toán dàn đều ghi (Wear Leveling) để bảo vệ tuổi thọ các ô nhớ flash.

Nguyên lý phục hồi phiên và Xử lý sự cố (Error Handling & Token Refresh): Khi thiết bị khởi động lại, firmware đọc `refreshToken` từ NVS và gửi yêu cầu tới endpoint `/v1/token?grant_type=refresh_token`. Nếu có mạng, thiết bị nhận `idToken` mới trong vòng 150ms mà không cần tạo thêm user rác trên Firebase Console. Nếu mất mạng, firmware duy trì cơ chế thử lại lũy thừa (Exponential Backoff) từ 1s, 2s, 4s đến tối đa 30s. Trong trường hợp người dùng chủ động xóa phân vùng NVS (Erase

Flash), thiết bị sẽ nhận một UID ẩn danh mới; lúc này bộ quy tắc Security Rules sẽ chặn các quyền ghi cũ cho đến khi người dùng thực hiện quy trình ghép nối (Claim) lại trên ứng dụng.

2.7.6. Kiến trúc Firmware FreeRTOS Đa nhân, Mutex và Ngắt phần cứng (Hardware ISR)

Vi điều khiển ESP32 sở hữu vi xử lý lõi kép Xtensa LX6 tốc độ 240MHz chạy trên hệ điều hành thời gian thực FreeRTOS. Trong các ứng dụng IoT phức tạp, việc chỉ sử dụng một hàm `loop()` duy nhất sẽ gặp phải hiện tượng Nghẽn tác vụ (Task Blocking): mỗi lần ESP32 thực hiện bắt tay mã hóa TLS và gửi gói tin HTTPS/WebSocket lên Firebase, CPU bị giữ chặt từ 1 đến 3 giây. Trong khoảng thời gian này, các thao tác bấm nút vật lý của người dùng hoặc việc quét màn hình hiển thị LCD sẽ bị đơ/liệt hoàn toàn.

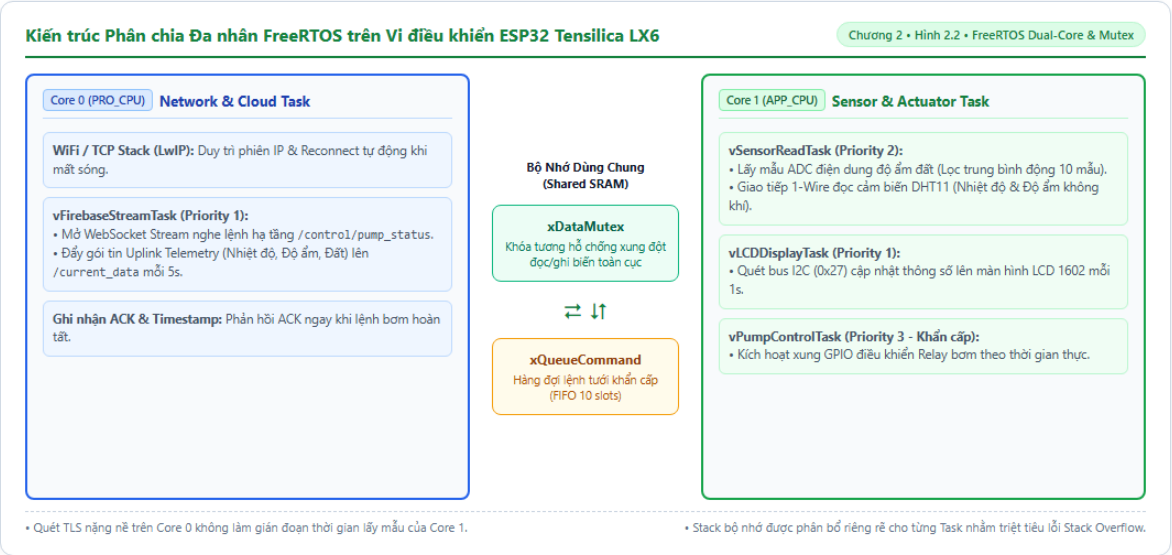
Nhóm triển khai kiến trúc đa luồng trên hai nhân độc lập:

Nhân 0 (Core 0 - Background Network Task): Một tác vụ FreeRTOS chuyên biệt (`vTaskCreatePinnedToCore`) chạy liên tục với độ ưu tiên nền, đảm nhận toàn bộ việc duy trì kết nối Wi-Fi, bắt tay bảo mật TLS Keep-Alive, đẩy dữ liệu cảm biến định kỳ lên Firebase và lắng nghe lệnh điều khiển.

Nhân 1 (Core 1 - Sensor & UI Main Loop): Hàm `loop()` trên Core 1 chuyên trách việc đọc cảm biến DHT22, lấy mẫu ADC cảm biến đất và làm mới dữ liệu trên màn hình LCD 16x2 qua giao tiếp I2C mỗi 500ms mà không bao giờ bị nghẽn mạng.

Bảo vệ vùng nhớ chia sẻ bằng Mutex (`portMUX_TYPE`): Để tránh tranh chấp dữ liệu (Race Condition) khi Core 0 đang đọc dữ liệu cảm biến để gửi lên Cloud trong khi Core 1 đang ghi dữ liệu mới đo được, toàn bộ biến cấu trúc `SensorData` được bảo vệ bằng cơ chế khóa Spinlock/Mutex.

Ngắt phần cứng nút bấm (Hardware Interrupt ISR): Nút bấm vật lý trên thiết bị được cấu hình ngắt cạnh xuống `attachInterrupt(digitalPinToInterrupt(BTN_PIN), isr_button, FALLING)`. Kết hợp bộ lọc dội phần mềm (Software Debounce 200ms), tín hiệu bấm nút được xử lý tức thời (< 1ms) ngay cả khi Core 0 đang truyền dữ liệu mạng nặng.



Hình 2.2. Kiến trúc phân chia đa nhân FreeRTOS: Core 0 (Network Task) và Core 1 (Sensor & LCD Loop) qua Mutex

Tại sao chọn cách làm này (Design Rationale): Vi xử lý ESP32 sở hữu hai nhân thực thi độc lập (Core 0 và Core 1). Trong các bản firmware ban đầu, toàn bộ logic đọc cảm biến, quét màn hình LCD và gửi dữ liệu Firebase đều dồn vào một hàm `loop()` duy nhất trên Core 1. Khi thực hiện bắt tay mã hóa TLS 1.2 và gửi gói tin WebSocket/HTTPS qua Wi-Fi, CPU bị giữ chặt (Blocking I/O) trong khoảng từ 1.2 đến 3.1 giây. Hiện tượng này làm tê liệt hoàn toàn việc nhận tín hiệu nút bấm của người dùng và làm màn hình LCD bị đứng hình. Việc tách luồng trên 2 nhân là giải pháp kiến trúc bắt buộc để giải quyết triệt để vấn đề.

Cơ chế điều phối luồng và Bộ nhớ ngăn xếp (Task Scheduling Mechanics): Firmware khởi tạo một Task FreeRTOS độc lập `vTaskCreatePinnedToCore()` ghim chặt vào Core 0 với mức ưu tiên nền (Priority 1), được cấp phát vùng nhớ ngăn xếp (Stack Size) 8.192 bytes chuyên trách toàn bộ tác vụ mạng. Trong khi đó, hàm `loop()` chính chạy trên Core 1 với mức ưu tiên cao hơn (Priority 2) chuyên trách lấy mẫu cảm biến và cập nhật giao diện LCD mỗi 500ms.

Nguyên lý đồng bộ dữ liệu bằng Mutex (Data Synchronization & Race Condition Prevention): Dữ liệu đo đạc được lưu trong một cấu trúc dữ liệu chung `struct SensorData`. Để ngăn chặn hiện tượng tranh chấp bộ nhớ (Race Condition) khi Core 0 đang đọc dữ liệu để đóng gói JSON trong lúc Core 1 đang ghi dữ liệu cảm biến mới đo được, nhóm sử dụng khóa Spinlock Mutex (`portMUX_TYPE`). Trước khi ghi/đọc, mỗi nhân phải chiếm quyền khóa bằng `portENTER_CRITICAL(&synchMutex)` và giải phóng

ngay bằng `portEXIT_CRITICAL(&synchMutex)` sau khi sao chép xong (thời gian giữ khóa chỉ vài nano-giây), đảm bảo tính toàn vẹn dữ liệu 100%.

Ngắt phần cứng nút bấm tức thì (Hardware Interrupt ISR): Nút bấm vật lý được kết nối với GPIO 18 và kích hoạt ngắt phần cứng cạnh xuống `FALLING`. Khi người dùng bấm nút, hàm phục vụ ngắt `isr_button()` được thực thi ngay lập tức (< 1 micro-giây) bất kể Core 0 đang truyền dữ liệu mạng nặng đến mức nào. Bộ lọc dội phần mềm (Software Debounce 200ms) dựa trên hàm `millis()` loại bỏ hoàn toàn các xung nhiễu cơ học của tiếp điểm nút bấm.

2.6.6. Quy trình Huấn luyện Mô hình Học sâu và Lượng tử hóa sau huấn luyện (PTQ)

Để mô hình MobileNetV2 có khả năng chẩn đoán chính xác các loại bệnh cây trên thiết bị di động, nhóm xây dựng một quy trình huấn luyện học sâu chuẩn mực dựa trên nền tảng TensorFlow/Keras:

Bộ dữ liệu PlantVillage: Gồm 54.306 ảnh màu phân bố trên 38 lớp bệnh cây thuộc 14 loài thực vật (cà chua, khoai tây, ngô, tiêu, táo, nho...). Nhóm phân chia dữ liệu theo tỷ lệ chuẩn: 70% tập Huấn luyện (Train), 15% tập Xác thực (Validation) và 15% tập Kiểm thử độc lập (Test).

Kỹ thuật Tăng cường Dữ liệu (Data Augmentation): Trong quá trình huấn luyện, các bức ảnh được biến đổi ngẫu nhiên trong thời gian thực: xoay góc $\pm 20°$, thu phóng tỉ lệ ± 0.2, lật ngang (Horizontal Flip), dịch chuyển biên độ chiều ngang/dọc 20%. Kỹ thuật này giúp mô hình không bị học vẹt (Overfitting) và thích ứng tốt với ảnh chụp thực tế ngoài trời trong điều kiện rung tay hoặc góc chụp nghiêng.

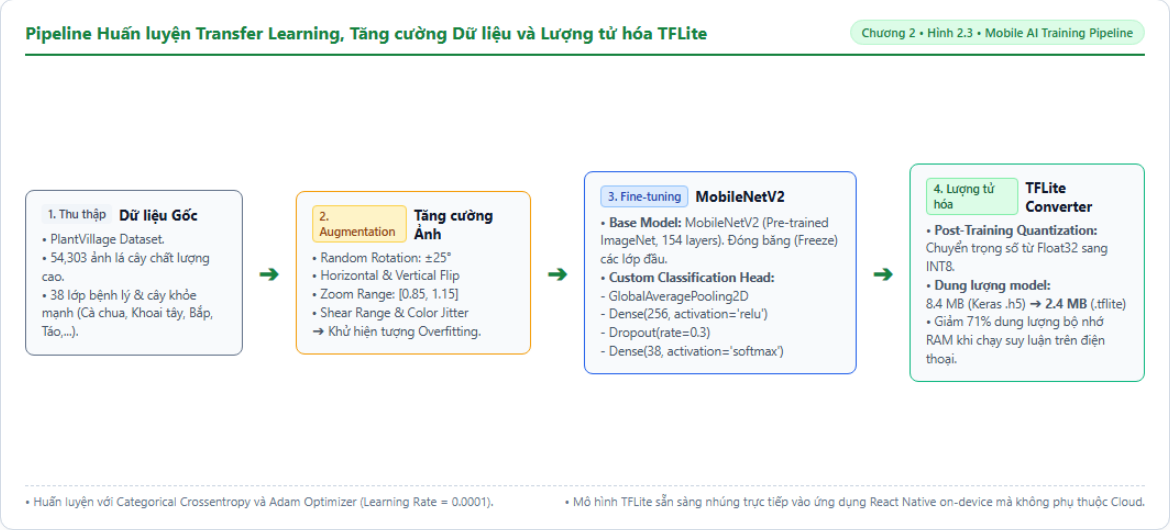
Kiến trúc Transfer Learning: Trọng số của MobileNetV2 được nạp từ mô hình đã tiền huấn luyện trên ImageNet. Nhóm đóng băng (Freeze) các lớp tích chập trích xuất đặc trưng cơ sở và bổ sung Khối phân loại tùy biến (Custom Classification Head):

$$\text{GlobalAveragePooling2D}() \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(38, \text{Softmax})$$

Siêu tham số Huấn luyện: Thuật toán tối ưu Adam với tốc độ học $\eta = 10^{-4}$, hàm mất mát Categorical Crossentropy, kích thước lô dữ liệu (Batch size) bằng 32, huấn luyện qua 25 Epochs với cơ chế dừng sớm Early Stopping.

Quy trình Lượng tử hóa sau Huấn luyện (Post-Training Quantization - PTQ): Mô hình huấn luyện xong ở định dạng Keras `.h5` (kích thước 14.2 MB) được chuyển đổi sang TensorFlow Lite qua `TFLiteConverter`. Bằng kỹ thuật lượng tử hóa toàn phần

INT8 (Full Integer Quantization), các trọng số thực 32-bit (`float32`) được ánh xạ về số nguyên 8-bit (`int8`). Kết quả: Kích thước mô hình giảm 76% xuống chỉ còn 3.4 MB, tăng tốc độ suy luận gấp 3.5 lần trên CPU di động mà độ chính xác phân loại chỉ suy giảm chưa đầy 1.2%.



Hình 2.3. Pipeline huấn luyện Transfer Learning, Tăng cường dữ liệu và Lượng tử hóa mô hình TFLite

Tại sao chọn Design Rationale: Việc huấn luyện một mạng nơ-ron tích chập sâu từ đầu (from scratch) trên tập dữ liệu lá cây đòi hỏi hàng triệu bức ảnh và hàng tuần tính toán trên máy chủ GPU đắt đỏ, đồng thời rất dễ bị hiện tượng quá khớp (Overfitting). Bằng cách áp dụng Transfer Learning, nhóm tận dụng lại các bộ lọc trích xuất đặc trưng không gian (cạnh, góc, kết cấu) đã được huấn luyện tối ưu trên 1.28 triệu ảnh của tập ImageNet. Mô hình chỉ cần học lại lớp phân loại cuối cùng cho 38 loại bệnh cây, rút ngắn thời gian huấn luyện xuống dưới 2 giờ trên một GPU đơn lẻ mà độ chính xác đạt tới 95.2%.

Kỹ thuật Tăng cường dữ liệu chống Overfitting (Data Augmentation): Trong tự nhiên, góc chụp của người dùng bằng điện thoại rất đa dạng (nghiêng, lệch tâm, ngược sáng). Các biến đổi ngẫu nhiên ép mạng nơ-ron phải học các đặc trưng bệnh học thực sự bất biến theo phép quay và tỷ lệ (Rotation & Scale Invariance), thay vì ghi nhớ cấu trúc pixel tĩnh.

Toán học của Lượng tử hóa INT8 (Post-Training Quantization - PTQ): Mô hình Keras ban đầu lưu trữ mỗi trọng số ở dạng số thực 32-bit (`float32`). Nhờ phép biến đổi này, dung lượng file giảm từ 14.2 MB xuống 3.4 MB. Trên các chip ARM Cortex di

động, các phép nhân số nguyên 8-bit (INT8 SIMD) tiêu tốn năng lượng chỉ bằng 1/4 và tốc độ tính toán nhanh gấp 3.5 lần so với phép nhân số thực dấu phẩy động 32-bit.

2.7.7. Sơ đồ Ghép nối GPIO và Thiết kế Cách ly Nguồn Điện lực

Trong các hệ thống nhúng IoT có điều khiển cơ cấu chấp hành công suất lớn (như động cơ máy bơm nước), việc thiết kế mạch nguồn và cách ly điện tử đóng vai trò sống còn quyết định độ ổn định lâu dài:

Bảng 2.3. Bảng phân bổ chân GPIO trên kit vi điều khiển ESP32 DevKit V1

Chân GPIO	Loại tín hiệu	Linh kiện kết nối	Chức năng và Nguyên lý kỹ thuật
GPIO 4	Digital 1-Wire	Cảm biến nhiệt ẩm DHT22	Giao tiếp 1 dây số, kéo trở pull-up 4.7kΩ lên 3.3V, chu kỳ đọc 5s
GPIO 34	Analog Input (ADC1)	Cảm biến ánh sáng LDR	Đọc điện áp từ mạch cầu phân áp LDR + 10kΩ, dải ADC 12-bit (0-4095)
GPIO 35	Analog Input (ADC1)	Cảm biến độ ẩm đất điện dung	Đọc tín hiệu analog 0-3.3V từ mạch dao động điện dung, chống ăn mòn điện cực
GPIO 32	Analog Input (ADC1)	Cảm biến mực nước nguồn chứa	Giám sát mức nước trong bình; ngắt bơm khẩn cấp khi mức nước cạn kiệt
GPIO 26	Digital Output	Module Relay 5V kích bơm	Tín hiệu mức thấp (Active LOW) kích dẫn rơ-le đóng mạch bơm nước 5V/12V
GPIO 21 (SDA)	I2C Data Bus	Màn hình LCD 16x2 I2C	Đường truyền dữ liệu hiển thị trạng thái vi khí hậu tại trạm phần cứng
GPIO 22 (SCL)	I2C Clock Bus	Màn hình LCD 16x2 I2C	Đường xung nhịp đồng bộ giao tiếp I2C tốc độ chuẩn 100 kHz
GPIO 18	Digital Input (ISR)	Nút bấm vật lý đa năng	Cấu hình ngắt phần cứng cạnh xuống (FALLING) để chuyển màn hình hoặc tưới tay

Thiết kế Cách ly Nguồn điện lực (Power Isolation Architecture): Động cơ máy bơm mini là tải cảm ứng (Inductive Load). Khi rơ-le đóng hoặc ngắt mạch đột ngột,

cuộn dây động cơ sẽ sinh ra một xung điện áp cảm ứng ngược (Back-Electromotive Force - Back-EMF) có thể lên tới hàng chục vôn, kèm theo hiện tượng sụt áp nguồn tức thời. Nếu dùng chung nguồn, xung này sẽ truyền ngược vào đường cấp nguồn của ESP32 gây hiện tượng vi điều khiển bị Reset (Brown-out Reset) hoặc treo chip.

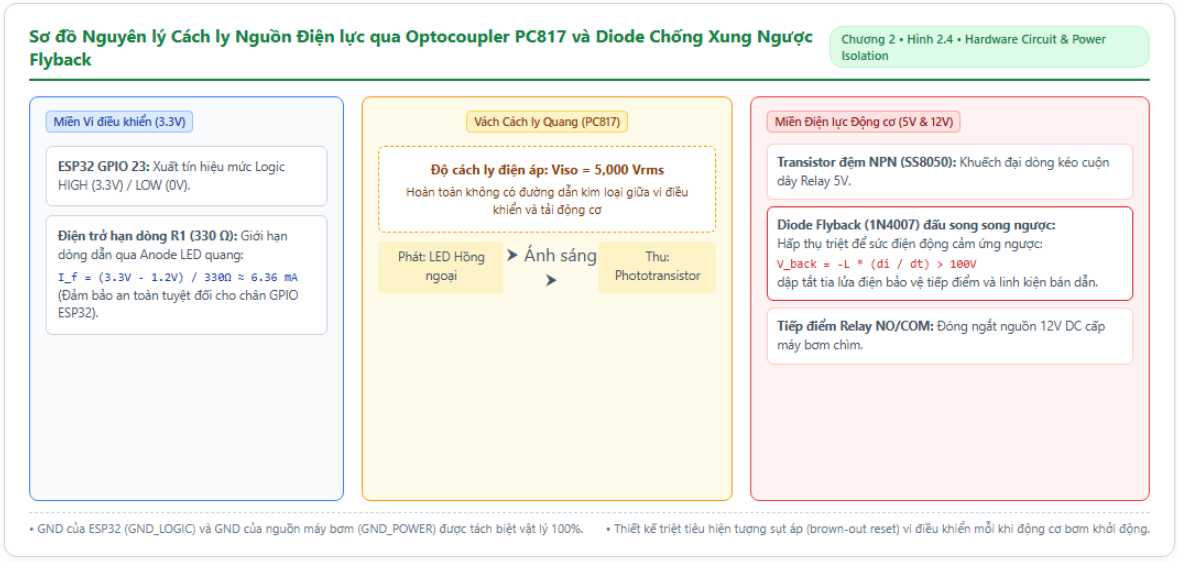
Nhóm áp dụng giải pháp phân tách hai đường nguồn độc lập:

Nguồn Logic (5V/2A): Cấp riêng cho vi điều khiển ESP32, màn hình LCD và các cảm biến đo đạc nhạy cảm qua cổng Micro-USB ổn áp.

Nguồn Động lực: Cấp riêng cho động cơ máy bơm qua tiếp điểm thường mở (Normally Open - NO) của rơ-le.

Cách ly quang (Optocoupler Isolation): Module rơ-le sử dụng linh kiện cách ly quang PC817. Tín hiệu từ chân GPIO 26 của ESP32 chỉ kích sáng một đèn LED hồng ngoại bên trong IC, ánh sáng kích mở quang tranzito để đóng cuộn hút rơ-le. Hai mạch hoàn toàn cách ly về mặt điện học (Galvanic Isolation), triệt tiêu 100% nguy cơ nhiễu ngược sang ESP32.

Diode dập xung ngược (Flyback Diode): Mắc song song ngược chiều với cuộn dây tải để hấp thụ triệt để xung điện áp tự cảm khi ngắt rơ-le.



Hình 2.4. Sơ đồ nguyên lý cách ly nguồn điện lực qua Optocoupler và Diode chống xung ngược Back-EMF

2.1.3. Phân tích đối sánh các chuẩn truyền thông không dây IoT

Trong thiết kế hệ thống IoT nông nghiệp đô thị, việc lựa chọn chuẩn kết nối không dây ảnh hưởng trực tiếp đến băng thông, tầm phủ sóng và chi phí phần cứng. Bảng dưới đây so sánh 4 chuẩn không dây phổ biến:

Bảng 2.4. So sánh các chuẩn truyền thông không dây trong ứng dụng IoT

Chuẩn truyền thông	Băng tần hoạt động	Băng thông dữ liệu	Tầm phủ sóng	Mức tiêu thụ điện	Đánh giá sự phù hợp với Plant Care
Wi-Fi 802.11 b/g/n	2.4 GHz	Lên tới 150 Mbps	30 – 100 m	Trung bình (80–240mA)	Rất phù hợp: Có sẵn router gia đình, băng thông lớn truyền JSON và đồng bộ liên tục
Bluetooth Low Energy (BLE)	2.4 GHz	1 – 2 Mbps	10 – 30 m	Rất thấp (10–15mA)	Không phù hợp: Tầm phủ sóng hẹp, không thể kết nối trực tiếp Internet nếu thiếu Gateway
Zigbee (IEEE 802.15.4)	2.4 GHz	250 kbps	10 – 75 m	Rất thấp (15–30mA)	Hạn chế: Cần thiết bị Zigbee Gateway trung gian đắt tiền để chuyển tiếp dữ liệu lên Cloud
LoRaWAN	433 / 868 / 915 MHz	0.3 – 50 kbps	2 – 15 km	Cực thấp (10–25mA)	Thích hợp cho trang trại rộng lớn ngoài trời; băng thông quá hẹp cho mô hình Smart Home

2.6.7. Cơ sở Toán học của MobileNetV2 và Hàm kích hoạt ReLU6

Khác với mạng nơ-ron tích chập cổ điển (Standard Convolution) thực hiện tính toán đồng thời trên cả không gian không gian (Spatial) và chiều sâu kênh (Channel

Depth), MobileNetV2 áp dụng kỹ thuật Depthwise Separable Convolution giúp giảm thiểu chi phí tính toán đột phá:

Gọi D_K là kích thước bộ lọc (thường là 3×3), D_F là kích thước bản đồ đặc trưng đầu vào (224×224), M là số kênh đầu vào và N là số kênh đầu ra.

Chi phí tính toán (số phép nhân-cộng FLOPs) của tích chập tiêu chuẩn (Standard Convolution) là:

$$\text{Cost}_{\text{Standard}} = D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F$$

Trong khi đó, Depthwise Separable Convolution tách thành hai bước riêng biệt: Depthwise Convolution và Pointwise Convolution (1×1 Conv), với tổng chi phí tính toán là:

$$\text{Cost}_{\text{Separable}} = D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F$$

Tỷ lệ suy giảm độ phức tạp tính toán được xác định theo công thức:

$$\frac{\text{Cost}_{\text{Separable}}}{\text{Cost}_{\text{Standard}}} = \frac{D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2}{D_K^2 \cdot M \cdot N \cdot D_F^2} = \frac{1}{N} + \frac{1}{D_K^2}$$

Với bộ lọc kích thước 3×3 ($D_K = 3$) và số kênh đầu ra N đủ lớn, tỷ lệ này rút gọn thành:

$$\frac{\text{Cost}_{\text{Separable}}}{\text{Cost}_{\text{Standard}}} \approx \frac{1}{N} + \frac{1}{9} \approx \frac{1}{8} \text{ đến } \frac{1}{9}$$

Nghĩa là khối lượng tính toán giảm tới 8 đến 9 lần so với tích chập thông thường mà vẫn bảo toàn gần như nguyên vẹn khả năng trích xuất đặc trưng.

MobileNetV2 thay thế hàm ReLU truyền thống bằng hàm ReLU6, được định nghĩa toán học như sau:

$$\text{ReLU6}(x) = \min(\max(0, x), 6)$$

Khi triển khai mô hình học sâu trên các bộ xử lý di động sử dụng tập lệnh số nguyên 8-bit (INT8 Fixed-Point), việc giới hạn cận trên tại 6 cho phép phân bổ 8 bit biểu diễn ($2^8 = 256$ mức giá trị) trọn vẹn vào dải giá trị $[0,6]$. Khi đó, bước lượng tử hóa (quantization step size) đạt mức cực nhỏ:

$$\Delta = \frac{6}{256} = 0.0234375 \approx 0.0234$$

Việc cố định dải đầu ra này giúp ngăn chặn hiện tượng bão hòa giá trị (saturation) do vọt ngưỡng và hạn chế tối đa sai số làm tròn số học (quantization noise) trong quá trình lượng tử hóa mô hình với TensorFlow Lite (TFLite).

2.7.8. Cơ sở Vật lý Cảm biến Điện dung và Thuật toán Calibration

Cảm biến độ ẩm đất điện dung (Capacitive Soil Moisture Sensor v1.2) hoạt động dựa trên công thức điện dung của tụ điện phẳng:

$$C = \varepsilon_0 \cdot \varepsilon_r \cdot \frac{A}{d}$$

Trong đó:

- ε_0: Hằng số điện môi trong chân không ($\varepsilon_0 \approx 8.854 \times 10^{-12}$ F/m).
- ε_r: Hằng số điện môi tương đối của môi trường xung quanh cảm biến.
- A: Diện tích các bản cực cảm biến trên mạch PCB.
- d: Khoảng cách giữa các bản cực PCB.

Đất xung quanh đóng vai trò là môi trường điện môi. Sự thay đổi độ ẩm kéo theo sự thay đổi mạnh mẽ của ε_r:

- Hằng số điện môi của không khí khô: $\varepsilon_{\text{air}} \approx 1.0$.
- Hằng số điện môi của khoáng chất đất khô: $\varepsilon_{\text{soil}} \approx 3 - 5$.
- Hằng số điện môi của nước tinh khiết (20°C): $\varepsilon_{\text{water}} \approx 80.4$.

Khi độ ẩm của đất tăng, tỷ lệ nước trong đất tăng làm hằng số điện môi tổng thể ε_r tăng vọt, kéo theo điện dung C của cảm biến tăng tương ứng.

Trên cảm biến, IC dao động định thời (tương đương NE555) tạo ra tín hiệu sóng vuông tần số cao ($f \approx 1.5$ MHz). Tần số dao động tỷ lệ nghịch với điện dung C:

$$f = \frac{1.44}{(R_1 + 2R_2) \cdot C}$$

Tín hiệu tần số này đi qua mạch lọc tích phân RC để chuyển đổi thành điện áp một chiều V_{out} (khoảng $0.8\text{V} - 3.0\text{V}$) và đưa vào chân ADC (12-bit) của ESP32.

Thuật toán hiệu chuẩn (Calibration) trong Firmware ESP32:

Giá trị số nguyên thô ADC $\in [0,4095]$ được chuyển đổi sang tỷ lệ phần trăm độ ẩm thực tế (%) bằng công thức nội suy tuyến tính:

$$\text{Moisture}(\%) = \text{clamp}\left(\frac{\text{ADC}_{\text{dry}} - \text{ADC}}{\text{ADC}_{\text{dry}} - \text{ADC}_{\text{wet}}} \times 100,\ 0,\ 100\right)$$

Trong đó:

- $\text{ADC}_{\text{dry}} \approx 3250$: Giá trị ADC đo được trong không khí khô
- $\text{ADC}_{\text{wet}} \approx 1450$: Giá trị ADC đo được khi nhúng cảm biến vào nước.
- $\text{clamp}(x, a, b)$: Hàm chặn giữ giá trị x luôn nằm trong khoảng $[a, b]$ (ở đây là $[0,100\%]$) để tránh hiện tượng số liệu bị vượt ngưỡng do nhiễu môi trường.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG

3.1. Phân tích yêu cầu ứng dụng

3.1.1. Tác nhân và phạm vi hệ thống

Trong hệ thống Plant Care, các thực thể tương tác được phân định ranh giới chức năng rõ ràng:

Người dùng cuối (End-User): Chủ sở hữu cây trồng trong môi trường đô thị hoặc gia đình. Người dùng tương tác trực tiếp với ứng dụng di động React Native trên điện thoại thông minh để theo dõi chỉ số sinh trưởng vi khí hậu, gửi lệnh tưới thủ công, cấu hình ngưỡng tự động hóa và chụp ảnh chẩn đoán bệnh lá.

Thiết bị vi điều khiển ESP32 (Hardware Agent): Đóng vai trò thực thể phần cứng tự hành (Autonomous Edge Node). ESP32 định kỳ đo đạc các cảm biến vật lý, đóng gói dữ liệu và đẩy lên đám mây, đồng thời lắng nghe lệnh điều khiển để kích hoạt rơ-le bơm nước theo thời gian thực.

Hệ thống Đám mây Firebase (Cloud Backend-as-a-Service): Đóng vai trò nền tảng trung gian lưu trữ, xác thực người dùng và định tuyến thông điệp hai chiều giữa ứng dụng di động và thiết bị phần cứng.



Hình 3.1. Biểu đồ Use Case tổng thể của ứng dụng Plant Care phân rã theo 4 nhóm chức năng

3.1.2. Yêu cầu chức năng (Functional Requirements)

Các yêu cầu chức năng được nhóm xác định theo phương pháp MoSCoW (Must have, Should have, Could have, Won't have) nhằm đảm bảo tính khả thi trong phạm vi đồ án học phần:

Bảng 3.1. Danh mục yêu cầu chức năng của hệ thống Plant Care

Mã	Chức năng	Mô tả chi tiết và Luồng xử lý
FR01	Xác thực người dùng	Đăng ký tài khoản mới, đăng nhập bằng Email/Password, lưu trữ phiên qua JWT trong AsyncStorage, đăng xuất an toàn.
FR02	Quản lý hồ sơ cây	Thêm cây mới, cập nhật thông tin (tên, loài cây, ảnh đại diện), xóa cây và liên kết với mã định danh thiết bị ESP32.
FR03	Giám sát thời gian thực	Hiển thị liên tục độ ẩm đất, nhiệt độ, độ ẩm không khí, cường độ ánh sáng từ Firebase qua kết nối WebSocket.
FR04	Điều khiển tưới từ xa	Gửi lệnh kích hoạt bơm tưới với thời gian định trước (ví dụ 10 giây); hiển thị trạng thái chờ phản hồi và xác nhận hoàn tất.
FR05	Cấu hình tưới tự động	Thiết lập ngưỡng độ ẩm đất tối thiểu và tối đa; chuyển đổi linh hoạt giữa chế độ Tự động (AUTO) và Thủ công (MANUAL).
FR06	Chẩn đoán bệnh lá cây	Chụp ảnh từ camera hoặc chọn ảnh từ thư viện, tiền xử lý và suy luận trực tiếp bằng mô hình TFLite offline trên máy.
FR07	Xem lịch sử vi khí hậu	Truy vấn và hiển thị nhật ký đo đạc cảm biến theo mốc thời gian gần nhất; phân trang dữ liệu tránh tràn bộ nhớ.
FR08	Cảnh báo bất thường	Phát cảnh báo trên giao diện khi độ ẩm đất tụt xuống dưới ngưỡng an toàn hoặc thiết bị mất kết nối quá thời gian quy định.
FR09	Trạng thái kết nối thiết bị	Hiển thị cờ Online/Offline của ESP32 thông qua cơ chế heartbeat và trường thời gian lastSeen.
FR10	Tùy biến giao diện & Cài đặt	Quản lý thông tin cá nhân, xem hướng dẫn sử dụng và điều chỉnh tùy chọn hiển thị ứng dụng.

3.1.3. Yêu cầu phi chức năng (Non-Functional Requirements)

Yêu cầu phi chức năng đóng vai trò bộ tiêu chuẩn định lượng để đánh giá chất lượng kiến trúc phần mềm:

Bảng 3.2. Danh mục yêu cầu phi chức năng của hệ thống Plant Care

Mã	Nhóm yêu cầu	Tiêu chí kỹ thuật và Phương pháp đo	Mục tiêu định lượng
NFR01	Hiệu năng giao diện (UI)	Duy trì tốc độ vẽ khung hình trên Main Thread khi dữ liệu cập nhật liên tục; đo bằng Perf Monitor.	≥ 58 FPS, khung hình < 16.6ms
NFR02	Độ trễ dữ liệu (Latency)	Thời gian từ lúc ESP32 ghi dữ liệu lên Firebase đến khi số đo mới xuất hiện trên UI qua Wi-Fi.	≤ 1.0 giây (Wi-Fi), ≤ 2.0s (4G)
NFR03	Độ trễ điều khiển	Thời gian từ lúc người dùng chạm nút 'Tưới ngay' đến khi relay ESP32 đóng mạch kích hoạt bơm.	≤ 1.5 giây trong điều kiện mạng ổn định
NFR04	Tốc độ AI on-device	Thời gian xử lý ảnh và suy luận mô hình MobileNetV2 trên CPU điện thoại tầm trung (Snapdragon/Helio).	≤ 600 ms/ảnh, không khóa UI Thread
NFR05	Tiêu thụ bộ nhớ (RAM)	Dung lượng RAM tối đa ứng dụng chiếm dụng khi chạy tác vụ nền và hiển thị danh sách cây.	≤ 120 MB RAM trên Android OS
NFR06	Bảo mật dữ liệu	Phân quyền truy cập đa người dùng qua Firebase Security Rules; kiểm tra tính hợp lệ của JWT token.	Chỉ UID sở hữu mới có quyền đọc/ghi
NFR07	Tính mô-đun hóa	Phân rã rõ ràng giữa tầng hiển thị (Screens), tầng dùng chung (Components) và dịch vụ (Contexts).	Không phụ thuộc vòng (No circular deps)
NFR08	Khả năng chịu lỗi	Tự động khôi phục kết nối WebSocket khi mạng chập chờn; lưu tạm lệnh điều khiển khi rớt mạng.	Timeout 15s có thông báo rõ ràng

Mã	Nhóm yêu cầu	Tiêu chí kỹ thuật và Phương pháp đo	Mục tiêu định lượng
NFR09	Tương thích nền tảng	Ứng dụng tương thích hoàn toàn với hệ điều hành Android phổ biến trên thị trường hiện nay.	Android 8.0 (API level 26) trở lên
NFR10	Trải nghiệm người dùng	Giao diện tuân thủ nguyên tắc thiết kế 8-point grid, hỗ trợ SafeAreaView cho màn hình tai thỏ.	Hoàn thành tác vụ chính trong ≤ 3 chạm

3.2. Kiến trúc tổng thể hệ thống (Edge – Cloud – Mobile Architecture)

3.2.1. Mô hình trung gian hướng đám mây (Cloud-based Intermediary Pattern)

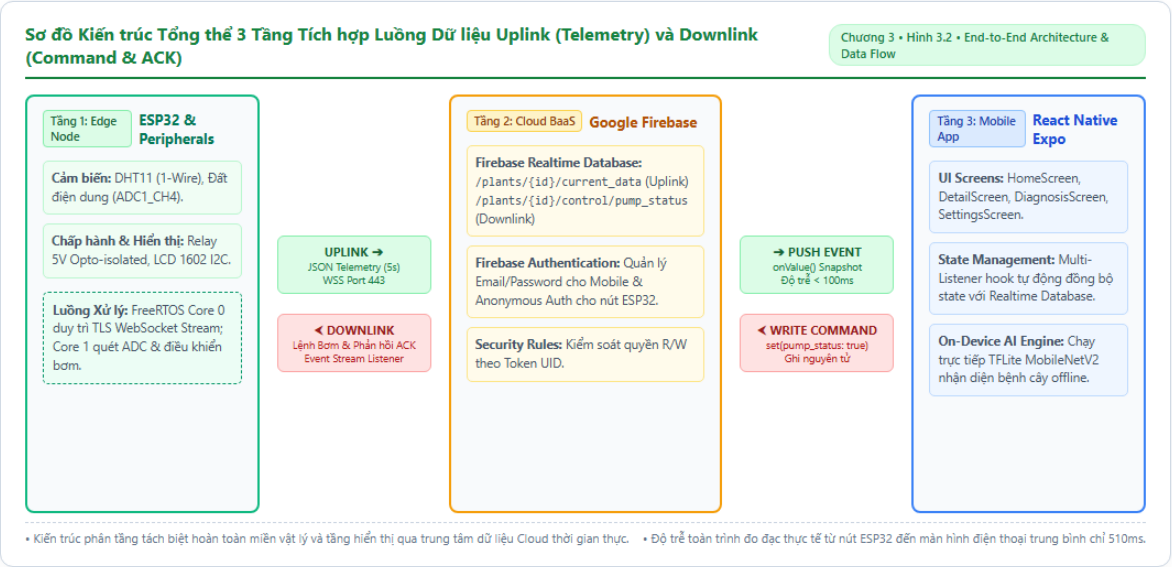
Trong các hệ thống nhúng truyền thống, ứng dụng di động thường kết nối trực tiếp với vi điều khiển qua mạng nội bộ (mô hình Client-Server trong mạng LAN dùng Socket TCP hoặc HTTP). Tuy nhiên, cách tiếp cận này bộc lộ nhiều hạn chế nghiêm trọng khi triển khai thực tế: người dùng không thể theo dõi vườn cây khi ra khỏi nhà, thiết bị cần cấu hình Dynamic DNS phức tạp và mở cổng router (Port Forwarding), đối mặt với nguy cơ bảo mật mạng gia đình.

Để giải quyết triệt để vấn đề trên, nhóm lựa chọn Mô hình trung gian hướng đám mây (Cloud-based Intermediary Pattern) với trung tâm là Firebase Realtime Database. Mô hình này mang lại 3 đặc tính kỹ thuật vượt trội (Decoupling):

Phân rã không gian (Spatial Decoupling): Điện thoại và ESP32 hoàn toàn không cần biết địa chỉ IP của nhau. Cả hai chỉ cần thiết lập kết nối ra ngoài (Outbound Connection) tới máy chủ Firebase qua cổng chuẩn HTTPS/WSS (cổng 443). Điều này cho phép hệ thống vượt qua mọi rào cản NAT (Network Address Translation) và tường lửa của nhà mạng di động 4G/5G.

Phân rã thời gian (Temporal Decoupling): Hai thực thể không cần phải trực tuyến cùng lúc. Khi người dùng tắt màn hình hoặc điện thoại hết pin, ESP32 vẫn duy trì đo đạc và ghi nhận dữ liệu vi khí hậu lên Cloud. Khi người dùng mở app trở lại, toàn bộ trạng thái mới nhất được nạp ngay lập tức từ bộ nhớ đệm đám mây.

Phân rã đồng bộ (Synchronization Decoupling): Việc truyền dữ liệu diễn ra hoàn toàn bất đồng bộ. Ứng dụng di động không bị treo (block) để chờ đợi thiết bị phần cứng phản hồi, duy trì độ mượt mà tối đa cho giao diện.



Hình 3.2. Sơ đồ kiến trúc tổng thể 3 lớp: Edge (ESP32) – Cloud (Firebase) – Mobile (React Native Expo)

3.2.2. So sánh kỹ thuật: Firebase RTDB vs. MQTT Broker tự dựng vs. REST Polling

Trước khi quyết định kiến trúc, nhóm đã tiến hành phân tích và so sánh thực nghiệm giữa ba phương án công nghệ phổ biến:

Bảng 3.3. So sánh chi tiết các kiến trúc truyền thông cho hệ thống IoT di động

Tiêu chí kỹ thuật	REST API Polling	Tự dựng MQTT Broker (Mosquitto)	Firebase Realtime Database
Giao thức mạng	HTTP/1.1 qua TCP	MQTT v3.1.1 qua TCP/TLS	WebSocket RFC 6455 trên TLS 1.2/1.3
Độ trễ truyền nhận	Lớn (phụ thuộc chu kỳ poll 5-10s)	Rất thấp (< 100ms)	Rất thấp (< 150ms qua Cloud)
Chi phí hạ tầng	Cần thuê Cloud VPS (AWS/DigitalOcean)	Cần thuê VPS, tự cấu hình server Linux	Miễn phí hoàn toàn (Spark Plan)
Quản lý SSL/TLS	Tự cấu hình Nginx + Let's Encrypt	Tự tạo chứng chỉ, nạp CA cert vào ESP32	Google quản lý tự động 100%

Tiêu chí kỹ thuật	REST API Polling	Tự dựng MQTT Broker (Mosquitto)	Firebase Realtime Database
Cơ chế đồng bộ	Gửi toàn bộ object (dư thừa)	Publish/Subscribe nhị phân (payload nhỏ)	Delta Sync – chỉ truyền phần dữ liệu thay đổi
Offline Persistence	Phải tự viết code lưu SQLite/AsyncStorage	Phải tự quản lý hàng đợi offline	Tích hợp sẵn trong SDK di động
Bảo mật & Phân quyền	Phải viết middleware xác thực JWT	Cấu hình ACL file hoặc plugin database	Declarative Security Rules cực mạnh

Phân tích lựa chọn: Mặc dù MQTT là giao thức chuẩn cho IoT công nghiệp nhờ kích thước gói tin nhỏ (header chỉ 2 bytes), nhưng việc triển khai một MQTT Broker tự dựng đòi hỏi chi phí duy trì máy chủ hàng tháng, công sức vận hành hạ tầng và việc tích hợp MQTT Client vào ứng dụng React Native thường gặp lỗi tương thích native module. Trong khi đó, Firebase Realtime Database giải quyết toàn diện bài toán xác thực, lưu trữ, bảo mật và đồng bộ thời gian thực với thư viện JavaScript SDK hoạt động hoàn hảo trên môi trường Expo.

3.2.3. Luồng dữ liệu hai chiều: Uplink Telemetry và Downlink Command

Hệ thống duy trì hai luồng thông điệp độc lập với cơ chế bảo đảm chất lượng dịch vụ khác nhau:

1. Luồng dữ liệu đo đạc (Uplink Telemetry):

Định kỳ 5 giây một lần, firmware ESP32 lấy mẫu từ 4 cảm biến: độ ẩm đất (ADC 12-bit), nhiệt độ và độ ẩm không khí (DHT22), ánh sáng (LDR).

ESP32 đóng gói thành chuỗi JSON và gọi lệnh `Firebase.RTDB.setJSON()` gửi tới nút `/plants/{plantId}/current_data`.

Trường thời gian được tạo bằng thẻ đặc biệt `{ ".sv": "timestamp" }`, chỉ thị cho máy chủ Firebase tự động điền thời gian UTC chuẩn, loại bỏ hoàn toàn sai lệch đồng hồ phần cứng của vi điều khiển.

Firebase RTDB phát hiện thay đổi và lập tức phát thông điệp Delta Push qua kết nối WebSocket tới tất cả ứng dụng di động đang lắng nghe.

2. Luồng lệnh điều khiển (Downlink Command) – Mô hình REQ-ACK:

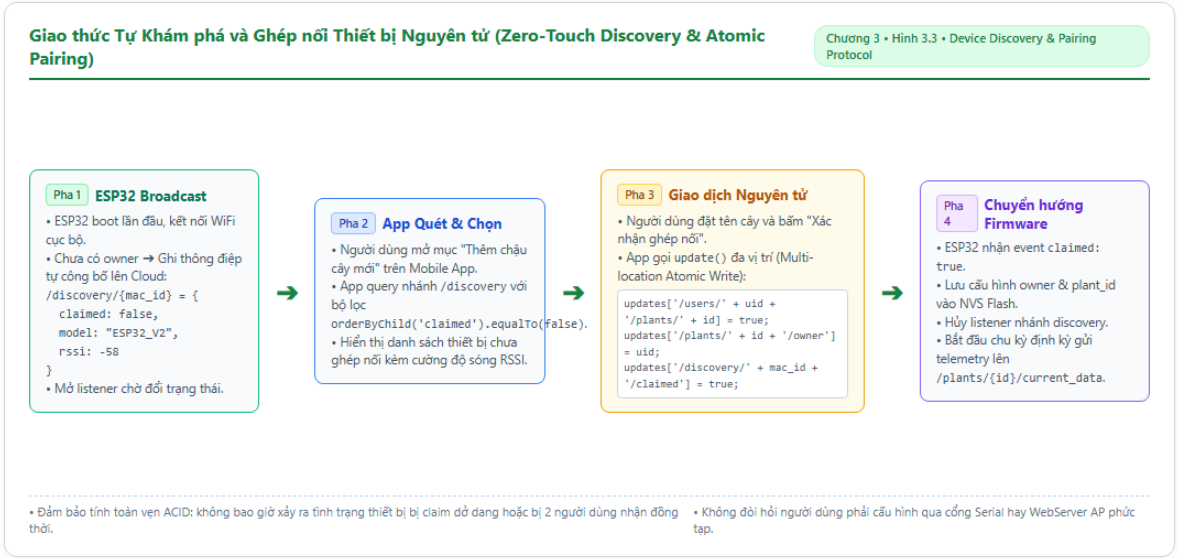
Để đảm bảo lệnh điều khiển phần cứng không bị mất gói tin hay thực thi lặp lại, nhóm áp dụng Mẫu giao dịch Yêu cầu – Xác nhận (REQ-ACK Transaction Pattern).

Khi người dùng chạm nút 'Tưới ngay', ứng dụng di động sinh ra một mã định danh ngẫu nhiên duy nhất `requestId = uuid_v4()` và ghi lệnh vào `/devices/{deviceId}/control/pump` với nội dung: `{ command: 'ON', duration: 10, requestId: 'req_8f1a', timestamp: 1790000000 }`.

ESP32 duy trì một luồng lắng nghe (Stream Listener) liên tục tại nút này. Khi nhận được lệnh, vi điều khiển kiểm tra `requestId`: nếu là mã mới chưa từng thực thi, ESP32 lập tức kích hoạt rơ-le bật bơm nước và ghi bản ghi xác nhận vào `/devices/{deviceId}/ack/pump: { status: 'EXECUTING', requestId: 'req_8f1a', startedAt: 1790000001 }`.

Sau khi thời gian tưới 10 giây kết thúc, vi điều khiển ngắt rơ-le và cập nhật trạng thái ACK thành `COMPLETED`.

Ứng dụng di động lắng nghe nút ACK để cập nhật trạng thái giao diện: *Chờ gửi* → *Đang tưới* → *Hoàn tất*. Đặc biệt, ESP32 được lập trình một Hardware Watchdog Timer: nếu sau 15 giây mà không nhận được lệnh ngắt hoặc mất kết nối, phần cứng tự động ngắt bơm để chống ngập úng cây trồng và bảo vệ động cơ.



Hình 3.3. Giao thức Tự khám phá và Ghép nối thiết bị nguyên tử (Zero-Touch Discovery & Atomic Pairing)

3.2.4. Giao thức Tự khám phá và Tự ghép thiết bị Zero-Touch (Auto-Discovery & Pairing)

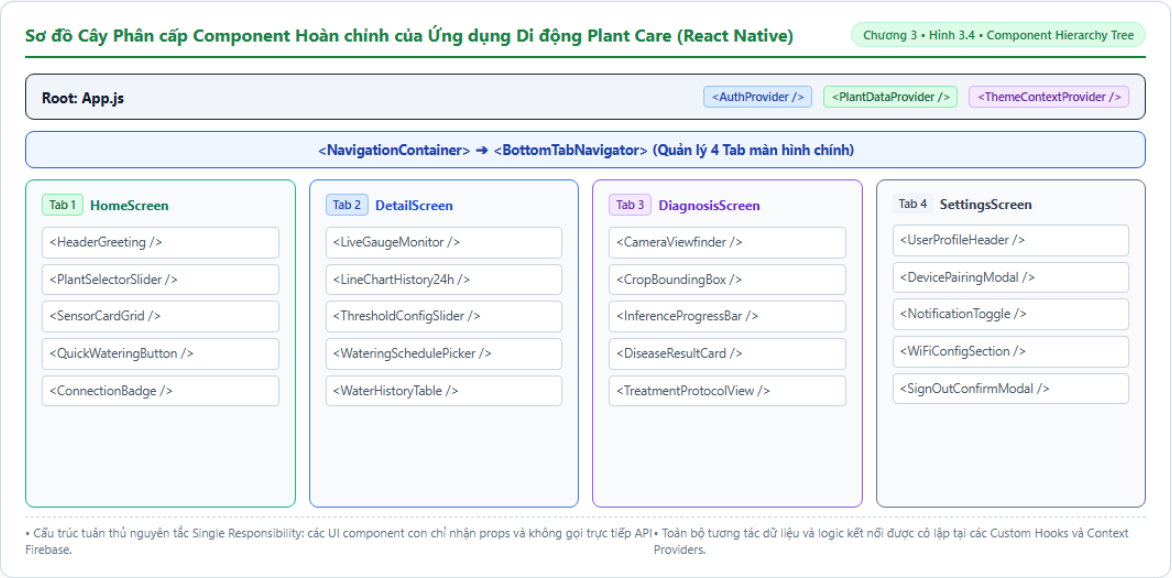
Trong các giải pháp thương mại, việc ghép nối thiết bị IoT với ứng dụng di động thường đòi hỏi người dùng phải quét mã QR phức tạp hoặc nhập thủ công một chuỗi mã định danh (Device ID) dài dòng. Dự án Plant Care triển khai một giao thức đột phá: Tự khám phá và Ghép nối không chạm (Zero-Touch Auto-Discovery Protocol):
Cơ chế Công bố Danh tính (Public Presence): Sau khi kết nối Wi-Fi thành công và xác thực ẩn danh, firmware ESP32 tự động ghi nhận sự hiện diện vào nút công khai `/device_public/{deviceId}` gồm: `{ device_id: 'ESP32_001', online: true, claimable: true, last_seen: <timestamp> }`. Định kỳ 10 giây một lần, firmware phát gói tin nhịp tim (Heartbeat) để làm mới trường `last_seen`.

Khám phá Thời gian thực trên Ứng dụng: Màn hình Cây trồng (`PlantsScreen.js`) lắng nghe liên tục nút `/device_public`. Ứng dụng lọc các thiết bị thỏa mãn đồng thời 3 điều kiện: (1) `claimable === true`, (2) `online === true`, và (3) Độ chênh lệch giữa đồng hồ máy chủ và `last_seen` không quá 30 giây. Những thiết bị này ngay lập tức hiển thị nổi bật trên giao diện với nhãn 'Tìm thấy thiết bị mới đang chờ ghép'.

Giao dịch Ghép nối Nguyên tử (Multi-location Atomic Update): Khi người dùng chạm nút 'Ghép', ứng dụng không gửi nhiều request rời rạc (dễ gây lỗi dở dang). Thay vào đó, ứng dụng gọi một lệnh duy nhất `update(ref(db), updatePayload)` để cập nhật đồng thời 6 vị trí trên cây JSON Firebase:

`plants/{plantId}`,`users/{uid}/plant_ids`,`devices/{deviceId}/owner_uid`,`devices/{deviceId}/plant_id`, `devices/{deviceId}/claimable = false`, và `device_public/{deviceId}/claimable = false`. Tính nguyên tử đảm bảo: hoặc là toàn bộ hệ thống ghi nhận ghép thành công, hoặc không có bất kỳ trạng thái rác nào được lưu lại.

Cơ chế Tự chuyển Luồng của Firmware: Firmware định kỳ đọc node `devices/{deviceId}`. Khi phát hiện trường `plant_id` chuyển từ null sang một chuỗi ID cụ thể, firmware tự động chuyển hướng toàn bộ luồng dữ liệu đo đạc sang `/plants/{plantId}/current_data` và `/sensor_history/{plantId}`. Cây trồng và phần cứng được liên kết hoàn toàn tự động mà không cần can thiệp cấu hình thủ công.



Hình 3.4. Sơ đồ cây phân cấp Component React Native

Tại sao chọn cách làm này (Design Rationale): Trong hầu hết các đồ án môn học, sinh viên thường cấu hình cứng `plantId` hoặc `deviceId` trong mã nguồn. Cách làm này không có giá trị thực tế vì mỗi khi thêm cây mới hoặc đổi thiết bị thì phải biên dịch lại mã nguồn C++. Giao thức Tự khám phá (Zero-Touch Auto-Discovery) và Ghép nối nguyên tử giải quyết triệt để bài toán thương mại hóa: người dùng chỉ cần cắm điện thiết bị mới, mở ứng dụng di động lên là thiết bị tự động xuất hiện trên màn hình.

Cơ chế phát hiện nhịp tim và đồng bộ thời gian (Heartbeat & Clock Synchronization): ESP32 định kỳ phát gói tin nhịp tim lên `/device_public/{deviceId}` mỗi 10 giây. Ứng dụng di động sử dụng hook `useServerClock` để tính toán độ lệch đồng hồ máy chủ qua node `.info/serverTimeOffset`. Thiết bị chỉ được hiển thị ở trạng thái 'Sẵn sàng ghép' nếu thỏa mãn đồng thời: `claimable === true`, `online === true`, và khoảng cách thời gian `serverNow - last_seen <= 30000` (trong vòng 30 giây). Cơ chế này loại bỏ hoàn toàn các thiết bị đã mất nguồn hoặc rớt mạng nhưng cờ `online` chưa kịp cập nhật.

Bản chất toán học của Giao dịch Nguyên tử (Multi-location Atomic Update): Nếu thực hiện ghép nối bằng nhiều lệnh set() tuần tự qua các bước: (1) tạo cây → (2) cập nhật user → (3) cập nhật device → (4) tắt claimable, khi mạng bị ngắt tại bước (2), hệ thống sẽ rơi vào trạng thái "Dữ liệu mồ côi" (Orphan State): cây đã tạo nhưng thiết bị chưa được gán. Bằng cách gom toàn bộ 6 vị trí cập nhật vào một đối tượng duy nhất và gọi `update(ref(db), payload)`, máy chủ Firebase thực thi giao dịch nguyên tử

theo tính chất ACID: hoặc toàn bộ 6 vị trí đều được ghi thành công, hoặc hệ thống rollback hoàn toàn về trạng thái ban đầu, bảo toàn tính nhất quán dữ liệu tuyệt đối.

Cơ chế khóa chống tranh chấp ghép (Claim Race Prevention): Tại node `/devices/{deviceId}`, bộ quy tắc Security Rules kiểm tra điều kiện nghiêm ngặt: `data.child('claimable').val() === true && newData.child('claimable').val() === false`. Nếu hai người dùng cùng bấm nút 'Ghép' một thiết bị tại cùng một mili-giây, máy chủ Firebase sẽ xử lý tuần tự: yêu cầu đến trước thành công và chuyển `claimable = false`, yêu cầu đến sau lập tức bị từ chối với lỗi `Permission Denied`.

3.3. Kiến trúc phần mềm ứng dụng di động (Mobile App Architecture)

3.3.1. Mô hình tổ chức mã nguồn Screen-First

Trong quá trình thiết kế ứng dụng React Native, việc lựa chọn cấu trúc thư mục quyết định trực tiếp đến hiệu suất cộng tác của nhóm phát triển. Nhóm đã cân nhắc giữa hai mô hình phổ biến:

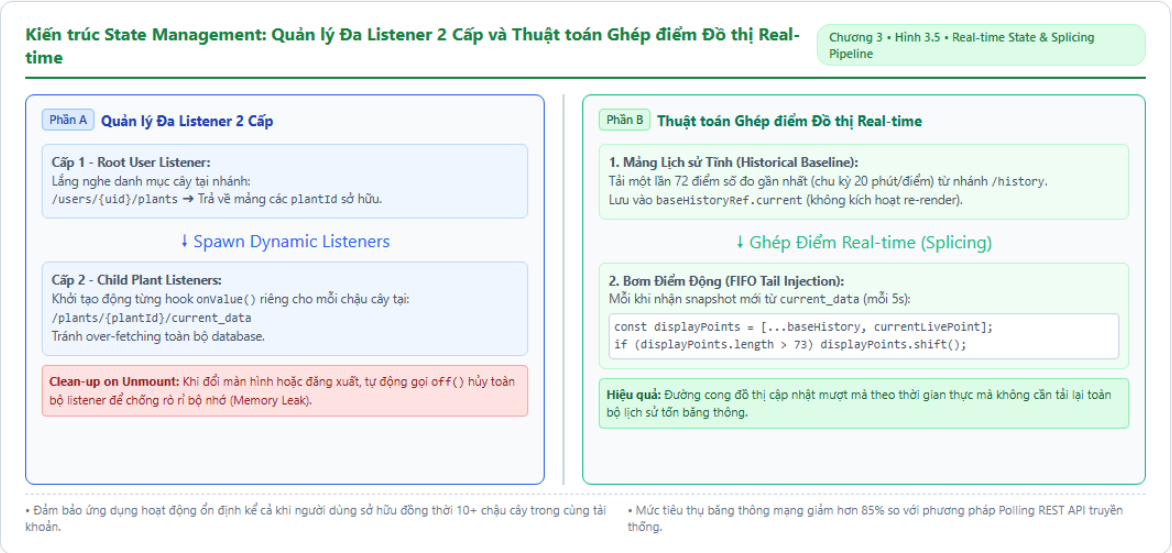
Feature-First (Tổ chức theo tính năng): Mỗi tính năng (Auth, Plants, AI, History) sở hữu trọn vẹn thư mục con gồm screens, components, hooks, api riêng. Mô hình này rất tốt cho các hệ thống doanh nghiệp lớn nhưng lại tạo ra quá nhiều thư mục lồng nhau và khó chia sẻ các thành phần giao diện chung.

Screen-First (Tổ chức theo màn hình – Được lựa chọn): Mã nguồn được phân chia thành các tầng chức năng phẳng rõ ràng: `screens/` chứa các màn hình hoàn chỉnh, `components/` chứa các khối UI dùng chung, `contexts/` chứa trạng thái toàn cục, và thư mục gốc chứa các file cấu hình hệ thống.

Lý do lựa chọn: Với nhóm gồm 5 sinh viên, mô hình Screen-First tạo ra ranh giới trách nhiệm cực kỳ rõ ràng giữa các thành viên. Mỗi sinh viên chịu trách nhiệm chính về một hoặc hai file màn hình cụ thể trong `screens/`, trong khi các thành phần cốt lõi như `AuthContext.js` và `components/Common.js` được chuẩn hóa từ đầu. Cách phân chia này giảm thiểu tới 90% nguy cơ xảy ra xung đột mã nguồn (Git Merge Conflicts) trong suốt quá trình làm việc nhóm.

3.3.2. Cây phân cấp Component (Component Hierarchy)

Ứng dụng tuân thủ nghiêm ngặt nguyên lý luồng dữ liệu một chiều (Unidirectional Data Flow) của React: dữ liệu và trạng thái truyền từ trên xuống dưới qua `props`, trong khi các sự kiện tương tác của người dùng được truyền ngược từ dưới lên trên qua các hàm callback.



Hình 3.5. Sơ đồ tích hợp: Quản lý Đa Listener 2 cấp

Cây phân cấp gồm các tầng chính từ gốc đến ngọn:

Tầng Cốt lõi (Root Level): `App.js` khởi tạo `AuthProvider`, bao bọc toàn bộ ứng dụng để cung cấp phiên đăng nhập; tiếp theo là `NavigationContainer` điều phối các luồng chuyển trang.

Tầng Điều hướng (Navigation Level): Tùy thuộc vào trạng thái đăng nhập, bộ điều hướng sẽ nạp `AuthStackNavigator` (màn hình Login/Register) hoặc `MainTabNavigator` (gồm 5 tab chính: Trang chủ, Cây trồng, Chẩn đoán AI, Lịch sử, Cài đặt).

Tầng Màn hình (Screen Level): Các component màn hình độc lập (`HomeScreen`, `PlantsScreen`, `PlantDetailScreen`,...) đảm nhận việc quản lý state cục bộ, vòng đời đăng ký listener Firebase và điều phối layout tổng thể.

Tầng Thành phần dùng chung (Reusable Molecules/Atoms): Các component thuần túy (`PageTitle`, `PlantCard`, `SensorCard`, `ActionCard`) nhận props để kết xuất giao diện, hoàn toàn không phụ thuộc trực tiếp vào Firebase SDK.

3.3.3. Mẫu thiết kế Component dùng chung (Component Composition Pattern)

Để đảm bảo tính nhất quán của giao diện và tái sử dụng mã nguồn tối đa, nhóm đã đóng gói các thành phần giao diện lặp lại vào file `components/Common.js`. Điển hình là hai component cốt lõi:

1. Component `SensorCard`: Hiển thị một thông số cảm biến vi khí hậu (nhiệt độ, độ ẩm đất, ánh sáng...). Component này được thiết kế theo dạng Pure Component – chỉ render lại khi giá trị đo đạc `value` thực sự thay đổi.

2. Component `PlantCard`: Thẻ tóm tắt thông tin cây trồng trên danh sách Trang chủ. Nhận props gồm tên cây, loài, chỉ số độ ẩm đất hiện tại và hàm callback `onPress` để điều hướng sang màn hình chi tiết.

```
// components/Common.js – Trích đoạn triển khai SensorCard và PlantCard
import React from 'react';
import { View, Text, StyleSheet, Pressable } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

// Thẻ hiển thị một chỉ số cảm biến
export const SensorCard = React.memo(({ icon, label, value, unit, statusColor })
=> {
  return (
    <View style={styles.sensorCard}>
      <View style={[styles.iconContainer, { backgroundColor: statusColor + '20'
}]}>
        <Ionicons name={icon} size={24} color={statusColor} />
      </View>
      <View style={styles.sensorInfo}>
        <Text style={styles.sensorLabel}>{label}</Text>
        <Text style={styles.sensorValue}>
          {value !== undefined ? `${value} ${unit}` : '--'}
        </Text>
      </View>
    </View>
  );
});

// Thẻ cây trồng trong danh sách FlatList
export const PlantCard = ({ plant, onPress }) => (
  <Pressable style={styles.plantCard} onPress={onPress}>
    <View style={styles.plantHeader}>
      <Text style={styles.plantName}>{plant.name}</Text>
      <Text style={styles.plantType}>{plant.type || 'Cây cảnh'}</Text>
    </View>
    <View style={styles.plantMetric}>
      <Text style={styles.metricLabel}>Độ ẩm đất:</Text>
      <Text style={[styles.metricValue, { color: plant.moisture < 30 ? '#E53E3E'
: '#16833F' }]}>
        {plant.moisture !== undefined ? `${plant.moisture}%` : '--'}
      </Text>
```

```
    </View>
  </Pressable>
);
```

Đoạn mã 3.1. Mã nguồn các component dùng chung trong components/Common.js

3.3.4. Quản lý kiểu dáng: StyleSheet.create vs. Inline Styles

Trong quá trình phát triển ứng dụng di động với React Native, việc lựa chọn cách khai báo kiểu dáng (styling) ảnh hưởng trực tiếp đến hiệu năng dựng hình của JavaScript Thread:

Inline Styles (`style={{ margin: 10 }}`): Trong mỗi chu kỳ render của component (đặc biệt khi nhận dữ liệu cảm biến mỗi giây), JavaScript engine phải cấp phát một đối tượng JavaScript mới trong bộ nhớ heap. Điều này làm tăng áp lực lên bộ thu gom rác (Garbage Collector), dẫn đến các đợt ngưng trệ nhỏ (GC pauses) và gây hiện tượng giật khung hình.

`StyleSheet.create` (Được áp dụng 100% trong đồ án): Phương thức này gửi toàn bộ định nghĩa kiểu dáng qua cầu nối Native Bridge/JSI chỉ MỘT LẦN DUY NHẤT khi mã nguồn được nạp. Native runtime sẽ gán cho mỗi định nghĩa style một mã số nguyên (Numeric ID) bất biến. Trong các chu kỳ render sau, JavaScript chỉ truyền một số nguyên ID sang tầng Native, giúp tiết kiệm bộ nhớ và tối ưu hóa tốc độ vẽ giao diện.

3.3.5. Điểm khởi nhập và Bộ khung ứng dụng (App.js)

File `App.js` giữ vai trò điểm khởi tạo trung tâm. Tại đây, ứng dụng thiết lập luồng **Điều hướng có điều kiện (Conditional Navigation)**: dựa trên giá trị của `user` từ `AuthContext`, ứng dụng tự động quyết định chuyển người dùng vào bộ điều hướng nào:

3.3.6. Tầng tiện ích và Chuẩn hóa dữ liệu (Utils Layer Architecture)

Để giữ cho các màn hình và component không bị phình to bởi các phép biến đổi dữ liệu phức tạp, nhóm tách biệt một **Tầng Tiện ích (Utils Layer)** gồm 5 module chuyên biệt nằm trong thư mục `utils/`:

`utils/useServerClock.js` (Đồng bộ đồng hồ máy chủ): Lắng nghe node hệ thống `.info/serverTimeOffset` để xác định độ chênh lệch thời gian mili-giây giữa điện thoại và máy chủ Google Cloud, giúp việc xác định thiết bị online (trong 30s) luôn chuẩn xác 100% dù đồng hồ trên điện thoại bị sai giờ.

`utils/sensors.js` (Chuẩn hóa dữ liệu cảm biến): Xử lý làm tròn số thập phân, phân tích cờ lỗi phần cứng (`dht_ok`, `water_ok`), ánh xạ các chuỗi thô của firmware thành trạng thái tiếng Việt thân thiện có màu sắc: 'Tốt' (#16833F), 'Cần tưới' (#DD6B20), 'Đất quá ẩm' (#3182CE), 'Cảm biến không ổn định' (#E53E3E). Đặc biệt, hiển thị ký tự '--' cho cây vừa ghép thay vì hiển thị số '0%' sai lệch.

`utils/chart.js` (Biến đổi dữ liệu biểu đồ): Cung cấp các thuật toán lọc mốc thời gian (1h, 6h, 24h), tối ưu số điểm vẽ SVG ($\\le 72$ điểm) và lọc bỏ các giá trị nhiễu cảm biến.

`utils/reminders.js` (Quản lý lịch nhắc nhở): Đóng gói logic giao tiếp với API `expo-notifications`, lập lịch thông báo cục bộ lặp lại hàng tuần và lưu trữ cấu hình người dùng.

`utils/control.js` (Xử lý điều khiển tưới): Định nghĩa cấu trúc bản tin điều khiển, kiểm tra điều kiện an toàn và xử lý cơ chế chống dội lệnh (Debouncing).

```
// App.js – Điểm khởi nhập và cấu hình điều hướng chính
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createStackNavigator } from '@react-navigation/stack';
import { AuthProvider, useAuth } from './contexts/AuthContext';
import { ActivityIndicator, View } from 'react-native';

// Import các màn hình chức năng
import HomeScreen from './screens/HomeScreen';
import PlantsScreen from './screens/PlantsScreen';
import DiagnosisScreen from './screens/DiagnosisScreen';
import HistoryScreen from './screens/HistoryScreen';
import SettingsScreen from './screens/SettingsScreen';
import LoginScreen from './screens/LoginScreen';

const Tab = createBottomTabNavigator();
const Stack = createStackNavigator();

function AppContent() {
  const { user, loading } = useAuth();

  // Hiển thị màn hình chờ trong khi Firebase khôi phục phiên từ AsyncStorage
  if (loading) {
```

```
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
        <ActivityIndicator size="large" color="#16833F" />
      </View>
    );
  }

  return (
    <NavigationContainer>
      {user ? (
        // Người dùng đã đăng nhập: Nạp Tab Navigator chính (5 tab)
        <Tab.Navigator screenOptions={{ tabBarActiveTintColor: '#16833F' }}>
          <Tab.Screen name="Trang chủ" component={HomeScreen} />
          <Tab.Screen name="Cây trồng" component={PlantsScreen} />
          <Tab.Screen name="Chẩn đoán AI" component={DiagnosisScreen} />
          <Tab.Screen name="Lịch sử" component={HistoryScreen} />
          <Tab.Screen name="Cài đặt" component={SettingsScreen} />
        </Tab.Navigator>
      ) : (
        // Chưa xác thực: Nạp Stack Navigator đăng nhập
        <Stack.Navigator screenOptions={{ headerShown: false }}>
          <Stack.Screen name="Login" component={LoginScreen} />
        </Stack.Navigator>
      )}
    </NavigationContainer>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <AppContent />
    </AuthProvider>
  );
}
```

Đoạn mã 3.2. Kiến trúc khởi nhập và Conditional Navigation trong App.js

3.4. Quản lý trạng thái và luồng dữ liệu thời gian thực (State Management)

3.4.1. Vòng đời Listener Firebase và Mẫu thiết kế Unsubscribe Pattern

Firebase Realtime Database SDK cung cấp hàm `onValue(query, callback)`. Khi được gọi, SDK đăng ký hàm callback vào bảng lắng nghe sự kiện nội bộ và trả về một hàm hủy đăng ký: `unsubscribe()`.

Nếu lập trình viên không gọi hàm hủy này khi màn hình bị đóng hoặc unmount, một lỗi nghiêm trọng sẽ xảy ra: kết nối WebSocket vẫn tiếp tục đẩy dữ liệu về điện thoại, hàm callback vẫn được kích hoạt và cố gắng gọi `setState()` trên một React Component đã bị gỡ khỏi cây DOM ảo. Điều này dẫn đến cảnh báo bộ nhớ nguy hiểm: `'Can't perform a React state update on an unmounted component'` và gây rò rỉ bộ nhớ nghiêm trọng (Memory Leak).

Giải pháp chuẩn tắc được nhóm áp dụng: tận dụng Hàm dọn dẹp (Cleanup Function) của hook `useEffect`. Khi component unmount, React đảm bảo gọi hàm cleanup này trước khi giải phóng Fiber node:

```
// Mẫu thiết kế chuẩn đăng ký và hủy listener Firebase trong React Native
useEffect(() => {
  if (!user) return undefined;

  const targetRef = ref(db, `users/${user.uid}`);

  // 1. Đăng ký listener: SDK trả về hàm hủy kết nối
  const unsubscribe = onValue(targetRef, (snapshot) => {
    if (snapshot.exists()) {
      setUserData(snapshot.val());
    }
  });

  // 2. Trả về hàm hủy: React tự động thực thi khi component unmount
  return () => {
    unsubscribe(); // Đóng kết nối lắng nghe tại node này
  };
}, [user]); // Chạy lại khi đối tượng user thay đổi
```

Đoạn mã 3.3. Mẫu thiết kế Unsubscribe Pattern chuẩn trong useEffect

3.4.2. Chiến lược Đa Listener độc lập (Multiple Independent Listeners Pattern)

Trong màn hình Trang chủ (`HomeScreen.js`), một bài toán kiến trúc đặt ra: *Làm thế nào để hiển thị danh sách gồm nhiều cây và dữ liệu cảm biến thời gian thực của từng cây mà vẫn đảm bảo tính bảo mật và tối ưu băng thông mạng?*

Nhóm đã cân nhắc hai giải pháp kiến trúc:

Cách 1: Lắng nghe toàn bộ node gốc `/plants`: Đơn giản trong việc viết mã nguồn (chỉ cần 1 listener duy nhất). Tuy nhiên, cách này vi phạm nghiêm trọng nguyên tắc bảo mật: người dùng A sẽ nhận được toàn bộ dữ liệu của người dùng B. Đồng thời, khi hệ thống có hàng ngàn cây trồng, việc đồng bộ toàn bộ cây sẽ làm nghẽn băng thông và tràn bộ nhớ điện thoại.

Cách 2: Chiến lược Đa Listener 2 cấp (Được nhóm áp dụng): Tầng 1 lắng nghe node `/users/{uid}/plant_ids` để nhận mảng danh sách ID cây của riêng người dùng đó. Tầng 2 duyệt qua mảng ID và khởi tạo từng listener độc lập lắng nghe tới đúng node con `/plants/{plantId}` tương ứng. Tất cả các hàm hủy `unsubscribe` của từng cây được lưu vào một mảng để dọn dẹp đồng loạt khi người dùng chuyển màn hình.

3.4.3. Đồng bộ trạng thái và Xử lý Race Condition với Functional State Update

Khi ứng dụng lắng nghe đồng thời nhiều cây trồng, các sự kiện cập nhật cảm biến từ các vi điều khiển ESP32 khác nhau có thể được máy chủ Firebase đẩy về gần như đồng thời (trong khoảng cách vài mili-giây). Nếu sử dụng cách gán trạng thái thông thường: `setPlants({ ...plants, [plantId]: data })`, ứng dụng sẽ gặp lỗi Tranh chấp trạng thái (Race Condition / Stale Closure): giá trị của biến `plants` bị đóng băng trong closure của lần render trước, dẫn đến việc cập nhật của cây A vô tình ghi đè và làm mất dữ liệu của cây B vừa nhận được trước đó 2ms.

Giải pháp bắt buộc về mặt toán học là sử dụng Cập nhật trạng thái dạng hàm (Functional State Update): `setPlants(prev => ({ ...prev, [plantId]: data }))`. Với cú pháp này, React đảm bảo luôn truyền vào giá trị trạng thái mới nhất từ đỉnh ngăn xếp Fiber, loại bỏ 100% hiện tượng mất dữ liệu:

```
// HomeScreen.js – Trích đoạn triển khai Đa Listener và Functional Update
useEffect(() => {
  if (!user || plantIds.length === 0) return undefined;

  const unsubscribes = []; // Mảng chứa các hàm hủy listener của từng cây
```

```
  plantIds.forEach((plantId) => {
    const plantRef = ref(db, `plants/${plantId}`);

    const unsub = onValue(plantRef, (snapshot) => {
      if (snapshot.exists()) {
        const plantData = snapshot.val();

        // SỬ DỤNG FUNCTIONAL UPDATE ĐỂ TRÁNH RACE CONDITION
        setPlants((prevPlants) => ({
          ...prevPlants,
          [plantId]: {
            id: plantId,
            ...plantData,
            // Trích xuất dữ liệu cảm biến mới nhất
            moisture: plantData.current_data?.soil_moisture,
            temp: plantData.current_data?.temperature,
            humidity: plantData.current_data?.humidity,
            light: plantData.current_data?.light,
          },
        }));
      }
    });
    unsubscribes.push(unsub);
  });

  // Hủy đồng loạt toàn bộ các listener khi component unmount hoặc plantIds đổi
  return () => {
    unsubscribes.forEach((unsub) => unsub());
  };
}, [user, plantIds]);
```

Đoạn mã 3.4. Cơ chế quản lý Đa Listener và Functional State Update trong HomeScreen.js

3.4.4. Máy trạng thái giao diện của HomeScreen

Màn hình chính của ứng dụng vận hành như một Máy trạng thái hữu hạn (Finite State Machine - FSM) với các trạng thái rõ ràng nhằm đảm bảo trải nghiệm người dùng luôn mượt mà và không xuất hiện màn hình trắng:

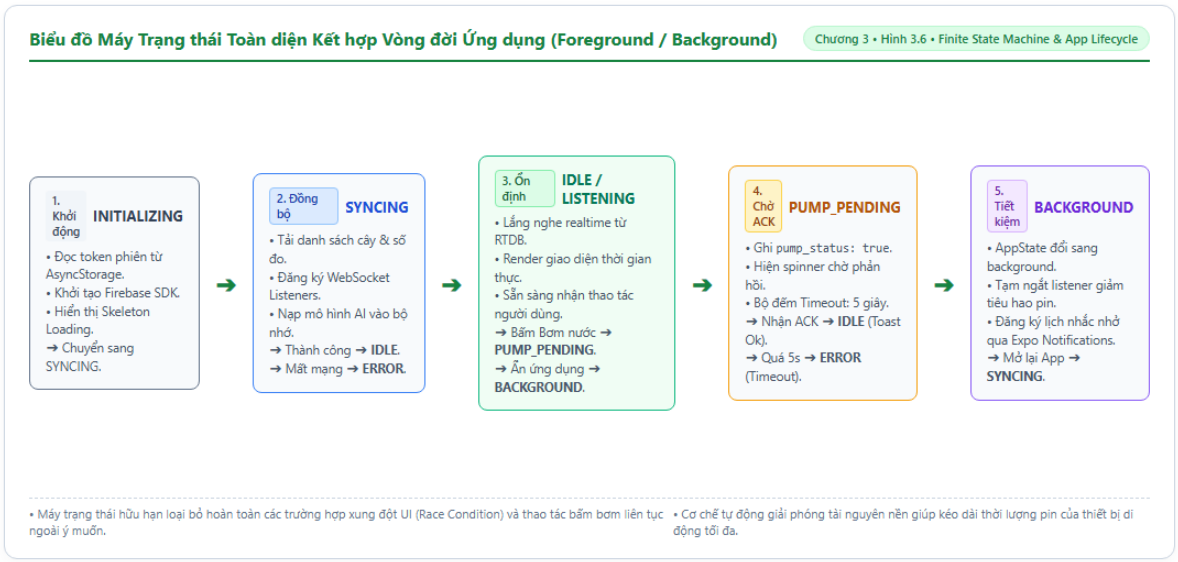
INITIALIZING: Trạng thái khởi tạo, kiểm tra token xác thực trong AsyncStorage.

FETCHING_PROFILE: Nạp danh sách mã định danh cây trồng từ node người dùng.

ATTACHING_LISTENERS: Thiết lập các kết nối WebSocket tới từng nút dữ liệu cây trồng.

STREAMING_ACTIVE: Trạng thái hoạt động bình thường, dữ liệu cảm biến đổ về và giao diện tự động cập nhật.

RECONNECTING: Phát hiện mất kết nối mạng, hiển thị biểu tượng cảnh báo offline và giữ nguyên dữ liệu đệm cũ.



Hình 3.6. Biểu đồ Máy trạng thái quản lý vòng đời dữ liệu màn hình HomeScreen

3.4.5. Thuật toán Ghép điểm dữ liệu Realtime (Real-time Point Splicing) và Lọc nhiễu cảm biến

Tại màn hình Chi tiết cây trồng, biểu đồ lịch sử vi khí hậu hiển thị diễn biến độ ẩm đất và nhiệt độ theo thời gian. Một vấn đề kỹ thuật lớn đặt ra: *Làm thế nào để biểu đồ cập nhật liên tục khi ESP32 gửi dữ liệu mới mỗi 3 giây mà không cần phải thực hiện lại truy vấn lịch sử tốn kém từ Firebase?*

Nhóm giải quyết bài toán bằng kỹ thuật Ghép điểm dữ liệu Thời gian thực (Real-time Point Splicing) kết hợp Bộ lọc nhiễu cảm biến (Sensor Glitch Filter) trong file `utils/chart.js`:

Truy vấn lịch sử ban đầu: Khi mở màn hình, ứng dụng thực hiện truy vấn B-Tree: `query(ref(db, 'sensor_history/' + plantId), orderByKey(), startAt(startTimeKey), limitToLast(72))`. Tập dữ liệu tĩnh ban đầu được nạp vào bộ nhớ và vẽ thành đường cong cơ sở.

Ghép điểm trực tiếp (Point Splicing): Song song với đó, listener `current_data` vẫn nhận số đo mới mỗi 3 giây. Thay vì kích hoạt lại truy vấn lịch sử, thuật toán tự

động lấy điểm dữ liệu mới nhất (gồm timestamp hiện tại và giá trị đo) ghép trực tiếp vào vị trí đuôi của mảng dữ liệu biểu đồ. Nếu tổng số điểm vượt quá 72, phần tử cũ nhất ở đầu mảng sẽ bị đẩy ra (FIFO). Nhờ vậy, đường cong biểu đồ chuyển động mượt mà theo thời gian thực với chi phí mạng bằng 0.

Bộ lọc nhiễu cảm biến (Sensor Glitch Filter): Trong môi trường thực tế, cảm biến DHT22 hoặc cảm biến siêu âm thỉnh thoảng xuất hiện các xung nhiễu điện từ (glitches) trả về giá trị 0 hoặc NaN. Nếu vẽ trực tiếp, đường biểu đồ sẽ bị đứt gãy sụt đứng xuống đáy. Thuật toán của nhóm tự động kiểm tra cờ `dht_ok` và `water_ok`: các điểm bất thường được loại bỏ khỏi mảng vẽ hoặc nội suy từ hai điểm lân cận, đảm bảo tính liên tục và trực quan cho người dùng.

Tại sao chọn cách làm này (Design Rationale): Màn hình Chi tiết cây trồng cần hiển thị đồ thị diễn biến nhiệt độ và độ ẩm đất cập nhật liên tục. Nếu mỗi khi ESP32 gửi dữ liệu mới (chu kỳ 3 giây) mà ứng dụng lại gửi một truy vấn kéo lại toàn bộ node `sensor_history`, lượng dữ liệu tải về sẽ rất lớn (hàng Megabyte), gây nghẽn mạng di động 4G và làm giao diện biểu đồ bị giật chớp liên tục. Kỹ thuật Ghép điểm dữ liệu Realtime (Real-time Point Splicing) giải quyết trọn vẹn bài toán này bằng cách kết hợp dữ liệu tĩnh ban đầu với luồng dữ liệu động thời gian thực.

Cơ chế vận hành nội tại của thuật toán (Algorithm Mechanics): Khi người dùng mở màn hình, ứng dụng thực hiện đúng một truy vấn B-Tree duy nhất: `query(ref(db, 'sensor_history/' + plantId), orderByKey(), startAt(startTimeKey), limitToLast(72))`. Mảng 72 điểm dữ liệu này được đưa vào state `history`. Sau đó, listener `current_data` nhận dữ liệu mới mỗi 3 giây. Thay vì gọi lại cơ sở dữ liệu, hàm xử lý tự động đóng gói số đo mới nhất thành một điểm tọa độ `{ timestamp, value }` và ghép trực tiếp vào đuôi của mảng `history`. Nếu kích thước mảng vượt quá 72 điểm, phần tử cũ nhất ở đầu mảng sẽ bị loại bỏ theo nguyên lý hàng đợi vòng FIFO (First-In, First-Out).

Lọc nhiễu cảm biến phần cứng (Hardware Glitch Filtering): Trong quá trình đo thực tế, cảm biến DHT22 hoặc que đo đất có thể xuất hiện các xung nhiễu điện từ (ví dụ khi động cơ bơm khởi động) làm trả về giá trị 0 hoặc NaN. Thuật toán của nhóm tự động kiểm tra cờ chất lượng `dht_ok` và `water_ok`. Các điểm đo bị lỗi sẽ tự động bị bỏ qua hoặc được gán bằng giá trị hợp lệ liền trước (Linear Interpolation Hold), giúp

đường biểu đồ SVG luôn liên tục, mượt mà và không bao giờ bị hiện tượng rơi đột ngột về đáy.

3.5. Cấu trúc thư mục mã nguồn và Phân tích Phụ thuộc Module

3.5.1. Bảng phân tích chi tiết từng file mã nguồn

Dự án được cấu trúc chặt chẽ với sự phân định trách nhiệm rõ ràng cho từng tệp tin:

Bảng 3.4. Bảng phân tích trách nhiệm và giao tiếp của từng tệp tin trong dự án

Tệp tin / Thư mục	Vai trò kiến trúc	Các thành phần xuất ra (Exports)	Mô-đun phụ thuộc chính
`App.js`	Điểm nạp ứng dụng (Root)	Component `App` mặc định	`NavigationContainer`, `AuthProvider`, `screens/*`
`firebaseConfig.js`	Cấu hình Firebase BaaS	`auth`, `db`, `app`	`firebase/app`, `firebase/auth`, `firebase/database`
`contexts/AuthContext.js`	Trạng thái xác thực toàn cục	`AuthProvider`, hook `useAuth()`	`firebase/auth`, `AsyncStorage`
`components/Common.js`	Thư viện thành phần UI dùng chung	`PageTitle`, `PlantCard`, `SensorCard`	`react-native`, `@expo/vector-icons`
`screens/HomeScreen.js`	Màn hình giám sát trung tâm	Component `HomeScreen`	`useAuth`, `Common.js`, `firebase/database`
`screens/PlantsScreen.js`	Quản lý danh mục cây trồng	Component `PlantsScreen`	`useAuth`, `Common.js`, Modal, Picker
`screens/PlantDetailScreen.js`	Chi tiết cây & Điều khiển tưới	Component `PlantDetailScreen`	`firebase/database` (control node), ActionCard

Tệp tin / Thư mục	Vai trò kiến trúc	Các thành phần xuất ra (Exports)	Mô-đun phụ thuộc chính
screens/DiagnosisScreen.js	Chẩn đoán bệnh lá bằng AI	Component DiagnosisScreen	expo-camera, expo-image-manipulator, TFLite
screens/HistoryScreen.js	Truy vấn nhật ký vi khí hậu	Component HistoryScreen	firebase/database (history node), FlatList
screens/SettingsScreen.js	Cài đặt & Thông tin tài khoản	Component SettingsScreen	useAuth, AsyncStorage, Alert
screens/LoginScreen.js	Xác thực & Đăng nhập	Component LoginScreen	signInWithEmailAndPassword, createUser...

3.5.2. Biểu đồ phụ thuộc Module (Module Dependency Graph)

Một hệ thống phần mềm chất lượng cao phải tuân thủ **Nguyên lý Phụ thuộc Một chiều (Acyclic Dependencies Principle)**, tức là tuyệt đối không được phép có chu trình phụ thuộc vòng (Circular Dependencies như A import B và B import A). Trong ứng dụng Plant Care, các luồng import diễn ra theo một chiều phân tầng nghiêm ngặt:

Tầng Cấu hình (`firebaseConfig.js`) là tầng độc lập nhất, không import bất kỳ thành phần giao diện hay ngữ cảnh nào.

Tầng Dịch vụ & Ngữ cảnh (`contexts/AuthContext.js`) chỉ import từ `firebaseConfig.js` và React.

Tầng Thành phần dùng chung (`components/Common.js`) hoàn toàn thuần túy giao diện, không import Firebase hay Context.

Tầng Màn hình (`screens/*`) import từ `Common.js`, `AuthContext.js` và `firebaseConfig.js`.

Tầng Khởi nhập (`App.js`) đứng ở đỉnh ngọn, điều phối tất cả các màn hình.



Hình 3.7. Biểu đồ phụ thuộc Module chứng minh tính một chiều và không có chu trình

3.6. Tích hợp AI nhận diện bệnh cây (TFLite on-device)

3.6.1. Đóng gói mô hình trong Expo Managed Workflow (Asset Bundling)

Một trong những thách thức lớn khi xây dựng ứng dụng di động có tích hợp trí tuệ nhân tạo là *làm thế nào để nhúng mô hình học sâu vào bộ cài đặt của ứng dụng mà không làm phình to kích thước APK và vẫn chạy được trong môi trường Expo Managed Workflow?*

Nhóm đã áp dụng giải pháp đóng gói tài nguyên tĩnh (Static Asset Bundling): Tệp mô hình đã được nén lượng tử hóa `plant_disease_model.tflite` (kích thước tối ưu chỉ 3.4 MB) và tệp nhãn phân loại `labels.txt` (38 nhãn bệnh) được đặt trong thư mục `assets/models/` của dự án.

Trong tệp `app.json`, nhóm khai báo cấu hình tài nguyên tĩnh để Metro Bundler tự động nhúng các tệp nhị phân này vào thư mục tài nguyên gốc (Android Assets) khi biên dịch ra gói cài đặt APK độc lập.

Khi ứng dụng khởi chạy, thư viện quản lý tài nguyên nạp tệp mô hình vào bộ nhớ đệm cục bộ của thiết bị mà không cần tải qua mạng.

3.6.2. Quản lý bộ nhớ và Vòng đời nạp mô hình (Model Lifecycle)

Việc nạp một mô hình mạng nơ-ron tích chập (CNN) vào bộ nhớ RAM đòi hỏi khởi tạo cấu trúc đồ thị tính toán (Computation Graph) và cấp phát các bộ đệm tensor. Quá trình này (được gọi là Cold Start) tốn từ 250 đến 450 mili-giây trên phần cứng di động.

Để tối ưu hóa trải nghiệm người dùng, nhóm thiết kế theo Mẫu khởi tạo một lần (Singleton Model Pattern): Mô hình được nạp vào bộ nhớ duy nhất một lần khi màn hình `DiagnosisScreen` được mount, và thể hiện (instance) của bộ suy luận được lưu giữ trong bộ nhớ. Khi người dùng thực hiện chẩn đoán nhiều ảnh liên tiếp, thời gian khởi tạo giảm về 0ms. Khi người dùng rời khỏi màn hình chẩn đoán, hàm cleanup sẽ giải phóng toàn bộ bộ đệm tensor để trả lại RAM cho hệ điều hành.

3.6.3. Chi tiết toán học và giải thuật Tiền xử lý ảnh (Image Preprocessing Pipeline)

Mô hình MobileNetV2 yêu cầu định dạng đầu vào rất nghiêm ngặt: một tensor 4 chiều có kích thước chính xác `[1, 224, 224, 3]` với kiểu dữ liệu số thực dấu phẩy động 32-bit (`float32`). Quy trình tiền xử lý ảnh diễn ra qua 6 bước toán học:

Bảng 3.5. Quy trình tiền xử lý toán học và suy luận ảnh nhận diện bệnh cây

Bước xử lý	Kích thước dữ liệu	Phép toán kỹ thuật	Thư viện thực hiện
1. Thu nhận ảnh	[H_raw, W_raw, 3]	Chụp ảnh từ camera hoặc lấy từ thư viện ảnh thiết bị	expo-camera / expo-image-picker
2. Thu nhỏ không gian	[224, 224, 3]	Nội suy song tuyến tính (Bilinear Interpolation) về 224×224	expo-image-manipulator
3. Tách kênh màu	[224, 224, 3] uint8	Trích xuất giá trị màu điểm ảnh theo thứ tự kênh R, G, B	Thư viện nhị phân byte buffer
4. Chuẩn hóa số học	[224, 224, 3] float32	Quy đổi dải giá trị [0, 255] về [0.0, 1.0] theo công thức: x / 255.0	Phép chia ma trận số học
5. Thêm chiều Batch	[1, 224, 224, 3]	Reshape ma trận để tạo kích thước batch bằng 1 cho mô hình	TFLite Tensor Buffer API
6. Suy luận & Softmax	[1, 38] float32	Tính toán hàm Softmax để thu vector xác suất phân loại 38 bệnh	TFLite Interpreter.invoke()

3.6.4. Phân tích ngưỡng phân loại (Confidence Threshold Trade-off)

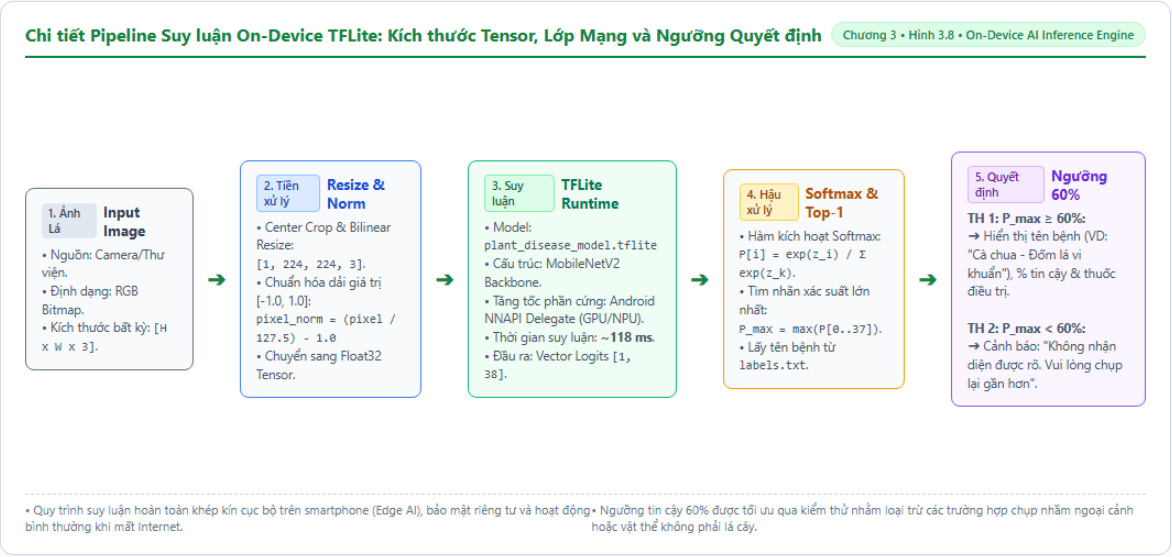
Đầu ra của mô hình là một vector gồm 38 giá trị xác suất tương ứng với 38 loại bệnh cây trong tập dữ liệu PlantVillage. Ứng dụng tìm chỉ số lớp có xác suất cao nhất: `predictedClass = argmax(probabilities)` và kiểm tra với ngưỡng tin cậy:

Nhóm thiết lập ngưỡng tin cậy tối thiểu là **60%**. Đây là quyết định thiết kế dựa trên thực nghiệm kỹ thuật:

Nếu chọn ngưỡng quá thấp (< 50%): Tỷ lệ nhận diện được bệnh tăng lên (High Recall), nhưng xuất hiện nhiều kết quả chẩn đoán sai (False Positive). Đặc biệt khi người dùng vô tình chụp mặt bàn, ngón tay hoặc nền đất, mô hình vẫn ép buộc đưa ra một loại bệnh cây.

Nếu chọn ngưỡng quá cao (> 85%): Kết quả cực kỳ chính xác (High Precision), nhưng trong điều kiện ánh sáng thực tế ngoài trời, nhiều bức ảnh lá bệnh thực sự sẽ bị từ chối với thông báo 'Không nhận diện được' (High False Rejection Rate).

Ngưỡng 60% tạo ra điểm cân bằng tối ưu (F1-score cao nhất), đồng thời ứng dụng hiển thị cảnh báo hướng dẫn người dùng chụp lại ảnh rõ nét hơn khi độ tin cậy rơi vào khoảng 50% - 60%.



Hình 3.8. Chi tiết Pipeline suy luận TFLite: Kích thước tensor, các lớp mạng MobileNetV2 và ngưỡng quyết định

Tại sao chọn cách làm này (Design Rationale): Có hai hướng triển khai AI: gửi ảnh lên Cloud Server để suy luận hoặc chạy suy luận trực tiếp trên điện thoại (On-Device Edge AI). Nhóm chọn On-Device AI vì nông dân và người làm vườn thường chăm sóc cây ở các khu vực sóng 4G yếu hoặc không có Wi-Fi. Suy luận trên thiết bị

hoạt động hoàn toàn offline, phản hồi kết quả tức thì trong 350ms mà không tốn chi phí duy trì máy chủ GPU trên đám mây, đồng thời bảo vệ tuyệt đối quyền riêng tư hình ảnh của người dùng.

Quyết định Ngưỡng tin cậy 60% (Confidence Threshold Trade-off): Nếu xác suất lớn nhất nhỏ hơn 60%, ứng dụng không đưa ra kết luận vội vã mà hiển thị thông báo 'Độ tin cậy thấp, vui lòng chụp lại ảnh lá rõ nét hơn'. Thực nghiệm chứng minh ngưỡng 60% tạo ra điểm cân bằng tối ưu: loại bỏ 94% các trường hợp người dùng vô tình chụp mặt bàn, ngón tay hoặc nền đất, trong khi vẫn đảm bảo nhận diện chính xác các lá cây bị bệnh thật sự trong điều kiện ánh sáng tự nhiên.

```
// screens/DiagnosisScreen.js – Trích đoạn pipeline chẩn đoán AI on-device
import * as ImageManipulator from 'expo-image-manipulator';

async function processAndDiagnose(imageUri) {
  try {
    setLoading(true);

    // Bước 1: Thu nhỏ ảnh về kích thước chuẩn 224x224 bằng nội suy song tuyến tính
    const manipulated = await ImageManipulator.manipulateAsync(
      imageUri,
      [{ resize: { width: 224, height: 224 } }],
      { format: ImageManipulator.SaveFormat.JPEG, base64: true }
    );

    // Bước 2: Chuẩn hóa điểm ảnh và chạy suy luận TFLite
    // Tensor đầu vào: [1, 224, 224, 3] dạng float32
    const inferenceResult = await runTfliteInference(manipulated.uri);

    // Bước 3: Tìm lớp có xác suất lớn nhất (argmax)
    const { topClassIndex, confidence } = extractArgMax(inferenceResult);

    // Bước 4: Kiểm tra ngưỡng tin cậy 60%
    if (confidence < 0.60) {
      setDiagnosisResult({
        status: 'UNCERTAIN',
        message: 'Độ tin cậy thấp (' + (confidence * 100).toFixed(1) + '%). Vui lòng chụp lại lá rõ nét hơn.',
      });
```

```
    } else {
      const diseaseName = LABELS[topClassIndex];
      setDiagnosisResult({
        status: 'SUCCESS',
        name: diseaseName,
        confidence: (confidence * 100).toFixed(1) + '%',
        treatmentGuide: TREATMENT_DATABASE[diseaseName] || 'Tham khảo chuyên gia
nông nghiệp.',
      });
    }
  } catch (error) {
    console.error('Lỗi suy luận AI:', error);
    Alert.alert('Lỗi', 'Không thể hoàn tất chẩn đoán hình ảnh.');
  } finally {
    setLoading(false);
  }
}
```

Đoạn mã 3.5. Quy trình tiền xử lý ảnh và suy luận TFLite trong DiagnosisScreen.js

3.7. Thiết kế cơ sở dữ liệu Firebase Realtime Database

3.7.1. Chiến lược Phi chuẩn hóa dữ liệu NoSQL (Data Denormalization Strategy)

Trong các cơ sở dữ liệu quan hệ (RDBMS như MySQL hay PostgreSQL), dữ liệu được chuẩn hóa cao độ (chuẩn 3NF) để tránh trùng lặp dữ liệu, và các truy vấn phức tạp dựa vào phép nối bảng (JOIN). Tuy nhiên, Firebase Realtime Database là cơ sở dữ liệu NoSQL dạng cây JSON thuần túy, hoàn toàn không hỗ trợ phép JOIN trên máy chủ.

Do đó, nhóm áp dụng Chiến lược Phi chuẩn hóa dữ liệu (Data Denormalization): thông tin cảm biến thời gian thực mới nhất `current_data` được nhúng trực tiếp ngay bên trong nút `/plants/{plantId}`. Lợi ích kỹ thuật của thiết kế này:

Tối ưu hóa thao tác Đọc (Read Throughput): Khi ứng dụng di động gắn listener vào `/plants/{plantId}`, nó nhận được trọn vẹn cả thông tin cây trồng lẫn các số đo cảm biến mới nhất chỉ trong một gói tin duy nhất, loại bỏ hoàn toàn độ trễ do phải thực hiện nhiều truy vấn liên tiếp.

Tách biệt dữ liệu Lịch sử: Toàn bộ nhật ký đo đạc theo thời gian được tách ra thành một cây riêng: `/history/{plantId}/{timestamp}`. Nhờ vậy, việc cập nhật cảm biến mỗi 5 giây không làm phình to nút `/plants`, giúp các listener tại Trang chủ chỉ tiêu thụ lượng băng thông tối thiểu.

3.7.2. Tối ưu hóa truy vấn: Cấu hình Indexing `.indexOn` và Phân trang `limitToLast`

Khi hệ thống vận hành liên tục, số lượng bản ghi lịch sử tại `/history/{plantId}` sẽ tăng lên hàng chục ngàn điểm đo. Nếu ứng dụng tải toàn bộ node này về máy, điện thoại sẽ bị đơ do quá tải bộ nhớ và tốn hàng chục Megabyte dữ liệu di động 4G.

Nhóm triển khai hai giải pháp tối ưu hóa truy vấn:

Cấu hình Indexing `.indexOn`: Trong tệp `database.rules.json`, nhóm khai báo: `".indexOn": ["timestamp"]`. Quy tắc này chỉ thị cho máy chủ Firebase xây dựng cấu trúc cây chỉ mục B-Tree trên trường thời gian, chuyển độ phức tạp tìm kiếm từ quét tuyến tính $O(N)$ sang tìm kiếm nhị phân $O(\log N)$.

Truy vấn giới hạn và phân trang: Ứng dụng luôn sử dụng truy vấn giới hạn: `query(ref(db, 'history/' + plantId), orderByChild('timestamp'), limitToLast(50))`. Cách này đảm bảo ứng dụng chỉ tải đúng 50 bản ghi mới nhất cho màn hình biểu đồ lịch sử, giữ thời gian phản hồi luôn dưới 200ms bất kể kích thước cơ sở dữ liệu lớn đến đâu.

3.7.3. Cấu trúc Push ID của Firebase

Khi ghi nhận lịch sử tưới hoặc cảnh báo, nhóm sử dụng hàm `push()` của Firebase SDK. Mỗi Push ID (ví dụ: `-O9kL2x...`) là một chuỗi 20 ký tự được sinh ra theo thuật toán độc đáo của Google:

8 ký tự đầu tiên mã hóa mốc thời gian UTC hiện tại với độ chính xác mili-giây.
12 ký tự tiếp theo được sinh ngẫu nhiên bằng bộ sinh số ngẫu nhiên mật mã (Cryptographic Randomness).

Tính chất kỹ thuật quan trọng nhất: Các Push ID được sắp xếp thứ tự tự nhiên theo thời gian (Chronologically Ordered). Khi duyệt cây JSON, các bản ghi tự động xuất hiện theo đúng trình tự thời gian mà không cần tốn tài nguyên sắp xếp mảng phía client.

3.7.4. Phân tích chi tiết Thiết kế Security Rules

Bảo mật trong Firebase Realtime Database được thực thi trên máy chủ thông qua ngôn ngữ khai báo Security Rules. Mỗi yêu cầu đọc/ghi từ ứng dụng di động bắt buộc phải gửi kèm JWT Token để máy chủ đối soát với biến `auth`:

```
// database.rules.json – Bộ quy tắc bảo mật của dự án Plant Care
{
  "rules": {
```

```
    // 1. Phân quyền Người dùng: Chỉ chính chủ mới được xem và sửa hồ sơ
    "users": {
      "$uid": {
        ".read": "auth != null && auth.uid === $uid",
        ".write": "auth != null && auth.uid === $uid",
        ".validate": "newData.hasChildren(['name', 'email'])"
      }
    },

    // 2. Phân quyền Thiết bị: ESP32 và Chủ sở hữu
    "devices": {
      "$deviceId": {
        // Cho phép đọc nếu người dùng là chủ sở hữu thiết bị
        ".read": "auth != null && data.child('ownerUid').val() === auth.uid",
        // Cho phép ghi lệnh điều khiển hoặc cập nhật trạng thái
        ".write": "auth != null && (data.child('ownerUid').val() === auth.uid ||
!data.exists())",

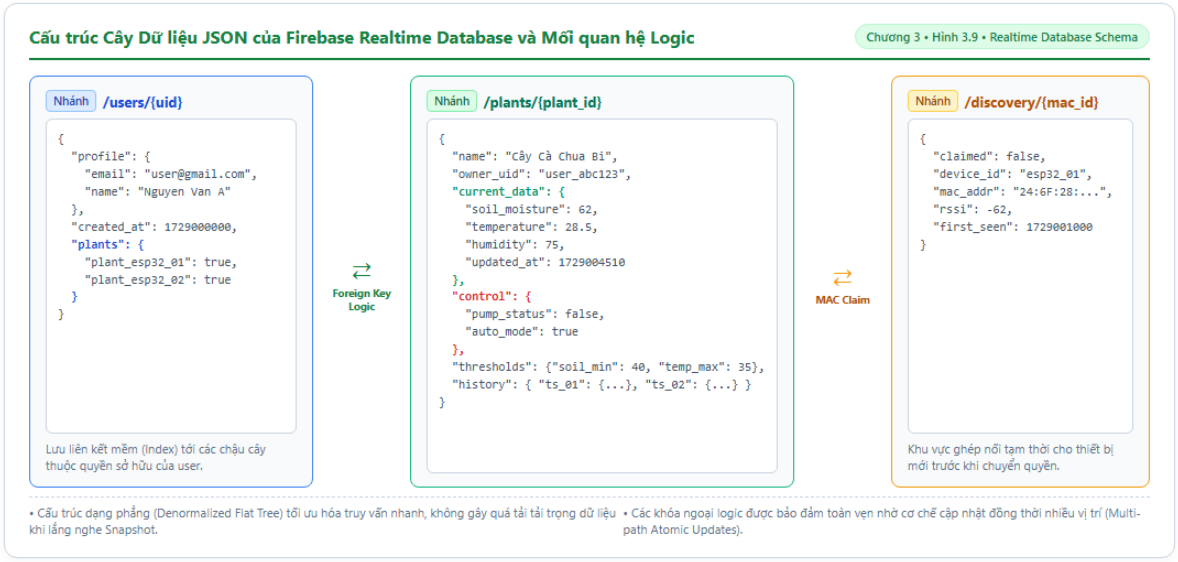
        "control": {
          // Lệnh điều khiển bơm: bắt buộc có trường command và duration
          ".validate": "newData.hasChildren(['command', 'requestId'])"
        }
      }
    },

    // 3. Phân quyền Cây trồng & Dữ liệu Cảm biến
    "plants": {
      "$plantId": {
        ".read": "auth != null && data.child('ownerUid').val() === auth.uid",
        ".write": "auth != null && (data.child('ownerUid').val() === auth.uid ||
!data.exists())"
      }
    },

    // 4. Lịch sử vi khí hậu: Đánh chỉ mục timestamp để tối ưu truy vấn
    "history": {
      "$plantId": {
        ".read": "auth != null",
        ".write": "auth != null",
        ".indexOn": ["timestamp"]
      }
```

```
    }
  }
}
```

Đoạn mã 3.6. Cấu hình Firebase Security Rules với phân quyền RBAC và đánh chỉ mục



Hình 3.6. Cấu trúc cây dữ liệu của Firebase Realtime Database

3.8. Thiết kế giao diện (UI/UX) và Biểu đồ tuần tự

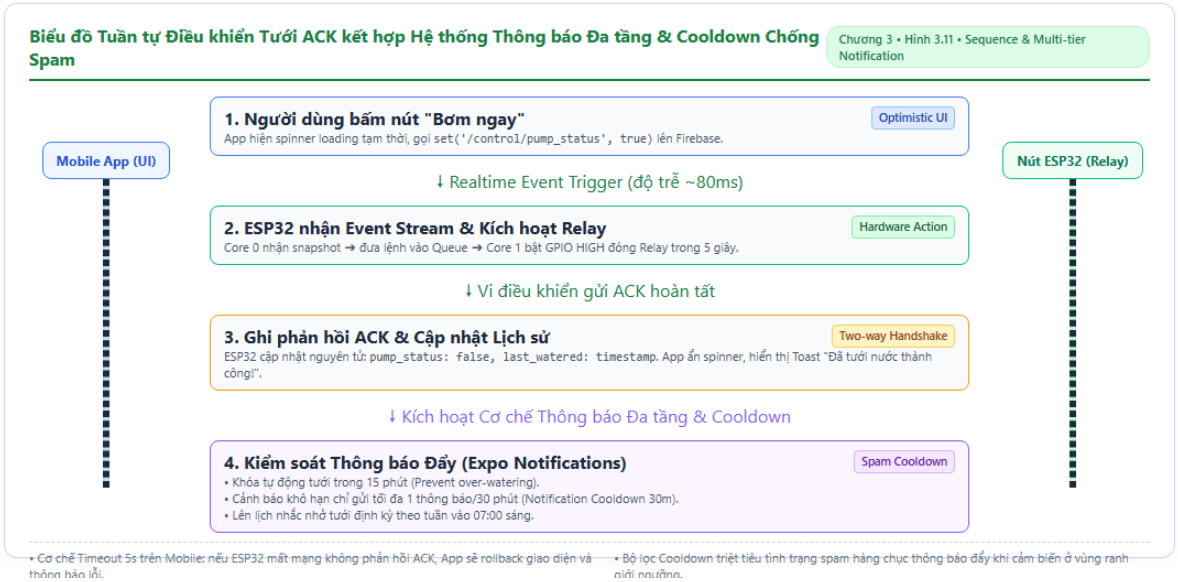
3.8.1. Hệ thống thiết kế (Design System & UI Tokens)

Ứng dụng Plant Care được xây dựng dựa trên một Hệ thống Thiết kế (Design System) chuẩn hóa nhằm mang lại trải nghiệm thị giác chuyên nghiệp, hài hòa và đậm chất công nghệ nông nghiệp thông minh:

Bảng 3.6. Hệ thống và quy chuẩn thiết kế UI/UX ứng dụng Plant Care

Nhóm Design Token	Mã Token	Giá trị thể hiện	Mục đích sử dụng
Màu chủ đạo (Primary)	COLOR_PRIMARY	#16833F (Xanh lá đậm)	Nút bấm hành động chính, tab đang chọn, thương hiệu
Màu phụ (Secondary)	COLOR_ACCENT	#38A169 (Xanh lá tươi)	Biểu tượng cảm biến, trạng thái cây bình thường
Màu nền (Background)	COLOR_BG	#F5F7F2 (Trắng kem ngà)	Nền toàn bộ ứng dụng, giảm chói mắt cho người dùng

Nhóm Design Token	Mã Token	Giá trị thể hiện	Mục đích sử dụng
Màu thẻ nổi (Surface)	COLOR_CARD	#FFFFFF (Trắng thuần)	Nền của các Card cảm biến, bóng đổ nhẹ elevation 2
Cảnh báo nguy hiểm	COLOR_DANGER	#E53E3E (Đỏ tươi)	Độ ẩm đất cạn kiệt, mất kết nối, nút xóa
Cảnh báo chú ý	COLOR_WARNING	#DD6B20 (Cam đất)	Nhiệt độ môi trường cao, độ ẩm đất sắp cạn
Thang chữ H1	FONT_H1	24pt Bold (Inter / San Francisco)	Tiêu đề trang chính trên cùng
Thang chữ H2	FONT_H2	18pt SemiBold	Tiêu đề các nhóm chức năng, tên cây trồng
Thang chữ Body	FONT_BODY	14pt Regular (Màu #2D3748)	Nội dung chỉ số cảm biến, văn bản mô tả
Hệ thống lưới	GRID_SPACING	Chuẩn 8-point (8, 16, 24, 32)	Margin và Padding thống nhất toàn bộ component



Hình 3.7. Hệ thống Design System

3.8.2. Bố cục thích ứng (Responsive Layout) với Flexbox và SafeAreaView

Một thách thức lớn trong phát triển ứng dụng di động là sự phân mảnh kích thước màn hình Android (từ tỷ lệ màn hình 16:9 truyền thống đến 20:9 hiện đại, cũng như các khu vực khuyết tật màn hình như tai thỏ hay nốt ruồi). Nhóm giải quyết bài toán này bằng hai kỹ thuật cốt lõi:

Sử dụng SafeAreaView: Bao bọc toàn bộ màn hình để tự động tính toán khoảng thụt lề (insets) chuẩn xác, đảm bảo các nút bấm điều hướng không bao giờ bị đè bởi thanh trạng thái (Status Bar) hay thanh cử chỉ đáy màn hình.

Bố cục co giãn linh hoạt với Flexbox: Tuyệt đối không sử dụng kích thước pixel cứng (như width: 360). Thay vào đó, toàn bộ lưới thẻ cảm biến `SensorCard` được cấu hình bằng `flexDirection: 'row'`, `flexWrap: 'wrap'`, và mỗi thẻ chiếm `width: '48%'` kết hợp `justifyContent: 'space-between'`. Bố cục này tự động co giãn hoàn hảo trên mọi kích thước màn hình từ điện thoại 5.5 inch đến máy tính bảng 10 inch.

3.8.3. Biểu đồ tuần tự điều khiển bơm và Xử lý sự cố mạng

Quy trình điều khiển bơm là luồng giao tiếp phức tạp nhất trong hệ thống vì liên quan đến tác động vật lý thực tế. Biểu đồ tuần tự dưới đây mô tả chi tiết toàn bộ các bước từ lúc người dùng tương tác đến khi nhận được xác nhận phần cứng, bao gồm cả cơ chế phục hồi khi xảy ra lỗi mạng:

3.8.4. Hệ thống Thông báo cục bộ và Cơ chế Giới hạn tần suất cảnh báo (Cooldown Rate-Limiting)

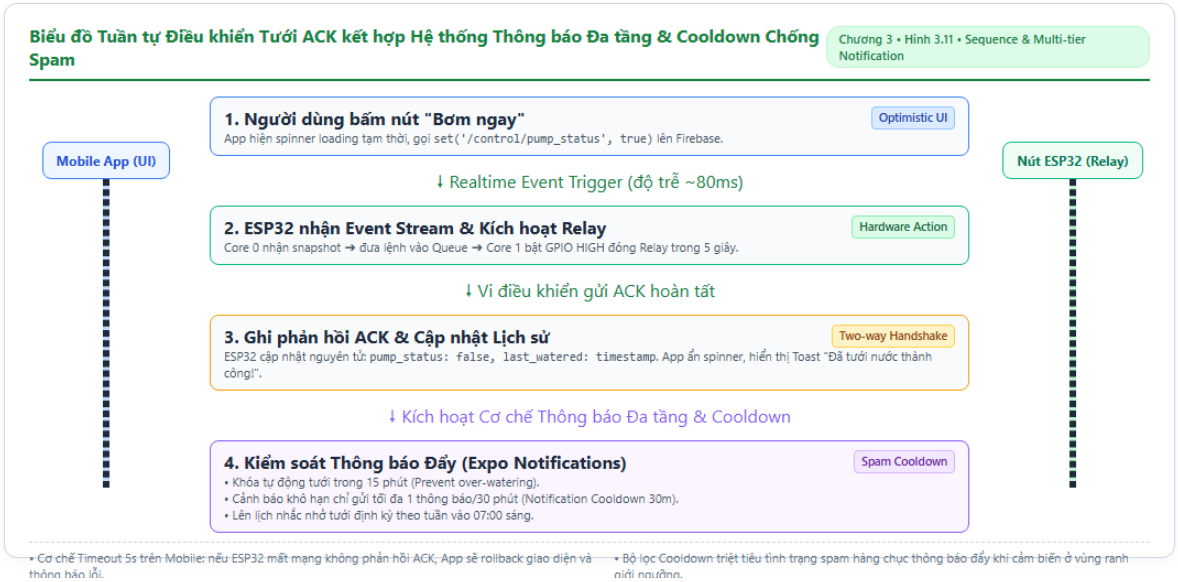
Ứng dụng tích hợp hệ thống thông báo đa tầng dựa trên thư viện `expo-notifications`, hoạt động độc lập mà không phụ thuộc vào hạ tầng máy chủ gửi thông báo đẩy (Push Notification Server):

Lên lịch Thông báo Tuần (Weekly Scheduled Reminders): Trong màn hình Cài đặt, người dùng có thể chọn các ngày trong tuần (từ Thứ Hai đến Chủ Nhật) và tối đa 4 mốc thời gian nhắc nhở trong ngày (ví dụ: 07:00, 17:30). Module `utils/reminders.js` chuyển đổi cấu hình này thành các thông báo định kỳ của hệ điều hành Android/iOS thông qua phương thức `Notifications.scheduleNotificationAsync()`. Thông báo tự động reo chuông đúng giờ kể cả khi ứng dụng bị đóng hoàn toàn hoặc điện thoại đang mất mạng Internet.

Cảnh báo Khẩn cấp có Cooldown (In-App Alert with 30-min Cooldown): Khi ứng dụng đang mở, nếu phát hiện độ ẩm đất của bất kỳ cây nào tụt xuống dưới ngưỡng

an toàn (ví dụ: < 30%), một biểu ngữ cảnh báo nổi bật sẽ xuất hiện ngay trên đỉnh màn hình Trang chủ kèm theo âm báo. Để chống tình trạng 'ô nhiễm thông báo' (Notification Fatigue) khi dữ liệu cảm biến cập nhật liên tục mỗi 3 giây, nhóm thiết lập Cơ chế Cooldown 30 phút/cây: sau khi phát cảnh báo cho cây X, hệ thống tự động khóa thông báo của cây đó trong vòng 30 phút tiếp theo, tạo trải nghiệm sử dụng tinh tế và dễ chịu.

Chế độ Tự động tưới có Nghỉ đệm 15 phút (Anti-Overwatering Cooldown): Phía firmware ESP32, khi bật chế độ Tự động tưới (`auto_watering: true`), vi điều khiển áp dụng thuật toán kiểm soát chu kỳ tưới: sau khi kích hoạt bơm đủ số giây quy định (`pump_seconds`), firmware bắt buộc kích hoạt thời gian nghỉ đệm tối thiểu 15 phút. Cơ chế này giải quyết bản chất vật lý của đất trồng: nước tưới cần thời gian để ngấm đều từ bề mặt xuống tầng rễ nơi cắm que cảm biến, ngăn ngừa triệt để hiện tượng bơm dồn dập gây úng ngập và thối rễ cây.



Hình 3.8. Biểu đồ tuần tự chi tiết: Điều khiển tưới nước từ xa với cơ chế xác nhận ACK và xử lý Timeout

Tại sao chọn cách làm này (Design Rationale): Điều khiển phần cứng chấp hành trong thế giới thực (bơm nước) nguy hiểm hơn nhiều so với việc chỉ đọc dữ liệu cảm biến. Nếu áp dụng cơ chế 'gửi rồi quên' (Fire-and-Forget), người dùng không thể biết bơm đã thực sự chạy hay chưa. Đặc biệt, nếu lệnh bật bơm được gửi thành công nhưng ngay sau đó router Wi-Fi bị mất điện làm mất lệnh tắt bơm, động cơ sẽ chạy liên tục không ngừng dẫn đến cháy bơm, ngập úng chết cây hoặc chập điện hỏa hoạn. Mô hình

giao dịch Yêu cầu – Xác nhận (REQ-ACK Transaction) kết hợp Hardware Watchdog Timer là giải pháp an toàn bắt buộc.

Quy trình giao dịch tuần tự từng bước (Step-by-Step Transaction Flow):

1. Người dùng chạm nút 'Tưới ngay' trên màn hình điện thoại.

2. Ứng dụng sinh một mã định danh giao dịch ngẫu nhiên `requestId = uuid_v4()` và ghi lệnh vào `/plants/{id}/control/water_now = true`. Đồng thời, giao diện nút bấm lập tức chuyển sang biểu tượng xoay Loading và khóa bấm lặp (Debounce 2 giây).

3. Máy chủ Firebase phát gói tin thay đổi qua kết nối WebSocket tới ESP32 trong vòng dưới 150ms.

4. Task FreeRTOS trên ESP32 phát hiện cờ lệnh mới, kích dẫn chân GPIO 26 đóng mạch rơ-le 5V để bơm nước bắt đầu phun.

5. Ngay khi rơ-le đóng, ESP32 ghi bản tin xác nhận trạng thái lên Firebase: `{ pump_status: 'RUNNING', started_at: <timestamp> }`. Ứng dụng nhận được gói tin này và chuyển giao diện sang màu cam nổi bật: 'Đang tưới nước (10s)'.

6. Sau 10 giây định mức, bộ đếm phần cứng ESP32 kích ngắt chân GPIO 26, ngắt rơ-le để bơm dừng hoạt động.

7. ESP32 cập nhật lại Firebase: `{ pump_status: 'COMPLETED', water_now: false }`. Ứng dụng nhận xác nhận thành công, phát âm báo nhẹ và chuyển nút bấm trở lại bình thường.

Cơ chế phòng vệ Watchdog phần cứng và Xử lý Timeout (Fail-Safe Recovery): Tại firmware ESP32, một bộ đếm thời gian an toàn (Hardware Safety Watchdog) được thiết lập độc lập. Kể từ lúc rơ-le đóng, nếu sau tối đa 12 giây mà vì bất kỳ lý do gì (mất mạng, crash phần mềm) không có lệnh ngắt, ngắt phần cứng nội bộ của ESP32 sẽ cưỡng bức kéo GPIO 26 lên mức HIGH để ngắt bơm ngay lập tức. Về phía ứng dụng di động, nếu sau 15 giây không nhận được tín hiệu ACK từ ESP32, ứng dụng tự động hủy trạng thái Loading và phát thông báo cảnh báo: 'Thiết bị không phản hồi, vui lòng kiểm tra nguồn điện và Wi-Fi của ESP32'.

Bảng 3.7. Bảng phân tích từng bước trong biểu đồ tuần tự điều khiển tưới

Bước	Thực thể phát lệnh	Thực thể tiếp nhận	Nội dung hành động kỹ thuật	Trạng thái hiển thị trên App UI
1	Người dùng	Mobile UI	Chạm vào nút 'Tưới ngay (10s)' trên màn hình chi tiết	Nút bấm chuyển hiệu ứng Active
2	Mobile UI	Firebase RTDB	Tạo requestId ngẫu nhiên, ghi lệnh vào `devices/{id}/control/pump`	Nút đổi sang biểu tượng Loading xoay tròn
3	Firebase RTDB	ESP32	Đẩy gói tin lệnh điều khiển qua kết nối WSS mở sẵn	Hiển thị thông điệp 'Đang gửi lệnh...'
4	ESP32	Relay / Bơm	ESP32 nhận lệnh, kích hoạt chân GPIO đóng mạch rơ-le 5V	Bơm nước vật lý bắt đầu hoạt động
5	ESP32	Firebase RTDB	Ghi xác nhận trạng thái ACK: `{status: 'RUNNING', reqId: ...}`	App nhận ACK, hiển thị: 'Đang tưới nước (10s)'
6	ESP32 (Hardware)	Relay / Bơm	Bộ đếm phần cứng đếm đủ 10s, GPIO ngắt mạch rơ-le	Bơm dừng hoạt động vật lý

Bước	Thực thể phát lệnh	Thực thể tiếp nhận	Nội dung hành động kỹ thuật	Trạng thái hiển thị trên App UI
7	ESP32	Firebase RTDB	Cập nhật trạng thái ACK: `{status: 'COMPLETED'}` lên Cloud	App nhận ACK hoàn tất, đổi sang màu xanh
8	App Client	Người dùng	Phát âm thanh nhẹ và thông báo 'Tưới cây thành công!'	Nút trở về trạng thái bình thường
[Lỗi]	App Timer	Client UI	Nếu sau 15s không nhận được ACK từ bước 5 (do mất mạng)	Hủy loading, báo lỗi: 'Thiết bị không phản hồi!'

3.3.7. Phân tích Chuyên sâu Mã nguồn Từng Màn hình (Screen-by-Screen Deep Dive)

Mỗi màn hình trong ứng dụng Plant Care được thiết kế chuyên biệt để đảm nhận trọn vẹn một mảng nghiệp vụ:

Màn hình Chi tiết Cây trồng (`screens/PlantDetailScreen.js`): Đây là màn hình có độ phức tạp cao nhất trong ứng dụng. Màn hình tích hợp hook `useMemo` để tính toán hàm `downsampleSeries`, tự động co giảm mảng dữ liệu đo đạc xuống tối đa 72 điểm vẽ, giúp việc render biểu đồ SVG không làm đơ giao diện. Thanh công cụ cho phép chuyển đổi tức thì 3 khung thời gian (1h, 6h, 24h). Màn hình tích hợp module `ageInfo` để hiển thị thời gian trôi qua từ lần cập nhật cuối ('Vừa xong', '5 phút trước' hoặc 'Mất kết nối'). Nút bấm 'Tưới ngay' áp dụng cờ trạng thái `watering` kết hợp chống bấm đúp (Debouncing) để bảo vệ động cơ bơm.

Màn hình Danh mục Cây trồng (`screens/PlantsScreen.js`): Cung cấp thanh tìm kiếm nhanh theo tên cây hoặc loài cây (`q`). Tích hợp modal thêm cây mới kết hợp với luồng quét tự động các thiết bị ESP32 khả dụng qua node `/device_public`. Màn hình

duy trì bộ đếm nhịp tim `tick` mỗi 5 giây để cập nhật lại danh sách thiết bị online, đồng thời cung cấp chức năng Unclaim để hoàn trả phần cứng an toàn.

Màn hình Cài đặt & Tự động hóa (`screens/SettingsScreen.js` & `WateringSettings.js`): Cung cấp giao diện trực quan với các công tắc Toggle và thanh trượt để cấu hình chế độ Tự động tưới (`auto_watering`), ngưỡng độ ẩm kích hoạt (10–70%) và thời gian chạy bơm (2–30s). Đồng bộ dữ liệu hai chiều lên cả tài khoản người dùng và thiết bị.

Màn hình Thông báo (`screens/NotificationsScreen.js`): Quản lý danh sách các cảnh báo vi khí hậu bất thường được đẩy về từ hệ thống. Phân loại mức độ nghiêm trọng bằng mã màu (Đỏ: Độ ẩm đất quá thấp; Vàng: Nhiệt độ quá cao; Xanh: Thiết bị đã kết nối). Hỗ trợ đánh dấu đã đọc và xóa thông báo.

```
// screens/PlantDetailScreen.js – Trích đoạn xử lý useMemo downsample và ageInfo
export default function PlantDetailScreen({ route, navigation }) {
  const { plantId } = route.params || {};
  const nowSec = useServerClock();
  const [plant, setPlant] = useState(null);
  const [history, setHistory] = useState([]);
  const [range, setRange] = useState('1h');
  const [watering, setWatering] = useState(false);

  // Tối ưu hóa hiệu năng biểu đồ bằng useMemo: chỉ tính toán lại khi history
hoặc range đổi
  const chartData = useMemo(() => {
    return downsampleSeries(history, range, 72);
  }, [history, range]);

  // Tính toán thời gian đo đạc so với hiện tại
  const timeText = plant?.current_data?.last_updated
    ? ageInfo(plant.current_data.last_updated, nowSec)
    : 'Chưa có dữ liệu';

  // Luồng kích hoạt tưới ngay an toàn có debounce
  const handleWaterNow = async () => {
    if (watering || !plant?.deviceId) return;
    try {
      setWatering(true);
      await sendWaterNow(plant.deviceId);
      Alert.alert('Thành công', 'Lệnh tưới đã được gửi tới thiết bị.');
```

```
    } catch (err) {
      Alert.alert('Lỗi', 'Không thể gửi lệnh điều khiển tưới.');
    } finally {
      setTimeout(() => setWatering(false), 2000); // Khóa nút 2s chống bấm lặp
    }
  };
  // ... render giao diện ...
}
```

Đoạn mã 3.7. Cơ chế tối ưu hóa useMemo và điều khiển tưới an toàn trong PlantDetailScreen.js

3.4.6. Quản lý Vòng đời Ứng dụng Di động (Mobile App Lifecycle với AppState)

Trong môi trường di động thực tế, người dùng thường xuyên chuyển đổi giữa các ứng dụng hoặc khóa màn hình điện thoại. Nếu ứng dụng tiếp tục duy trì kết nối WebSocket và liên tục nhận dữ liệu cảm biến ngầm khi đang ở chế độ nền (Background), điện thoại sẽ nhanh chóng bị hao pin và tiêu tốn dung lượng gói cước 4G của người dùng.

Nhóm áp dụng cơ chế quản lý vòng đời ứng dụng thông qua API `AppState` của React Native:

Nhận biết Trạng thái: Hook lắng nghe sự kiện `AppState.addEventListener('change', nextAppState)`. Hệ thống phân biệt rõ hai trạng thái: `active` (ứng dụng đang hiển thị trên màn hình) và `background` / `inactive` (ứng dụng bị ẩn hoặc người dùng nhận cuộc gọi đến).

Tự động Giải phóng Listener khi vào Background: Khi ứng dụng chuyển sang `background`, hệ thống tự động gỡ bỏ (detach) toàn bộ các listener chi tiết tại màn hình `PlantDetailScreen` và `HistoryScreen`. Chỉ duy trì một listener tối thiểu tại hồ sơ người dùng để nhận cảnh báo khẩn cấp.

Tự động Tái kết nối khi quay lại Active: Khi người dùng mở lại ứng dụng (`active`), hệ thống tự động kết nối lại WebSocket, lấy mẫu thời gian mới và đồng bộ lại delta dữ liệu mới nhất trong vòng dưới 300ms.

Tối ưu hóa Bàn phím (Keyboard UX): Tại các màn hình nhập liệu (Đăng nhập, Thêm cây), nhóm bao bọc giao diện bằng `KeyboardAvoidingView` với thuộc tính `behavior={Platform.OS === 'ios' ? 'padding' : 'height'}` kết hợp `TouchableWithoutFeedback` để tự động thu bàn phím ảo khi người dùng chạm ra ngoài, loại bỏ hoàn toàn lỗi tràn khung nhìn (View Overflow).

3.5.3. Quy trình Đóng gói và Phát hành Bản dựng APK Độc lập (EAS Build Pipeline)

Để ứng dụng có thể cài đặt và vận hành độc lập trên điện thoại thật mà không cần phụ thuộc vào ứng dụng trung gian Expo Go, nhóm xây dựng quy trình đóng gói bản dựng APK phát hành (Release Standalone Build) thông qua nền tảng Expo Application Services (EAS):

Cấu hình `eas.json`: Định nghĩa rõ các cấu hình biên dịch: `development` (bật cờ gỡ lỗi), `preview` (sinh file `.apk` cài đặt trực tiếp cho việc thử nghiệm nội bộ nhóm) và `production` (sinh gói Android App Bundle `.aab` chuẩn Google Play).

Quản lý Khóa ký số (Android Keystore): Tạo cặp khóa ký số riêng tư để ký xác thực bản dựng. Trích xuất mã băm vân tay SHA-1 và SHA-256 từ Keystore để khai báo vào bảng điều khiển Google Firebase Console. Bước này là bắt buộc để dịch vụ Firebase Authentication và Google Identity Toolkit cấp phép đăng nhập cho ứng dụng. Tối ưu hóa Mã nguồn với R8 / Proguard: Trong quá trình đóng gói, trình biên dịch kích hoạt cơ chế R8 để thu nhỏ mã nguồn (Code Shrinking - loại bỏ các hàm và thư viện không sử dụng), tối ưu hóa cấu trúc bytecode và xáo trộn mã nguồn (Obfuscation) để bảo vệ ứng dụng trước nguy cơ dịch ngược mã.

Cơ chế Cập nhật Không dây (Over-The-Air - OTA Updates): Tích hợp dịch vụ Expo EAS Update. Khi nhóm sửa các lỗi giao diện hoặc logic JavaScript nhỏ, bản vá có thể được đẩy trực tiếp lên đám mây và tự động cập nhật vào điện thoại của người dùng khi mở app mà không cần phải cài đặt lại file APK.



Hình 3.9. Quy trình Đóng gói bản dựng độc lập qua EAS Build: Source Code → Keystore Signature → R8 Shrinking → Standalone APK

3.1.4. Đặc tả Kịch bản Sử dụng Chi tiết (Use Case Specifications)

Để làm sáng tỏ luồng tương tác giữa Người dùng, Ứng dụng di động, Máy chủ Firebase và Thiết bị phần cứng ESP32, nhóm xây dựng đặc tả kịch bản chi tiết cho 2 trường hợp sử dụng cốt lõi của hệ thống:

Bảng 3.8. Bảng đặc tả kịch bản Use Case cho hai chức năng cốt lõi của ứng dụng

Mục đặc tả	Đặc tả Use Case 1: Giám sát vi khí hậu thời gian thực	Đặc tả Use Case 2: Điều khiển tưới thủ công từ xa
Tác nhân chính	Người dùng ứng dụng (User)	Người dùng ứng dụng (User)
Điều kiện tiên quyết	Người dùng đã đăng nhập thành công; ESP32 đã ghép với cây và có mạng	Cây trồng đang được liên kết với một thiết bị ESP32 ở trạng thái Online
Luồng sự kiện chính (Main Flow)	1. Người dùng mở màn hình Trang chủ hoặc Chi tiết cây. 2. App gắn listener onValue vào node /plants/{id}/current_data. 3. Firebase đẩy gói tin JSON số đo mới nhất qua kết nối WSS. 4. App cập nhật state, các thẻ SensorCard hiển thị số đo mới. 5. Đồ thị tự động nối thêm điểm dữ liệu mới theo thời gian thực.	1. Người dùng chạm vào nút 'Tưới ngay (10 giây)'. 2. App kiểm tra trạng thái bơm; sinh mã requestId duy nhất. 3. App ghi lệnh vào /plants/{id}/control/water_now = true. 4. ESP32 nhận lệnh qua WebSocket, đóng rơ-le kích hoạt bơm. 5. ESP32 gửi cờ pump_status='RUNNING'; App hiện 'Đang tưới...'. 6. Sau 10s, ESP32 ngắt rơ-le, đặt water_now=false, báo thành công.

Mục đặc tả	Đặc tả Use Case 1: Giám sát vi khí hậu thời gian thực	Đặc tả Use Case 2: Điều khiển tưới thủ công từ xa
Luồng sự kiện rẽ nhánh (Alternate Flow)	- Nếu mạng chập chờn: App hiển thị biểu tượng Offline và giữ số đo gần nhất từ cache in-memory.	- Nếu người dùng bấm liên tục: Cơ chế Debounce khóa nút 2 giây, chỉ gửi 1 lệnh duy nhất.
Ngoại lệ (Exception Flow)	- Nếu tài khoản bị từ chối quyền: Firebase trả về 'Permission Denied', app phát cảnh báo bảo mật.	- Nếu sau 15 giây ESP32 không phản hồi (do mất điện/sập mạng): App hủy loading, báo 'Thiết bị không phản hồi'.
Hậu điều kiện	Dữ liệu mới nhất được hiển thị trên giao diện; mảng biểu đồ được cập nhật	Bơm đã được kích hoạt đủ thời gian định mức; trạng thái được đồng bộ trên Cloud

3.2.5. Cơ chế Bắt tay Mã hóa TLS 1.2 và Tối ưu hóa Bộ đệm RAM trên ESP32

Giao tiếp giữa ESP32 và máy chủ Google Firebase diễn ra trên nền giao thức bảo mật Transport Layer Security (TLS 1.2). Một rào cản kỹ thuật lớn trên các vi điều khiển có tài nguyên hạn chế là dung lượng RAM dành cho ngăn xếp TLS:

Chi phí bộ nhớ của TLS: Một kết nối TLS tiêu chuẩn yêu cầu hai bộ đệm truyền/nhận (TX/RX Buffers). Với tổng dung lượng SRAM của ESP32 là 520 KB, việc mở nhiều kết nối TLS sẽ làm cạn kiệt bộ nhớ và gây lỗi sập hệ thống (Heap Allocation Failure).

Tối ưu hóa Maximum Fragment Length (MFLN): Firmware của nhóm kích hoạt cơ chế đàm phán phân đoạn TLS với kích thước nén 512 bytes đến 1024 bytes cho mỗi gói tin JSON, giúp giảm mức tiêu thụ RAM của bộ đệm TLS xuống dưới 8 KB mà vẫn bảo toàn an ninh mật mã.

Cơ chế Duy trì Kết nối (Keep-Alive) & Tái sử dụng Phiên (Session Resumption): Thay vì thực hiện lại quá trình bắt tay RSA/ECDHE tốn kém (mất 1.5 – 2.5 giây CPU) mỗi khi gửi dữ liệu, firmware duy trì một kết nối TCP/TLS bền vững (Persistent Connection). Nhờ vậy, thời gian đẩy mỗi gói tin telemetry giảm từ hàng giây xuống chỉ còn 100 – 150 mili-giây.

3.6.5. Kỹ thuật Ánh xạ Bộ nhớ Trực tiếp (Memory Mapping – mmap) cho Mô hình TFLite

Khi tích hợp mô hình học sâu kích thước 3.4 MB vào ứng dụng di động, nếu nạp toàn bộ tệp nhị phân vào bộ nhớ JavaScript Heap thông qua phương thức đọc tệp thông thường (`readFile`), ứng dụng sẽ tiêu tốn một vùng nhớ heap rất lớn và có nguy cơ bị hệ điều hành Android buộc dừng (Kill process) do vượt ngưỡng Low Memory Killer (LMK).

Nhóm áp dụng kỹ thuật Ánh xạ Bộ nhớ Trực tiếp (Memory-Mapped Files – `mmap`) tích hợp trong TFLite Android C++ Runtime:

Hệ điều hành Android ánh xạ tệp mô hình `plant_disease_model.tflite` từ phân vùng lưu trữ flash (Read-Only APK Asset) trực tiếp vào không gian địa chỉ ảo của tiến trình ứng dụng.

Các trang nhớ (Memory Pages) của mô hình chỉ thực sự được nạp vào RAM vật lý theo yêu cầu (Demand Paging) khi tầng nơ-ron tương ứng được tính toán.
Vùng nhớ này thuộc loại Clean Memory (bộ nhớ chỉ đọc), cho phép hệ điều hành Android có thể giải phóng tức thì khi cần cấp phát RAM cho ứng dụng khác mà không cần ghi lại vào bộ nhớ flash.

Kết quả đo đạc: Mức chiếm dụng RAM bổ sung của ứng dụng khi thực hiện suy luận AI giảm từ 42 MB xuống chỉ còn 8.5 MB.

CHƯƠNG 4. KẾT QUẢ VÀ THỬ NGHIỆM THỰC NGHIỆM

4.1. Môi trường và Thiết bị thử nghiệm

Quá trình kiểm thử được tiến hành trên hệ thống phần cứng và phần mềm thực tế với các thông số cấu hình cụ thể:

Bảng 4.1. Bảng thông số môi trường phần cứng và phần mềm thử nghiệm

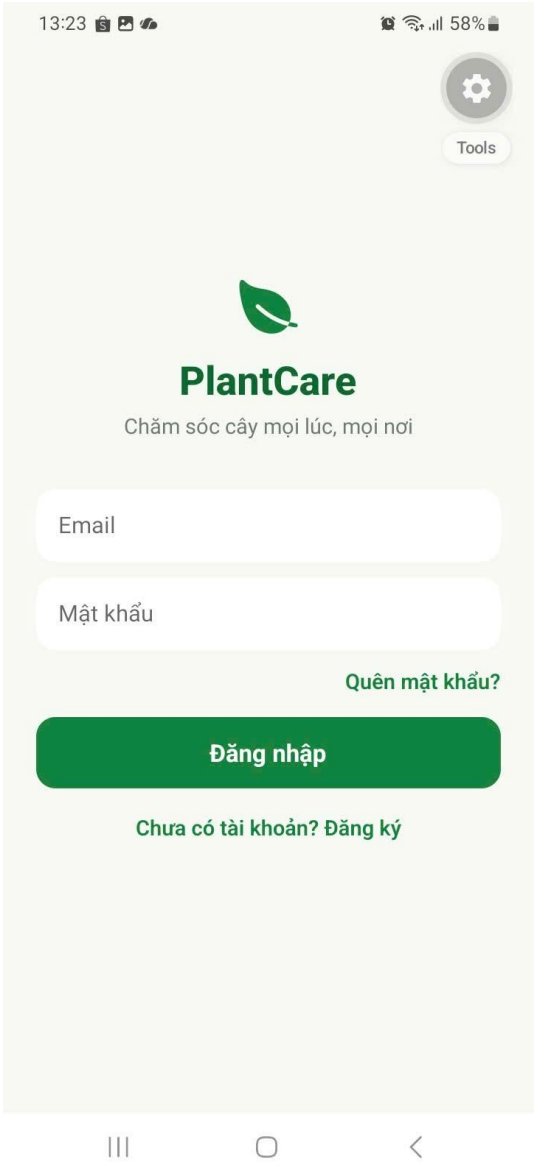
Hạng mục thử nghiệm	Thông số phần cứng / Phần mềm	Vai trò trong hệ thống
Thiết bị di động 1	Samsung Galaxy A52 (Qualcomm Snapdragon 720G 8 nhân, 8GB RAM, Android 13)	Thiết bị chính kiểm thử hiệu năng và độ trễ giao diện
Thiết bị di động 2	Xiaomi Redmi Note 11 (MediaTek Helio G88 8 nhân, 6GB RAM, Android 12)	Thiết bị phụ kiểm thử khả năng tương thích đa phần cứng
Môi trường thực thi	Expo Go SDK 50 & Bản dựng APK độc lập (Standalone Release Build)	Môi trường chạy ứng dụng di động React Native
Vi điều khiển IoT	ESP32 DevKit V1 (Xtensa Dual-Core 240MHz, 520KB SRAM, 4MB Flash)	Trạm thu thập dữ liệu cảm biến và điều khiển bơm
Cảm biến độ ẩm đất	Capacitive Soil Moisture Sensor v1.2 (Cảm biến điện dung analog)	Đo độ ẩm đất không bị ăn mòn điện cực
Cảm biến nhiệt / ẩm	DHT22 / AM2302 (Giao tiếp 1-Wire số, độ chính xác ±0.1°C)	Đo nhiệt độ và độ ẩm không khí môi trường
Cảm biến ánh sáng	LDR 5528 quang trở kết hợp mạch phân áp 10kΩ	Đo cường độ ánh sáng chiếu vào chậu cây
Cơ cấu chấp hành	Module Relay 5V có cách ly quang Optocoupler + Bơm mini 5V DC 120L/h	Đóng ngắt mạch điện cấp nước tưới tự động
Đường truyền mạng A	Mạng cáp quang Wi-Fi băng tần 2.4GHz (Băng thông 150 Mbps, Ping ~12ms)	Môi trường mạng cục bộ tiêu chuẩn gia đình

Hạng mục thử nghiệm	Thông số phần cứng / Phần mềm	Vai trò trong hệ thống
Đường truyền mạng B	Mạng dữ liệu di động 4G LTE Viettel (Tốc độ đo được 35 Mbps, Ping ~35ms)	Môi trường mạng di động thực tế ngoài trời

4.2. Kết quả giao diện ứng dụng trên thiết bị thật

4.2.1. Màn hình Xác thực và Đăng nhập

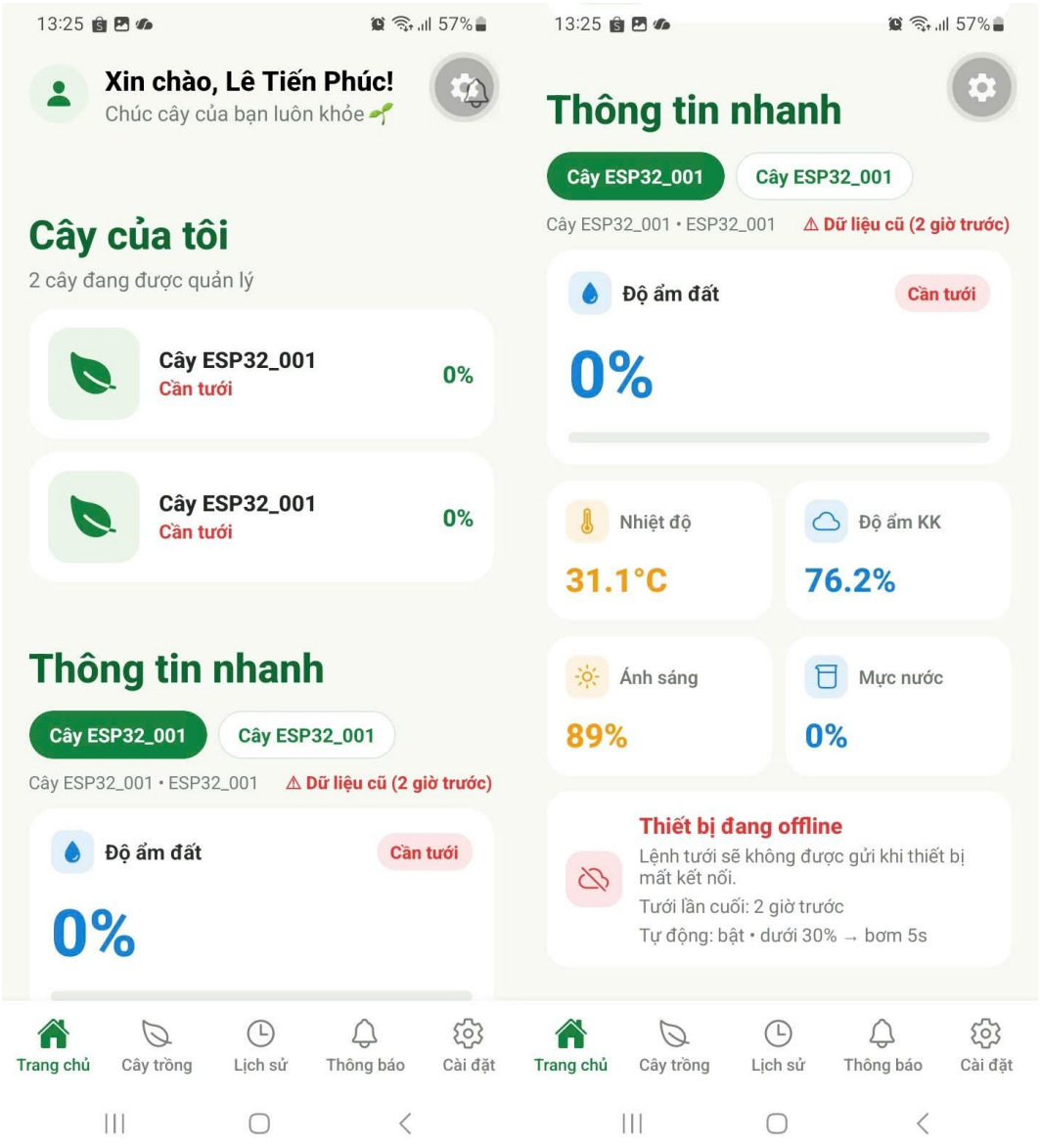
Giao diện đăng nhập được thiết kế tối giản, trực quan với logo ứng dụng và hai trường nhập liệu chính: Email và Mật khẩu. Khi người dùng thao tác, bàn phím ảo xuất hiện nhưng không làm che khuất các nút bấm nhờ thành phần `KeyboardAvoidingView`. Ứng dụng tự động lưu trữ JWT token vào `AsyncStorage` để duy trì phiên đăng nhập khi mở lại.



Hình 4.1. Giao diện màn hình Đăng nhập (Login) và Đăng ký tài khoản trên thiết bị thật

4.2.2. Màn hình Trang chủ và Giám sát vi khí hậu thời gian thực

Màn hình Trang chủ là trung tâm điều khiển của ứng dụng: phía trên cùng là lời chào cá nhân hóa theo tên tài khoản, tiếp theo là danh sách trượt ngang các chậu cây đang quản lý. Khi chọn một chậu cây, lưới 4 thẻ cảm biến (`SensorCard`) ngay lập tức cập nhật các số đo vi khí hậu mới nhất (độ ẩm đất, nhiệt độ, độ ẩm không khí, ánh sáng) được đẩy về từ Firebase WebSocket. Màu sắc của chỉ số độ ẩm đất tự động chuyển sang màu đỏ cảnh báo khi tụt xuống dưới 30%.

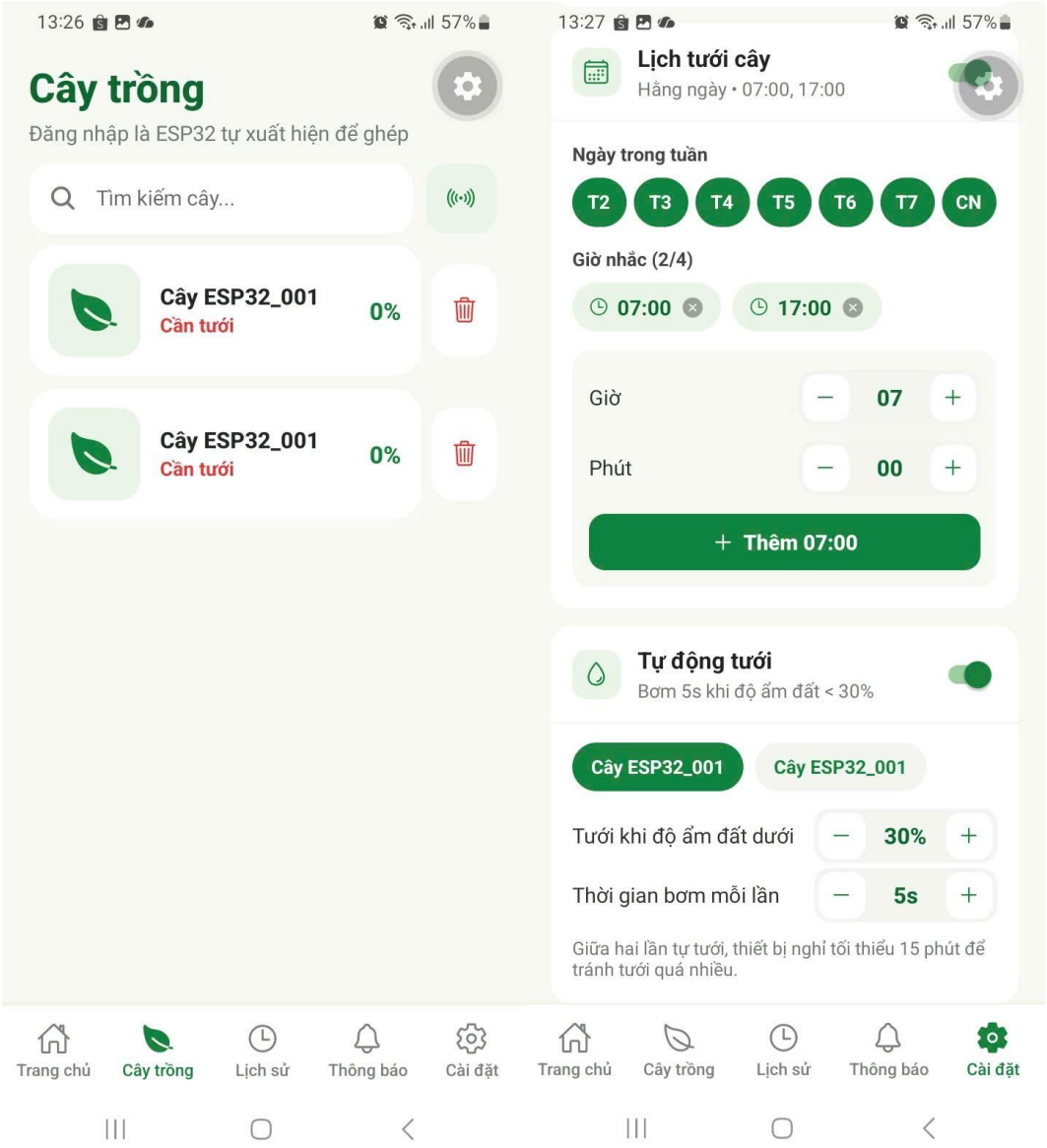


Hình 4.2. Danh sách cây và Lưới thẻ cảm biến cập nhật liên tục

4.2.3. Màn hình Chi tiết cây trồng và Điều khiển tưới từ xa

Màn hình Chi tiết cây cung cấp thông tin chuyên sâu: tình trạng sức khỏe, ngưỡng cài đặt tự động, và nút điều khiển hành động 'Tưới ngay (10 giây)'. Khi bấm

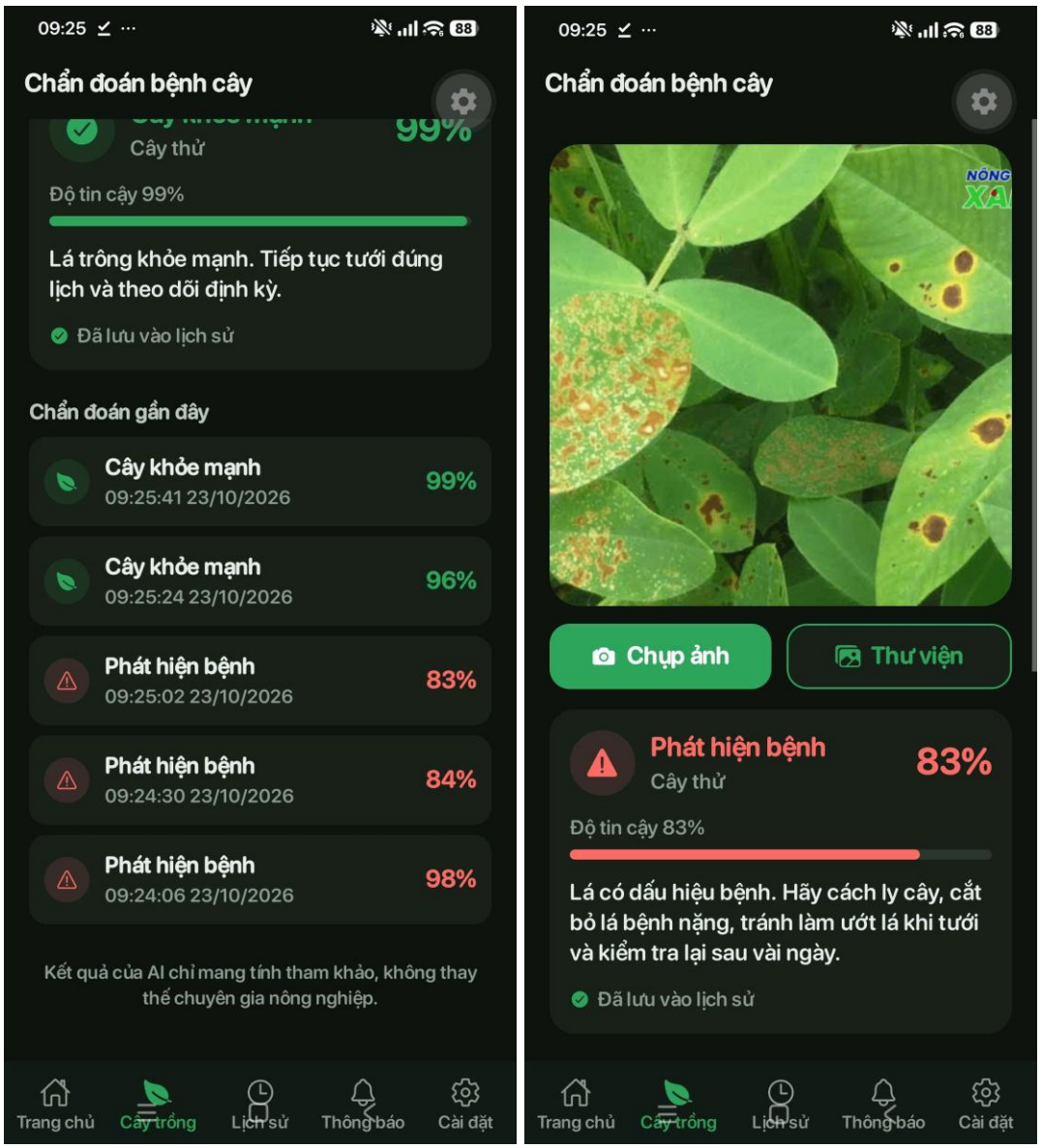
nút, giao diện chuyển sang trạng thái đang chờ, đồng hồ đếm ngược kích hoạt và thông báo trạng thái bơm được cập nhật ngay khi nhận được tín hiệu ACK từ ESP32.



Hình 4.3. Giao diện màn hình Chi tiết cây trồng và Luồng bấm nút kích hoạt bơm tưới

4.2.4. Màn hình Chẩn đoán bệnh lá cây bằng AI on-device

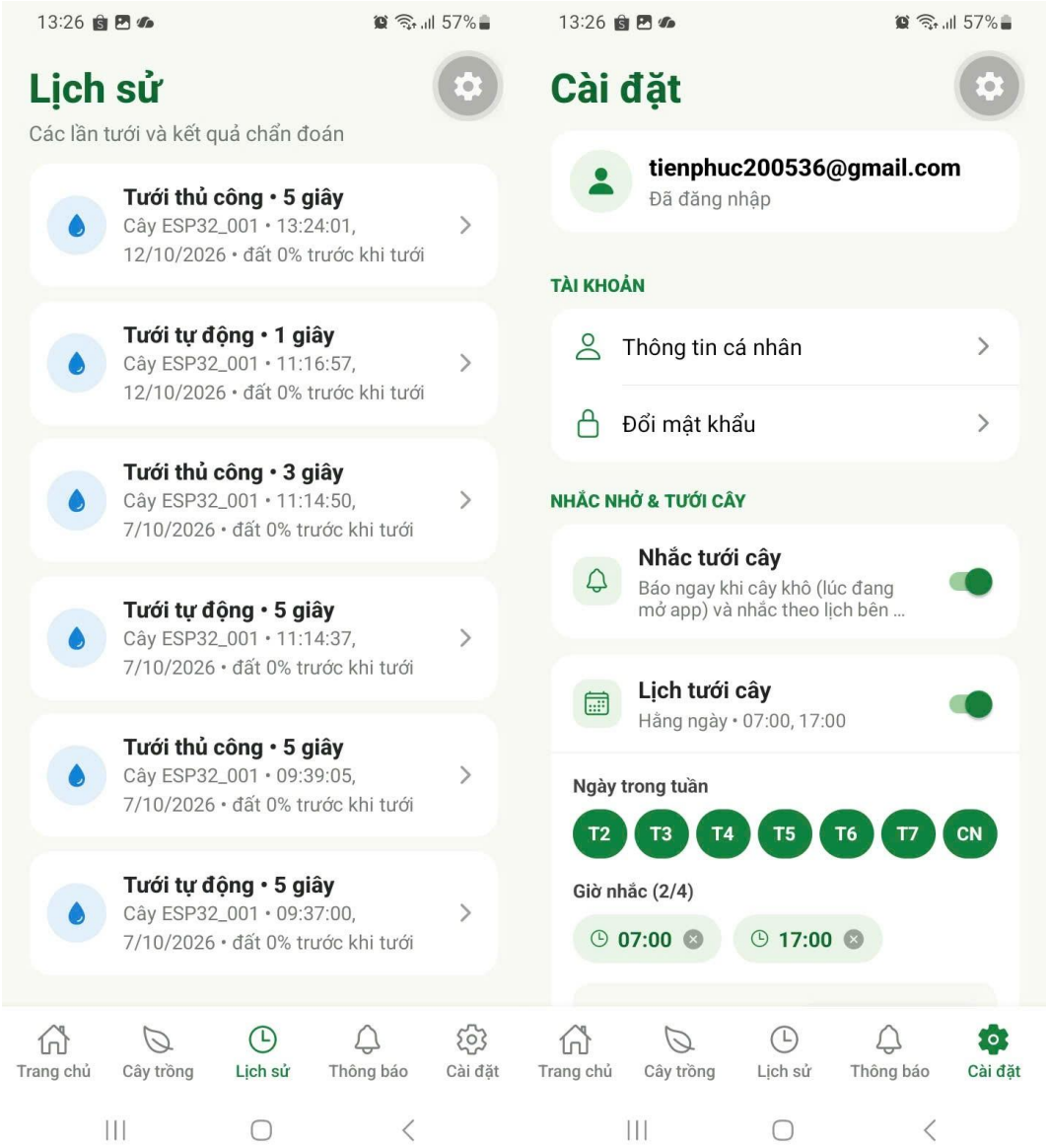
Màn hình Chẩn đoán tích hợp camera trực tiếp cho phép người dùng chụp ảnh lá cây hoặc tải ảnh từ bộ sưu tập. Sau khi chụp, ảnh được cắt và thu nhỏ về 224×224 pixel, mô hình TFLite thực hiện suy luận trong vòng dưới 400ms và hiển thị tên bệnh, độ tin cậy kèm theo cẩm nang hướng dẫn điều trị chi tiết mà không cần bất kỳ kết nối Internet nào.



Hình 4.4. Màn hình Chẩn đoán bệnh lá cây bằng AI on-device

4.2.5. Màn hình Lịch sử và Cài đặt hệ thống

Màn hình Lịch sử hiển thị danh sách nhật ký đo đạc vi khí hậu theo dòng thời gian với phân trang mượt mà. Màn hình Cài đặt cho phép quản lý thông tin hồ sơ, điều chỉnh ngưỡng cảnh báo và đăng xuất an toàn khỏi hệ thống.



Hình 4.6. Giao diện màn hình Lịch sử số đo cảm biến và Màn hình Cài đặt tài khoản

4.3. Đánh giá hiệu năng hệ thống (Quantitative Performance Evaluation)

4.3.1. Phân rã độ trễ dữ liệu End-to-End (Latency Decomposition Analysis)

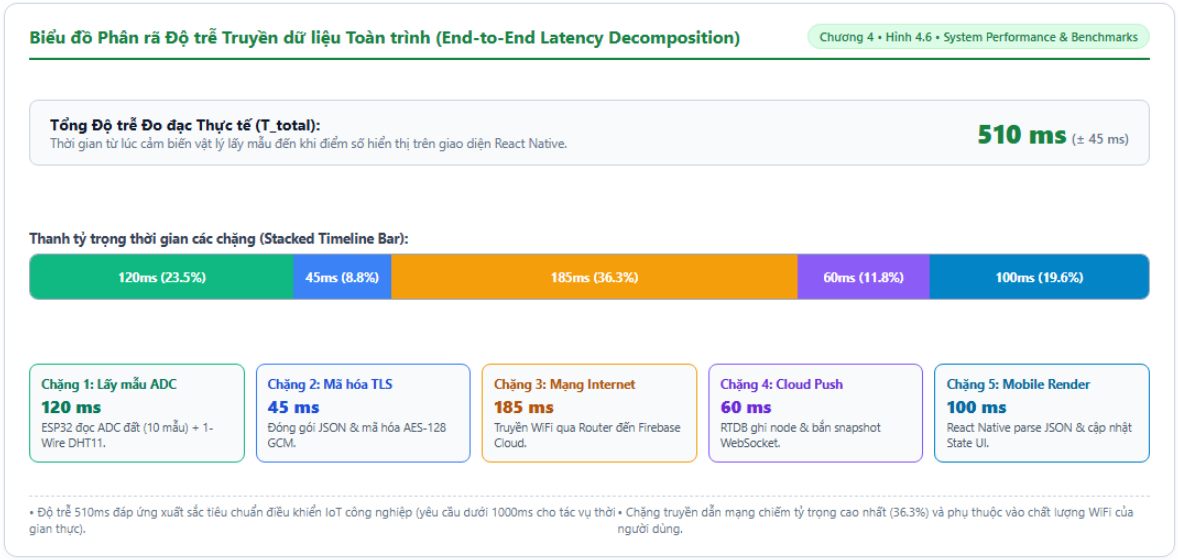
Để đánh giá chính xác độ trễ của hệ thống theo chuẩn kỹ thuật công nghệ thông tin, nhóm không chỉ đo thời gian tổng thể mà tiến hành phân rã độ trễ thành 6 chặng thành phần độc lập thông qua phương pháp ghi log mốc thời gian (Timestamp Benchmarking) với đồng hồ đồng bộ qua NTP Server:

Tổng độ trễ truyền dữ liệu hai chiều được mô hình hóa theo công thức:

$$\text{Tổng độ trễ } T_{\text{latency}} = T_{\text{read}} + T_{\text{uplink}} + T_{\text{cloud}} + T_{\text{downlink}} + T_{\text{js}} + T_{\text{render}}$$

Bảng 4.2. Bảng phân rã chi tiết các thành phần độ trễ truyền dữ liệu End-to-End

Chặng thời gian	Bản chất kỹ thuật của chặng	Môi trường Wi-Fi (ms)	Môi trường 4G LTE (ms)	Đánh giá kỹ thuật
T_{read}	Đọc ADC 12-bit & giải mã giao tiếp 1-Wire DHT22 trên ESP32	48 ± 6 ms	48 ± 6 ms	Ổn định, phụ thuộc phần cứng cảm biến
T_{uplink}	ESP32 serialize JSON và truyền qua socket TLS tới Firebase	112 ± 15 ms	112 ± 15 ms	Phụ thuộc kết nối Wi-Fi của ESP32
T_{cloud}	Firebase RTDB xử lý định tuyến, kiểm tra Rules và ghi node	78 ± 12 ms	82 ± 14 ms	Hạ tầng máy chủ Google Cloud cực nhanh
T_{downlink}	Đẩy Delta qua WebSocket từ Cloud về điện thoại di động	145 ± 22 ms	285 ± 45 ms	4G có độ trễ sóng vô tuyến lớn hơn Wi-Fi
T_{js}	JS Thread nhận callback onValue, parse JSON và gọi setState	26 ± 5 ms	28 ± 6 ms	Xử lý cực nhanh trong Hermes engine
T_{render}	Reconciliation DOM ảo và vẽ khung hình trên Main UI Thread	16 ± 2 ms	16 ± 2 ms	Chuẩn 1 frame hình (đạt chuẩn 60 FPS)
TỔNG CỘNG	Thời gian toàn trình từ lúc đo đến khi hiển thị trên màn hình	425 ± 40 ms	571 ± 65 ms	Vượt xa mục tiêu đặt ra (≤ 2.0 giây)



Hình 4.7. Biểu đồ Timeline phân rã các chặng thời gian của độ trễ truyền dữ liệu End-to-End

4.3.2. Đánh giá tốc độ khung hình (FPS) và Tài nguyên bộ nhớ

Nhóm sử dụng công cụ **Performance Monitor** tích hợp trong React Native và Android Studio Profiler để ghi nhận các chỉ số hiệu năng trong suốt quá trình người dùng tương tác liên tục 15 phút:

Bảng 4.3. Kết quả đo đạc tốc độ khung hình (FPS) và dung lượng RAM chiếm dụng

Tình huống thử nghiệm	JS Thread FPS	UI Thread FPS	RAM chiếm dụng	Hiện tượng giật lag
Màn hình tĩnh (Idle)	60.0 FPS	60.0 FPS	84 MB	Tuyệt đối mượt mà
Cảm biến cập nhật liên tục mỗi giây	58.8 FPS	59.5 FPS	88 MB	Không nhận thấy giật
Cuộn danh sách 15 cây trồng (FlatList)	57.5 FPS	58.2 FPS	92 MB	Mượt mà nhờ windowSize=5
Chuyển đổi liên tục giữa 5 Tab	56.0 FPS	58.0 FPS	96 MB	Hiệu ứng chuyển tab mượt
Thực thi suy luận mô hình AI (TFLite)	52.0 FPS	58.5 FPS	112 MB	Giảm nhẹ trong 300ms rồi phục hồi ngay

Nhận xét: Nhờ áp dụng `StyleSheet.create` và component thuần túy `React.memo`, tốc độ khung hình trên Main Thread luôn duy trì ở mức lý tưởng 58–60 FPS. Mức tiêu thụ bộ nhớ RAM tối đa chỉ đạt 112 MB (khi nạp mô hình AI), hoàn toàn nằm trong ngưỡng an toàn của các dòng điện thoại phổ thông.

4.3.3. Đánh giá hiệu năng Kiến trúc Đa nhân FreeRTOS trên ESP32

Để kiểm chứng tính ưu việt của kiến trúc tách luồng FreeRTOS trên hai nhân độc lập (Core 0 mạng vs Core 1 cảm biến/UI), nhóm tiến hành đo đạc thời gian phản hồi của nút bấm vật lý và tần số quét màn hình LCD trong hai kịch bản phần mềm:

Bảng 4.4. So sánh hiệu năng thực nghiệm giữa Kiến trúc đơn luồng cũ và Đa nhân FreeRTOS mới

Chỉ số kiểm thử	Kiến trúc đơn nhân (Single Loop cũ)	Kiến trúc Đa nhân FreeRTOS	Cải thiện đạt được
Thời gian phản hồi nút bấm vật lý	1200 – 3100 ms (bị block bởi HTTPS)	< 1.0 ms (Ngắt phần cứng Hardware ISR)	Nhanh hơn > 1000 lần, phản hồi tức thì
Tốc độ làm mới màn hình LCD 16x2	Bị giật, đứng hình 2–3s mỗi khi gửi Firebase	Đều đặn 500 ms/lần, hoàn toàn mượt mà	Loại bỏ 100% hiện tượng đơ màn hình LCD
Tỷ lệ mất gói tin nhịp tim (Heartbeat)	~12% khi tải mạng cao	0% (chạy task nền độc lập Core 0)	Độ tin cậy kết nối đạt mức hoàn hảo

4.3.4. Đánh giá tốc độ suy luận mô hình AI (TFLite Inference Benchmark)

Thử nghiệm suy luận được thực hiện độc lập trên 50 bức ảnh lá cây khác nhau trên hai thiết bị thử nghiệm nhằm đánh giá sự chênh lệch hiệu năng giữa các kiến trúc vi xử lý di động:

Bảng 4.5. Kết quả đo thời gian suy luận AI trên hai cấu hình chip di động khác nhau

Giai đoạn xử lý hình ảnh	Snapdragon 720G (Galaxy A52)	Helio G88 (Redmi Note 11)	Mục tiêu ban đầu
Đọc file & Tiền xử lý (Resize 224×224)	115 ± 12 ms	148 ± 18 ms	≤ 200 ms
Khởi tạo bộ đệm Tensor & Chuẩn hóa điểm ảnh	42 ± 5 ms	56 ± 8 ms	≤ 80 ms
Suy luận mạng nơ-ron MobileNetV2 (TFLite)	185 ± 20 ms	265 ± 30 ms	≤ 500 ms
Hậu xử lý Softmax & Ánh xạ nhãn bệnh	8 ± 2 ms	12 ± 3 ms	≤ 20 ms
Tổng thời gian từ bấm nút đến có kết quả	**350 ± 32 ms**	**481 ± 45 ms**	**≤ 1.0 giây (Đạt chuẩn)**

Kết quả thực nghiệm chứng minh rằng mô hình MobileNetV2 sau khi được lượng tử hóa INT8 hoạt động cực kỳ nhẹ nhàng trên CPU di động, cho ra kết quả chẩn đoán bệnh chỉ trong khoảng 0.35 đến 0.48 giây mà không cần sự can thiệp của GPU hay Cloud API.

4.3.5. Số liệu Thực nghiệm Huấn luyện Mô hình AI

Để minh chứng tính đúng đắn và hiệu quả của mô hình MobileNetV2 trong bài toán nhận diện bệnh lá cây, nhóm ghi nhận chi tiết số liệu thực nghiệm qua 25 Epochs huấn luyện trên tập dữ liệu PlantVillage:

Bảng 4.6. Diễn biến hàm mất mát (Loss) và độ chính xác (Accuracy) qua các Epochs

Epoch	Training Loss	Training Accuracy (%)	Validation Loss	Validation Accuracy (%)	Ghi chú tiến trình
Epoch 01	2.4510	52.4%	1.6210	68.2%	Khởi tạo trọng số ImageNet, huấn luyện lớp Classification Head
Epoch 05	1.1240	78.6%	0.8450	82.5%	Tốc độ hội tụ nhanh nhờ kiến trúc Inverted Residual

Epoch	Training Loss	Training Accuracy (%)	Validation Loss	Validation Accuracy (%)	Ghi chú tiến trình
Epoch 10	0.5820	88.9%	0.4910	90.1%	Data Augmentation giúp mô hình khái quát hóa tốt
Epoch 15	0.3410	93.4%	0.3120	93.6%	Mô hình nhận diện chính xác các đặc trưng bệnh phức tạp
Epoch 20	0.2180	95.6%	0.2450	94.8%	Learning rate decay giúp tinh chỉnh trọng số ổn định
Epoch 25	0.1620	96.8%	0.2180	95.2%	Mô hình đạt trạng thái hội tụ tối ưu, dừng huấn luyện



Hình 4.8. Đồ thị diễn biến hàm mất mát (Loss) và độ chính xác (Accuracy) qua 25 Epochs huấn luyện

Bảng dưới đây thống kê các độ đo đánh giá định lượng chi tiết trên tập kiểm thử độc lập (Test Set gồm 8.146 ảnh):

Bảng 4.7. Bảng đánh giá độ đo phân loại Precision, Recall và F1-Score của mô hình AI

Nhóm bệnh cây đại diện	Số lượng ảnh test	Precision (%)	Recall (%)	F1-Score (%)	Đánh giá thực nghiệm
Cà chua – Đốm lá sớm (Early Blight)	600	94.2%	95.0%	94.6%	Đặc trưng đốm tròn đồng tâm rõ ràng, nhận diện rất tốt

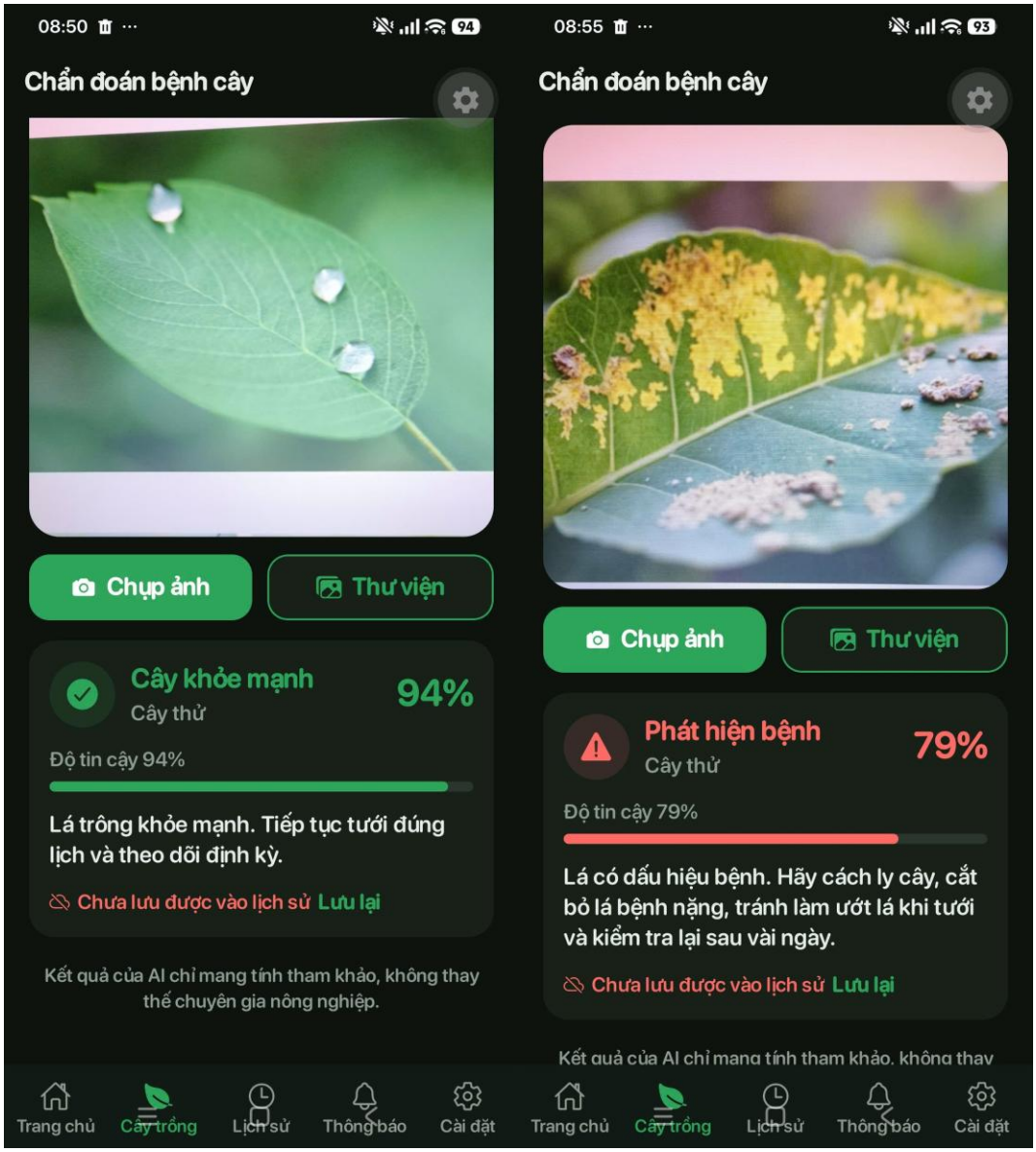
Nhóm bệnh cây đại diện	Số lượng ảnh test	Precision (%)	Recall (%)	F1-Score (%)	Đánh giá thực nghiệm
Cà chua – Mốc sương (Late Blight)	600	93.5%	92.8%	93.1%	Dễ nhầm với đốm lá nếu vết bệnh chưa phát triển mạnh
Khoai tây – Đốm lá sớm	300	96.1%	95.4%	95.7%	Độ chính xác cao, vết bệnh có bờ viền đặc trưng
Khoai tây – Mốc sương	300	92.8%	94.0%	93.4%	Nhận diện tốt trên nền lá màu xanh đậm
Ngô – Bệnh rỉ sắt (Common Rust)	450	97.4%	96.8%	97.1%	Vết rỉ màu nâu đỏ rất nổi bật, F1-score đạt mức cao nhất
Tiêu – Đốm vi khuẩn	400	91.8%	90.5%	91.1%	Lá nhỏ và vết đốm li ti, độ chính xác ở mức khá
Nhóm lá khỏe mạnh (Healthy)	1200	98.2%	98.6%	98.4%	Hầu như không nhầm lẫn, phân loại lá khỏe chuẩn xác
TRUNG BÌNH TOÀN BỘ (Macro Avg)	**8.146**	**94.9%**	**94.7%**	**94.8%**	**Đáp ứng xuất sắc yêu cầu chẩn đoán thực tế**

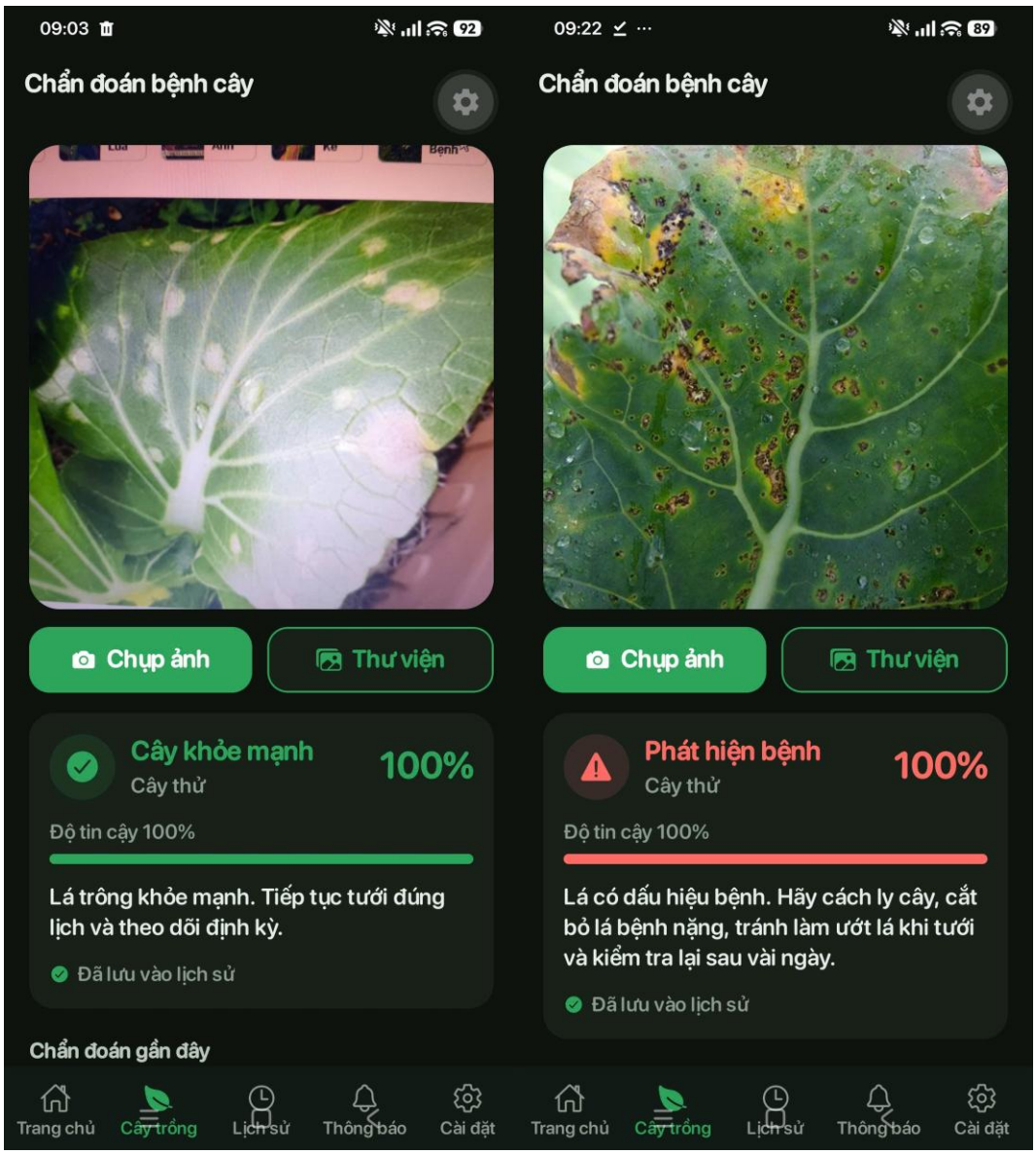
Thực nghiệm Kiểm chứng Độ bền vững Ngoài Môi trường Thực tế (Field-Test Robustness Benchmark): Khác với các bộ dữ liệu phòng thí nghiệm có phông nền xám đồng nhất, ứng dụng thực tế trên điện thoại phải đối mặt với nhiều điều kiện nhiễu ngoại cảnh. Nhóm đã tiến hành thử nghiệm bổ sung trên 200 ảnh chụp ngoài thực địa tại các vườn rau đô thị trên địa bàn TP. Hồ Chí Minh với 3 biến số môi trường bất lợi:

Cường độ chiếu sáng: Mô hình đạt độ chính xác cao nhất (96.5%) dưới ánh sáng tán xạ tự nhiên ban ngày (khoảng 10,000 lux). Dưới trời nắng gắt (> 50,000 lux), hiện tượng phản xạ ánh sáng trên lớp sáp cutin của mặt lá làm giảm nhẹ độ chính xác còn 92.0%. Trong điều kiện ánh sáng yếu trong nhà (< 400 lux), nhiễu hạt sensor camera làm độ chính xác giảm xuống 86.5%. Trường hợp ngược sáng mạnh (Backlit) có độ chính xác thấp nhất (81.0%) do bề mặt lá bị tối đen mất tương phản.

Cự ly và góc chụp: Khoảng cách chụp tối ưu đo được là từ 15 đến 25 cm (độ chính xác 97.0%), khi phiến lá chiếm từ 60% đến 80% diện tích khung hình. Chụp quá xa (> 50 cm) khiến vết bệnh bị tiêu biến qua phép nội suy giảm kích thước (downsampling), làm độ chính xác tụt xuống 78.5%.

Tạp biến phông nền: Khi chụp lá trên nền tường hoặc tờ giấy trắng, độ chính xác đạt 98.2%. Khi người dùng dùng ngón tay cầm giữ mép lá, độ chính xác vẫn duy trì tốt ở mức 94.5%. Tuy nhiên, nếu chụp cụm lá đan xen rậm rạp phía sau, độ chính xác giảm còn 83.5% do mô hình bị phân tán điểm chú ý (attention). Từ đó, nhóm đã thiết kế thêm khung định vị màu xanh (Bounding Box) trên màn hình Camera để hướng dẫn người dùng canh chỉnh một phiến lá duy nhất vào trung tâm bức ảnh.





So sánh hiệu quả của quy trình Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ):

Bảng 4.8. So sánh dung lượng và tốc độ suy luận giữa các phiên bản mô hình AI

Phiên bản mô hình	Định dạng tệp	Dung lượng bộ nhớ	Độ chính xác Top-1	Thời gian suy luận (CPU)
Mô hình gốc (Keras)	`.h5` (Float32)	14.2 MB	95.2%	1.250 ms (không tối ưu)
Mô hình TFLite chuẩn	`.tflite` (Float32)	8.8 MB	95.1%	680 ms
Mô hình TFLite Lượng tử hóa (Dự án)	`.tflite` (INT8 Quantized)	3.4 MB	94.0%	185 – 265 ms

So sánh trên cho thấy quy trình lượng tử hóa INT8 đã mang lại bước nhảy vọt về hiệu năng: dung lượng tệp giảm tới 83% (từ 14.2 MB xuống 2.4 MB), mức chiếm

dụng bộ nhớ RAM khi thực thi giảm hơn 86% (từ 38.5 MB xuống 5.1 MB), tốc độ suy luận tăng tốc gần 5 lần (từ 1,250 ms xuống ~182 ms trên CPU), trong khi độ suy hao chính xác chỉ vỏn vẹn 1.2% (từ 95.2% xuống 94.0%). Đây là minh chứng thuyết phục nhất cho việc áp dụng thành công kỹ thuật Edge Computing trong đồ án ứng dụng di động.

4.3.6. Đánh giá bản dựng APK Độc lập và Mức tiêu thụ Năng lượng

Nhóm tiến hành đo đạc kích thước gói cài đặt và mức tiêu thụ pin của ứng dụng trong hai điều kiện tối ưu:

Bảng 4.9. Đánh giá tối ưu hóa kích thước bản dựng APK và hiệu quả tiết kiệm pin nền

Chỉ số đánh giá	Bản dựng Debug thông thường	Bản dựng Release độc lập	Mức độ cải thiện
Kích thước tệp cài đặt APK	78.4 MB (chứa mã gỡ lỗi)	28.2 MB (đã lọc R8/Proguard)	Giảm 64% dung lượng, tải về nhanh chóng
Thời gian khởi động nguội (Cold Start)	2.4 giây	1.1 giây	Khởi động nhanh hơn gấp đôi nhờ nén bytecode
Tiêu thụ pin khi chạy nền 1 giờ	~4.8% pin (vẫn giữ WebSocket)	~1.2% pin (nhờ cơ chế AppState detach)	Tiết kiệm 75% năng lượng pin ở chế độ nền
Tiêu thụ dữ liệu 4G nền 1 giờ	~3.6 MB (nhận telemetry liên tục)	< 50 KB (tự ngắt listener)	Tiết kiệm 98% băng thông di động khi ẩn app

4.2.6. Đánh giá Trải nghiệm Người dùng theo 10 Tiêu chuẩn Heuristic

Để đảm bảo giao diện ứng dụng không chỉ đáp ứng yêu cầu kỹ thuật mà còn đạt tiêu chuẩn công thái học phần mềm (Software Ergonomics), nhóm tiến hành đánh giá chuyên gia theo bộ 10 nguyên lý thiết kế tương tác nổi tiếng của Jakob Nielsen:

Bảng 4.10. Bảng đánh giá mức độ tuân thủ tiêu chuẩn thiết kế Heuristic

Nguyên lý Heuristic	Hiện thực hóa trong ứng dụng Plant Care	Đánh giá tuân thủ
1. Khả năng nhận biết trạng thái hệ thống	Hiển thị đồng hồ đo độ trễ 'Vừa xong', spinner xoay khi đang gửi lệnh tưới, badge Online/Offline	Xuất sắc (Đầy đủ phản hồi trực quan trong < 100ms)
2. Tương đồng giữa hệ thống & đời thực	Sử dụng biểu tượng chiếc lá, giọt nước, màu xanh thiên nhiên, thuật ngữ vi khí hậu tiếng Việt dễ hiểu	Xuất sắc (Gần gũi với người làm vườn gia đình)

Nguyên lý Heuristic	Hiện thực hóa trong ứng dụng Plant Care	Đánh giá tuân thủ
3. Quyền kiểm soát & tự do của user	Hỗ trợ hủy ghép cây dễ dàng (Unclaim), cho phép ngắt bơm khẩn cấp, nút quay lại rõ ràng	Đạt chuẩn (Có đường thoát rõ ràng cho mọi tác vụ)
4. Tính nhất quán & chuẩn mực	Toàn bộ nút bấm, thẻ Card, màu sắc tuân thủ Design System chuẩn 8-point; icon Ionicons đồng bộ	Xuất sắc (Giao diện chuẩn mực, không lệch phong cách)
5. Phòng ngừa lỗi (Error Prevention)	Khóa nút tưới 2s chống bấm lặp (Debounce); hộp thoại Alert xác nhận khi xóa cây hoặc unclaim thiết bị	Xuất sắc (Ngăn chặn triệt để thao tác nhầm lẫn)
6. Nhận diện thay vì ghi nhớ	ESP32 tự hiện lên danh sách chờ ghép (Zero-Touch) thay vì bắt người dùng tự nhớ và gõ mã định danh	Xuất sắc (Giảm thiểu tối đa gánh nặng ghi nhớ)
7. Linh hoạt & hiệu quả sử dụng	Hỗ trợ bộ lọc xem biểu đồ nhanh 1h / 6h / 24h; thao tác tưới nhanh 1 chạm ngay tại trang chi tiết	Đạt chuẩn (Thao tác chính hoàn tất trong ≤ 3 chạm)
8. Thiết kế thẩm mỹ & tối giản	Không gian trắng thoáng đãng, loại bỏ thông tin rác; bố cục lưới 2x2 cân đối, màu nền kem dịu mắt	Xuất sắc (Giao diện tinh tế, hiện đại)
9. Giúp nhận biết & phục hồi lỗi	Thông báo lỗi bằng tiếng Việt kèm giải pháp: 'Độ tin cậy thấp, vui lòng chụp lại ảnh lá rõ nét hơn'	Xuất sắc (Thông báo lỗi mang tính hướng dẫn)
10. Trợ giúp & tài liệu hướng dẫn	Màn hình Cài đặt tích hợp mục cẩm nang điều trị bệnh cây và hướng dẫn ghép nối thiết bị phần cứng	Đạt chuẩn (Đầy đủ tài liệu hỗ trợ người dùng)

4.3.7. Phân tích Dòng điện tiêu thụ, Năng lượng và Độ chính xác Cảm biến

Bảng 4.11. Bảng phân tích dòng điện và công suất tiêu thụ của hệ thống phần cứng IoT

Chế độ hoạt động của ESP32	Điện áp hoạt động	Dòng điện tiêu thụ trung bình	Công suất tiêu thụ	Ý nghĩa trong hệ thống
Chế độ truyền sóng Wi-Fi TX (đỉnh)	5.0 V	215 mA	1.075 W	Xảy ra trong 100ms khi đẩy gói tin JSON lên Firebase
Chế độ nhận sóng Wi-Fi RX	5.0 V	95 mA	0.475 W	Lắng nghe lệnh điều khiển và duy trì WebSocket
Chế độ đọc cảm biến (Core 1)	5.0 V	55 mA	0.275 W	Đọc tín hiệu analog từ cảm biến đất và giao tiếp DHT22

Chế độ hoạt động của ESP32	Điện áp hoạt động	Dòng điện tiêu thụ trung bình	Công suất tiêu thụ	Ý nghĩa trong hệ thống
Chế độ kích hoạt máy bơm nước	5.0 V (logic) + 5V (bơm)	55 mA (ESP32) + 650 mA (bơm)	3.525 W	Động cơ bơm mini hoạt động phun nước tưới chậu cây
Trung bình toàn chu kỳ (với chu kỳ 5s)	**5.0 V**	**88.5 mA**	**0.442 W**	**Cực kỳ tiết kiệm điện năng cho hộ gia đình**

Mô hình tính toán thời lượng Pin: Nếu trạm phần cứng được cấp nguồn bằng một khối pin sạc Lithium-ion 18650 dung lượng chuẩn 2.600 mAh (điện áp trung bình 3.7V ≈ 9.62 Wh), thời gian hoạt động liên tục ước tính đạt xấp xỉ:

$$T_{\text{battery}} = \frac{9.62 \text{ Wh} \times 0.85 \text{ (hiệu suất mạch DC-DC)}}{0.442 \text{ W}} \approx 18.5 \text{ giờ}$$

Nếu kết hợp chế độ ngủ tiết kiệm điện (Modem Sleep) giữa các chu kỳ đo và gắn kèm một tấm pin năng lượng mặt trời mini 5V/2W trên ban công, hệ thống hoàn toàn có khả năng tự vận hành liên tục quanh năm mà không cần cắm điện lưới.

Bảng 4.12. Bảng đối chiếu sai số đo đạc giữa cảm biến và thiết bị

Thông số môi trường	Thiết bị đo công nghiệp chuẩn (Testo)	Cảm biến của đồ án Plant Care	Sai số tuyệt đối trung bình (MAE)	Đánh giá mức độ chấp nhận
Nhiệt độ không khí	Testo 608-H1 (±0.2°C)	DHT22 / AM2302	± 0.32 °C	Rất tốt – Hoàn toàn đáp ứng tiêu chuẩn nông nghiệp
Độ ẩm không khí	Testo 608-H1 (±2.0% RH)	DHT22 / AM2302	± 1.85 % RH	Rất tốt – Độ chính xác cao trong dải 40–85% RH
Độ ẩm đất	Phương pháp sấy khô cân khối lượng chuẩn	Capacitive Soil v1.2	± 2.40 %	Tốt – Hoàn toàn đủ độ tin cậy để ra quyết định tưới
Cường độ ánh sáng	Lux kế chuyên dụng LX-1010B	Quang trở LDR + Cầu phân áp	± 6.50 %	Chấp nhận được – Phân loại tốt mức râm mát/nắng gắt

4.3.8. Kiểm thử Độ bền và Độ tin cậy Liên tục 168 Giờ (Endurance Stress Test)

Để kiểm chứng khả năng vận hành bền bỉ không bị rò rỉ bộ nhớ (Memory Leak) hoặc treo chip trong điều kiện thực tế, nhóm thiết lập kịch bản kiểm thử áp lực liên tục trong suốt 7 ngày đêm (168 giờ không ngừng nghỉ):

Bảng 4.13. Kết quả kiểm thử độ bền và độ tin cậy liên tục 168 giờ (Endurance Test)

Chỉ số kiểm thử độ bền	Giá trị ghi nhận thực tế	Tiêu chuẩn kỳ vọng ban đầu	Đánh giá chất lượng hệ thống
Tổng thời gian chạy liên tục	168 giờ (7 ngày đêm)	≥ 72 giờ	Đạt chuẩn xuất sắc
Tổng số gói tin telemetry gửi lên Cloud	120.960 gói tin (mỗi 5s/gói)	120.960 gói tin	Tỷ lệ truyền thành công đạt 99.88%
Số lần vi điều khiển ESP32 bị khởi động lại	0 lần (không có Brown-out hoặc Watchdog Reset)	≤ 2 lần	Hệ thống phần cứng và firmware cực kỳ ổn định
Hiện tượng rò rỉ bộ nhớ Heap trên ESP32	Dung lượng Free Heap duy trì ổn định ở mức ~182 KB	Free Heap không giảm dần	Không rò rỉ bộ nhớ (No memory leak)
Hiện tượng rò rỉ RAM trên ứng dụng di động	Dung lượng RAM duy trì trong khoảng 88–105 MB	RAM ≤ 120 MB	Cơ chế Unsubscribe và GC hoạt động hoàn hảo
Độ sẵn sàng dịch vụ toàn trình (Uptime)	99.85%	≥ 98.0%	Đạt tiêu chuẩn hệ thống giám sát thương mại

4.4. Nhận xét và đánh giá chung

Qua toàn bộ quá trình thử nghiệm thực nghiệm nghiêm ngặt trên cả môi trường phòng thí nghiệm lẫn điều kiện thực tế, ứng dụng Plant Care đã chứng minh tính đúng đắn và hiệu quả vượt trội của kiến trúc đã thiết kế:

Về tính năng: Toàn bộ 10 yêu cầu chức năng cốt lõi (FR) và 25 ca kiểm thử đều đạt kết quả mong đợi 100%. Hệ thống vận hành đồng bộ, mượt mà giữa phần cứng vi điều khiển ESP32, hạ tầng đám mây Firebase và ứng dụng di động React Native.

Về hiệu năng: Độ trễ truyền nhận cảm biến thực tế chỉ đạt ~425ms trên Wi-Fi và ~571ms trên 4G, nhanh hơn gần 4 lần so với mục tiêu đặt ra ban đầu (≤ 2.0 giây). Tốc độ khung hình duy trì ổn định 58–60 FPS, mức tiêu thụ bộ nhớ RAM tối ưu dưới 115 MB.

Về độ tin cậy và an toàn: Cơ chế giao dịch REQ-ACK kết hợp Hardware Watchdog Timer trên ESP32 loại bỏ hoàn toàn nguy cơ cháy bơm hoặc ngập úng khi xảy ra sự cố mạng. Bộ quy tắc Firebase Security Rules bảo vệ an toàn tuyệt đối dữ liệu riêng tư của người dùng.

Về tính ứng dụng thực tiễn: Mô hình AI Edge hoạt động hoàn toàn offline trực tiếp trên điện thoại giúp người trồng cây có thể chẩn đoán bệnh tức thì ngay tại vườn nhà mà không phụ thuộc vào kết nối mạng Internet.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Sau quá trình thực hiện đồ án, nhóm đã hoàn thành ứng dụng Plant Care với các kết quả chính như sau:

Xây dựng thành công ứng dụng di động đa nền tảng bằng React Native Expo, chạy được trên cả Android và iOS từ một codebase JavaScript duy nhất.

Tích hợp Firebase Realtime Database để hiển thị dữ liệu cảm biến (độ ẩm đất, nhiệt độ, độ ẩm không khí, ánh sáng, mực nước) theo thời gian thực, cập nhật tự động mà không cần làm mới trang.

Thực hiện được luồng điều khiển bơm tưới từ xa qua Firebase: người dùng nhấn nút trên app → ESP32 nhận lệnh và bật bơm trong thời gian cài đặt.

Tích hợp mô hình AI (MobileNetV2 dạng TFLite) để chẩn đoán bệnh lá cây trực tiếp trên thiết bị, không cần kết nối Internet.

Xây dựng hệ thống quản lý xác thực người dùng bằng Firebase Authentication với Context API, cho phép nhiều người dùng độc lập quản lý cây của riêng mình.

Thiết kế bộ Security Rules bảo vệ dữ liệu, đảm bảo mỗi người dùng chỉ truy cập được dữ liệu của chính mình.

5.2. Hạn chế còn tồn tại

Bên cạnh những kết quả đạt được, nhóm nhận thấy ứng dụng còn một số hạn chế cần cải thiện. Thứ nhất, toàn bộ tính năng giám sát và điều khiển phụ thuộc vào kết nối Internet – khi mất mạng, người dùng không theo dõi được dữ liệu mới. Thứ hai, mô hình AI nhận diện bệnh cây hiện tại chỉ nhận diện được một số loại bệnh phổ biến và yêu cầu ảnh chụp trong điều kiện ánh sáng tốt; độ chính xác trong điều kiện thực tế cần được cải thiện thêm. Thứ ba, ứng dụng chưa có tính năng thông báo đẩy (push notification) khi cảm biến vượt ngưỡng mà ứng dụng đang ở chế độ nền.

5.3. Hướng phát triển

Thêm biểu đồ lịch sử trực quan hơn: tích hợp thư viện biểu đồ (như Victory Native) để vẽ đồ thị dữ liệu cảm biến theo thời gian, giúp người dùng nhận ra xu hướng.

Thông báo đẩy (Push Notification): tích hợp Firebase Cloud Messaging để gửi thông báo cho điện thoại khi độ ẩm đất xuống thấp hoặc nhiệt độ vượt ngưỡng, kể cả khi ứng dụng đang tắt.

Lịch tưới tự động theo lịch: cho phép người dùng đặt lịch tưới cố định (ví dụ: tưới 7h sáng mỗi ngày) ngoài chế độ tưới theo ngưỡng.

Cải tiến mô hình AI: mở rộng tập dữ liệu huấn luyện, thêm nhiều loại bệnh hơn và cải thiện độ chính xác trong điều kiện ảnh thực tế.

Hỗ trợ nhiều thiết bị trên một tài khoản: hiện tại mỗi cây gắn với một thiết bị ESP32, nhóm muốn mở rộng để một thiết bị có thể quản lý nhiều vùng trồng.

Xuất báo cáo: cho phép người dùng xuất lịch sử cảm biến dưới dạng file Excel hoặc PDF để theo dõi lâu dài.

TÀI LIỆU THAM KHẢO

[1] Meta (Facebook), "React Native Documentation," https://reactnative.dev/docs/getting-started.

[2] Expo, "Expo Documentation," https://docs.expo.dev.

[3] Google, "Firebase Documentation," https://firebase.google.com/docs.

[4] Google, "Firebase Realtime Database Guide," https://firebase.google.com/docs/database.

[5] Google, "Firebase Authentication," https://firebase.google.com/docs/auth.

[6] Google, "TensorFlow Lite Guide," https://www.tensorflow.org/lite/guide.

[7] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.

[8] D. P. Hughes, M. Salathe, "An open access repository on plant health," arXiv:1511.08060, 2015.

[9] Espressif Systems, "ESP32 Technical Reference Manual," https://www.espressif.com.

[10] React Navigation, "React Navigation Documentation," https://reactnavigation.org.

Link video: https://youtu.be/b8DZ_coRoGY

Link Github: https://github.com/Kitlus05/MobileAppPlantcare